MOCCA 2025
Fundamentals of Character Animation and Motion Control

Lecture 08

Actuating Simulated Characters

Libin Liu

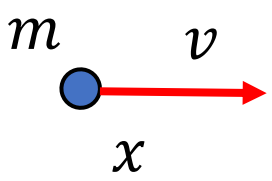
School of Intelligence Science and Technology
Peking University

VCL @ PKU

Outline

- Simulating & Actuating Characters
  - Joint torques
- PD (Proportional-Derivative) control
  - Stable PD control
- Feedforward Motion Control
  - Residual force
- Feedback Motion Control
  - Static balance

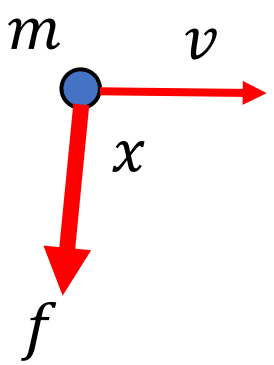
Recap: Dynamics of a Point Mass



x, v

$p = mv$

Recap: Dynamics of a Point Mass

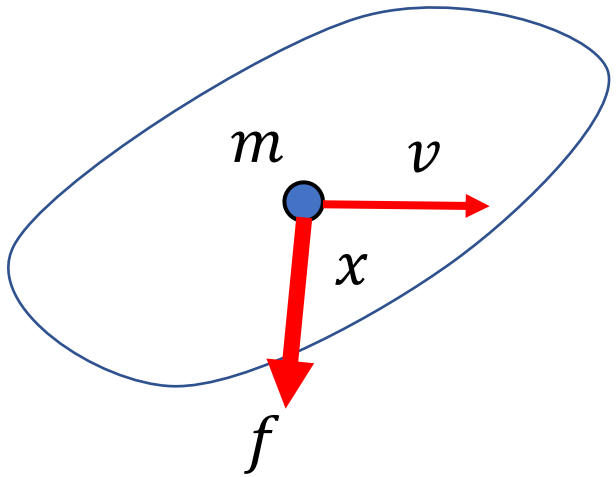


x, v

$p = mv$

Newton's Second Law: $\frac{dp}{dt} = f \Rightarrow m\dot{v} = f$

Recap: Rigid Body Dynamics



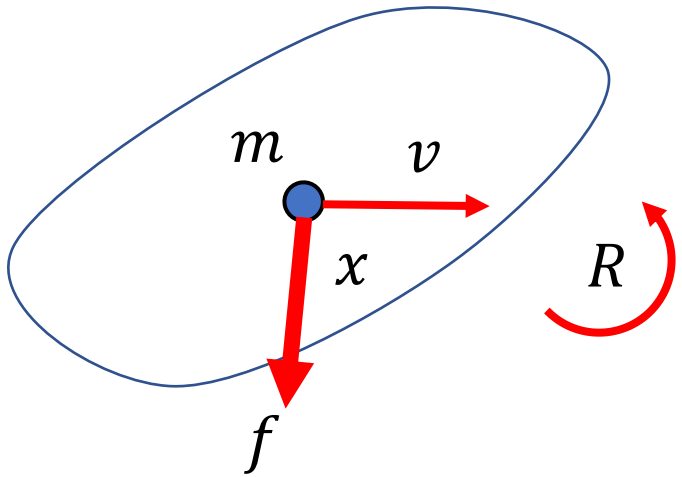
Linear

x, v

$p = mv$

Newton's Second Law: $\frac{dp}{dt} = f$ ➡ $m\dot{v} = f$

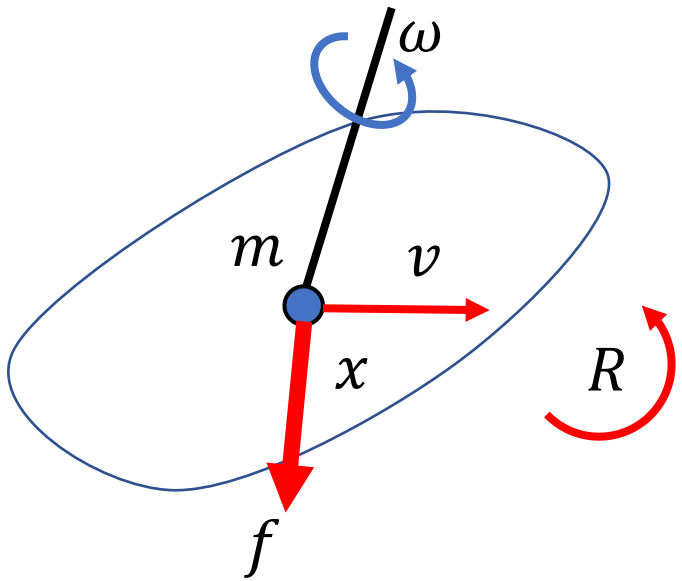
Recap: Rigid Body Dynamics



Linear	Angular
x, v	R
$p = mv$	

Newton's Second Law: $\frac{dp}{dt} = f \Rightarrow m\dot{v} = f$

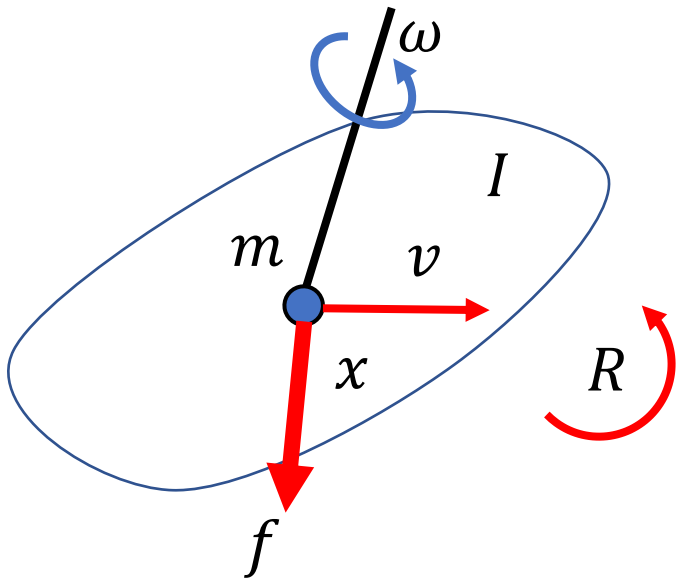
Recap: Rigid Body Dynamics



Linear	Angular
x, v	R, ω
$p = mv$	

Newton's Second Law: $\frac{dp}{dt} = f \Rightarrow m\dot{v} = f$

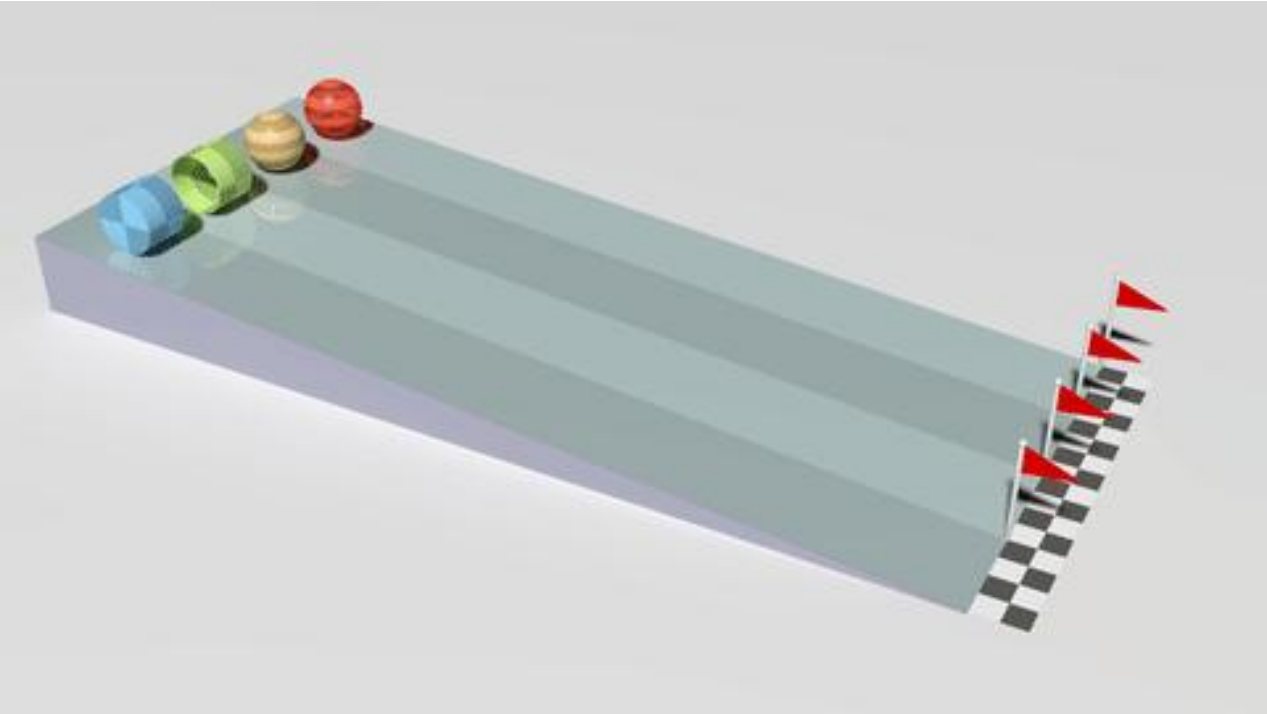
Recap: Rigid Body Dynamics



Linear	Angular
x, v	R, ω
$p = mv$	$L = I\omega$

Newton's Second Law: $\frac{dp}{dt} = f \Rightarrow m\dot{v} = f$

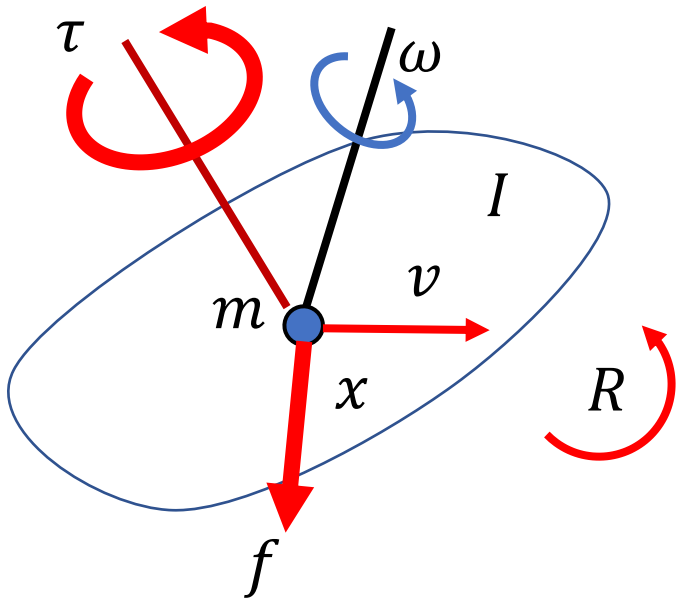
Recap: Moment of Inertia



Same mass, different shapes

Different Moments of Inertia

Recap: Rigid Body Dynamics

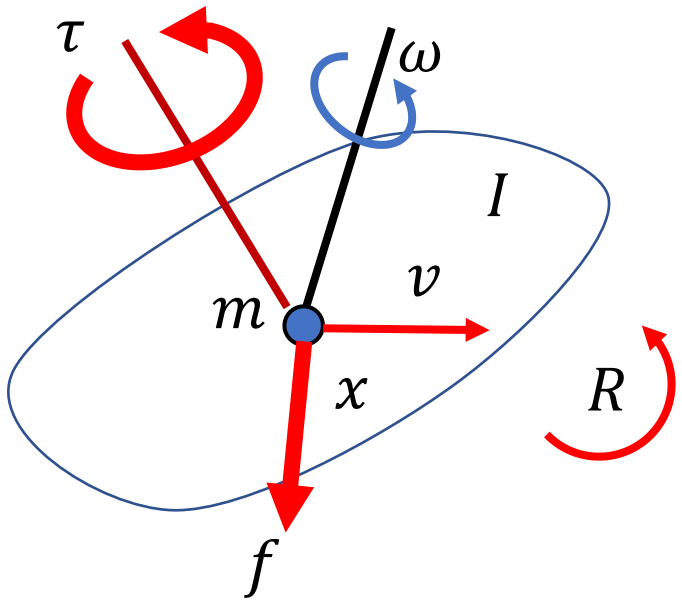


Linear	Angular
x, v	R, ω
$p = mv$	$L = I\omega$

Newton's Second Law: $\frac{dp}{dt} = f$ ➡ $m\dot{v} = f$

Euler's laws of motion: $\frac{dL}{dt} = \tau$ ➡ $I\dot{\omega} + \omega \times I\omega = \tau$

Recap: Rigid Body Dynamics



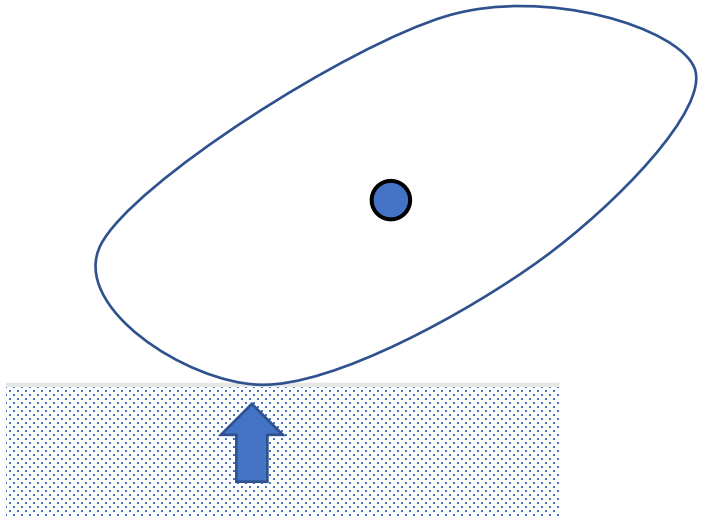
Linear	Angular
x, v	R, ω
$p = mv$	$L = I\omega$

Newton's Second Law:

Euler's laws of motion:

$$\begin{bmatrix} m\mathrm{I}_3 & 0 \\ 0 & I \end{bmatrix} \begin{bmatrix} \dot{v} \\ \dot{\omega} \end{bmatrix} + \begin{bmatrix} 0 \\ \omega \times I\omega \end{bmatrix} = \begin{bmatrix} f \\ \tau \end{bmatrix}$$

Defining a Rigid Body



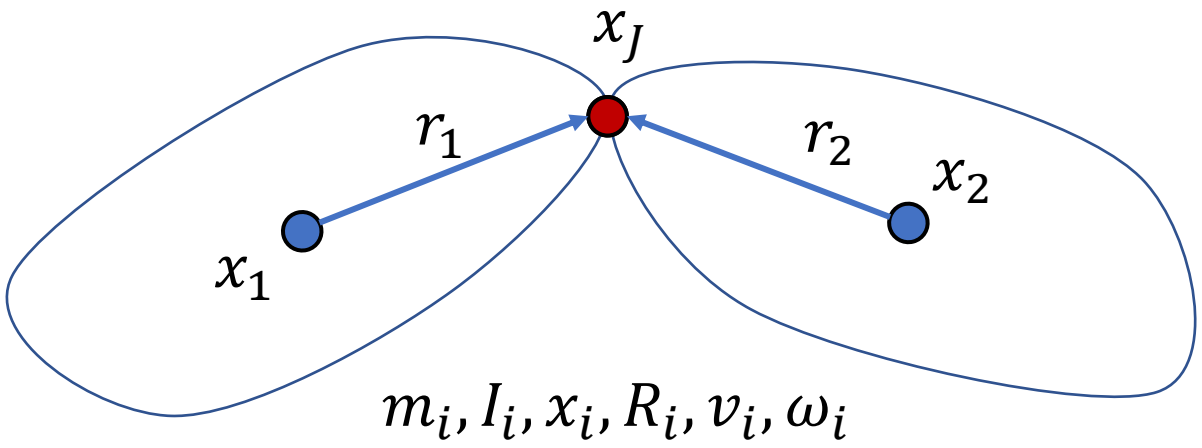
Masses: m, I

Kinematics: $\boldsymbol{x}, \boldsymbol{v}, R, \boldsymbol{\omega}$

Geometry:

- Box, Sphere, Capsule, Mesh, …
- Collision detection
- Compute m, I

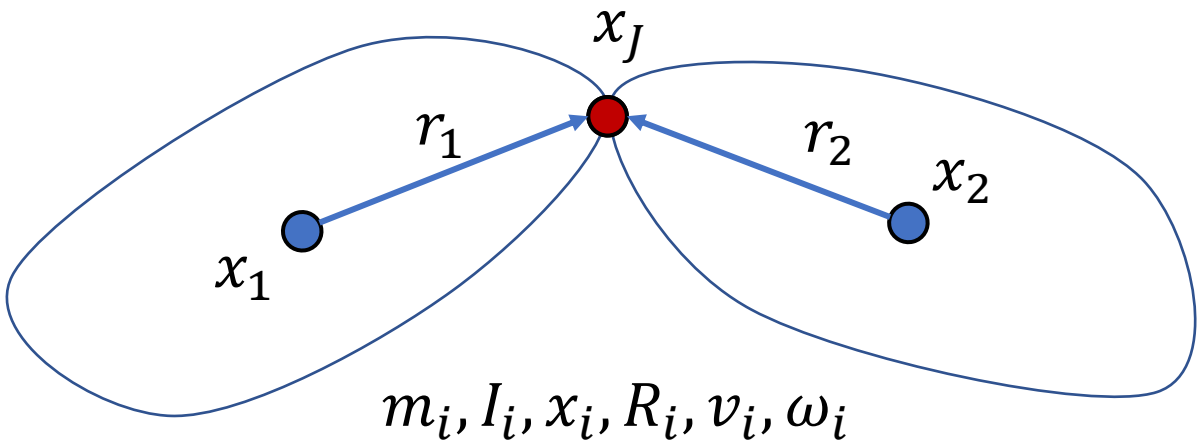
Recap: Dynamics of Articulated Rigid Bodies



$$\boxed{Jv = 0}$$

$$\begin{bmatrix} I_3 & -[r_1]_\times & -I_3 & [r_2]_\times \end{bmatrix} \begin{bmatrix} v_1 \\ w_1 \\ v_2 \\ w_2 \end{bmatrix} = 0$$

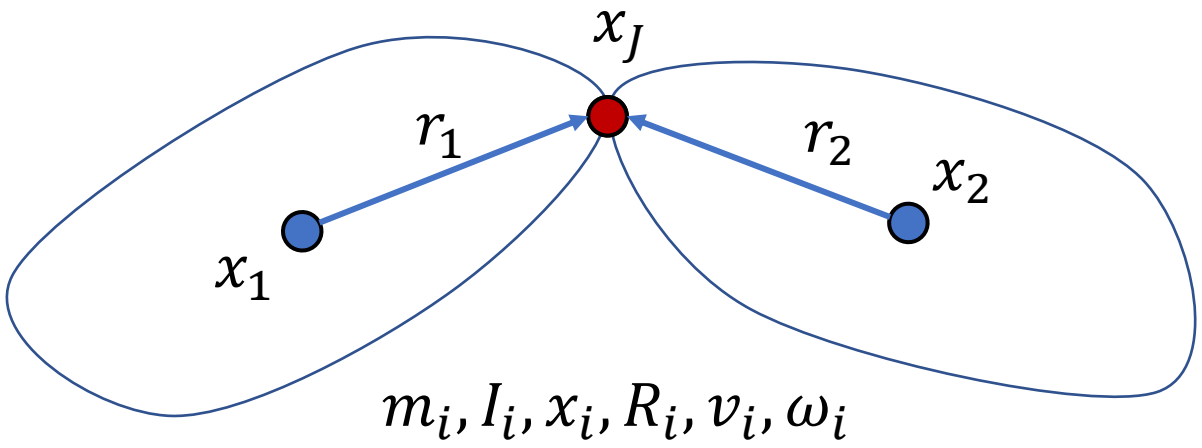
Recap: Dynamics of Articulated Rigid Bodies



$$\begin{bmatrix} m_1 \mathrm{I}_3 & & & \\ & I_1 & & \\ & & m_2 \mathrm{I}_3 & \\ & & & I_2 \end{bmatrix} \begin{bmatrix} \dot{v}_1 \\ \dot{\omega}_1 \\ \dot{v}_2 \\ \dot{\omega}_2 \end{bmatrix} + \begin{bmatrix} 0 \\ \omega_1 \times I_1 \omega_1 \\ 0 \\ \omega_2 \times I_2 \omega_2 \end{bmatrix} = \begin{bmatrix} f_1 \\ \tau_1 \\ f_2 \\ \tau_2 \end{bmatrix} + \begin{bmatrix} I_3 \\ [r_1]_\times \\ -I_3 \\ -[r_2]_\times \end{bmatrix} \lambda$$

$$Jv = 0$$

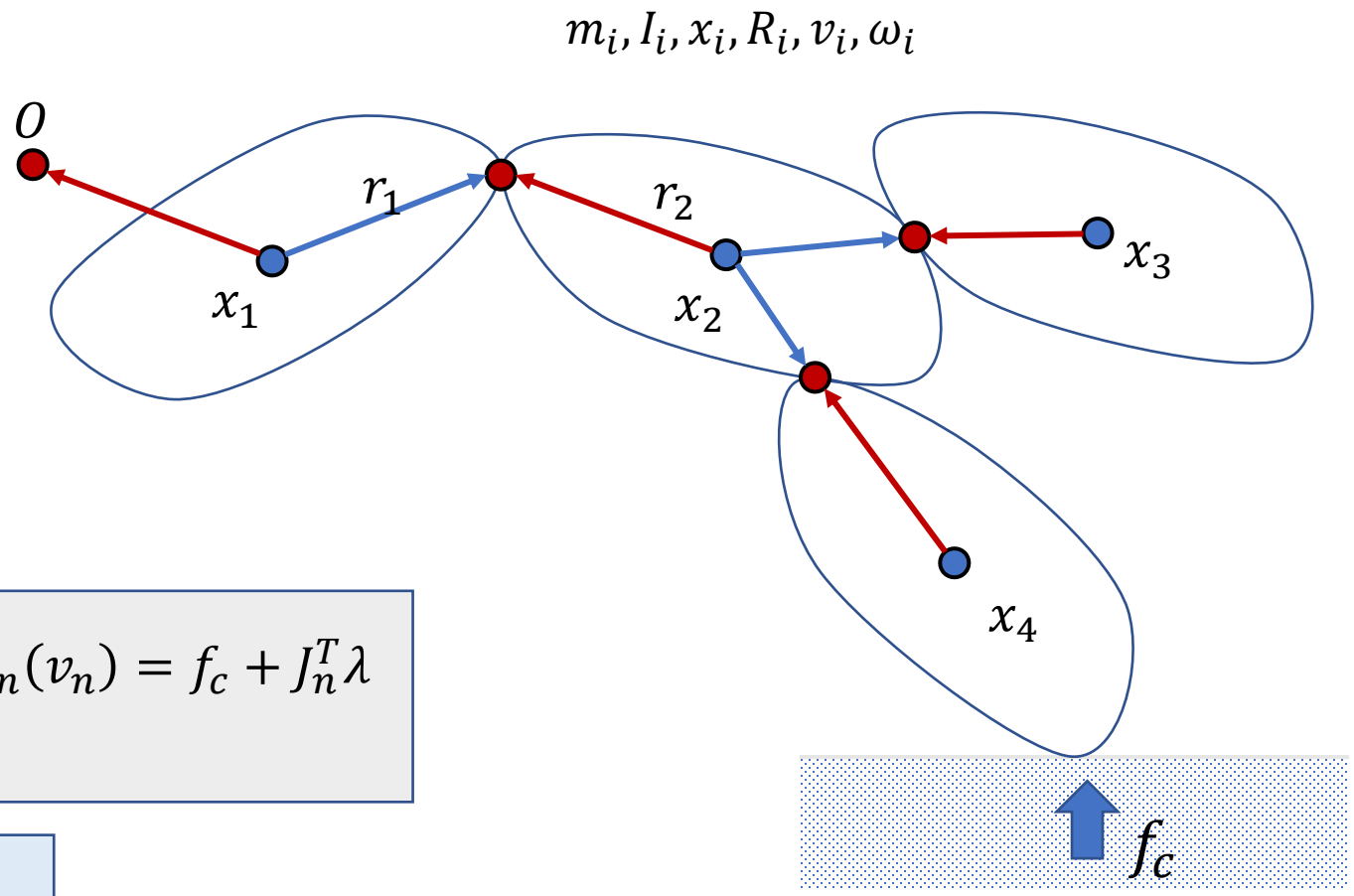
Recap: Dynamics of Articulated Rigid Bodies



$$M\dot{\boldsymbol{v}} + C(\boldsymbol{x}, \boldsymbol{v}) = \boldsymbol{f} + \boldsymbol{J}^{\boldsymbol{T}}\boldsymbol{\lambda}$$

$$Jv = 0$$

Recap: Simulation of a Rigid Body System



$$I_n = R_n I_0 R_n^T$$

$$M_n(v_{n+1} - v_n)/_h + C_n(v_n) = f_c + J_n^T \lambda$$

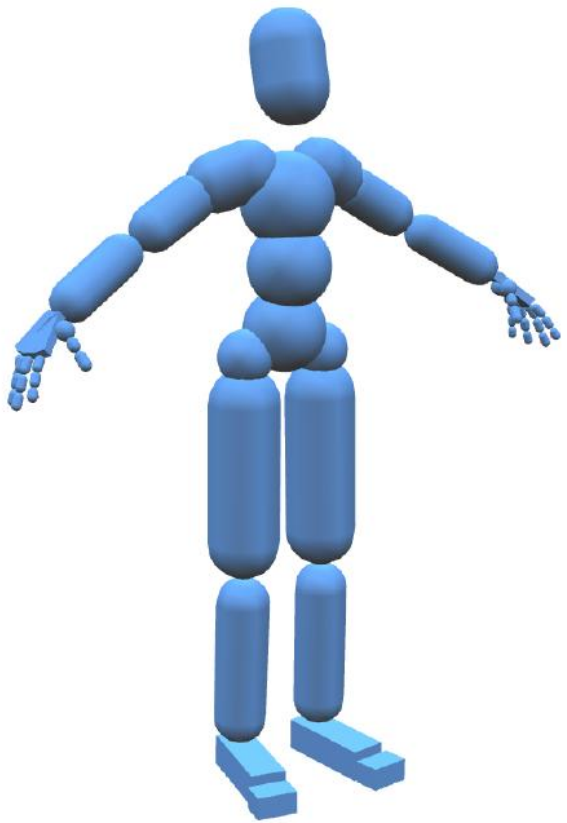
$$J_n v_{n+1} = c_n$$

$$x_{n+1} = x_n + h v_{n+1}$$

$$q_{n+1} = q_n + \frac{h}{2} \bar{\omega}_{n+1} q$$

Defining a Simulated Character

[Liu et al 2018]



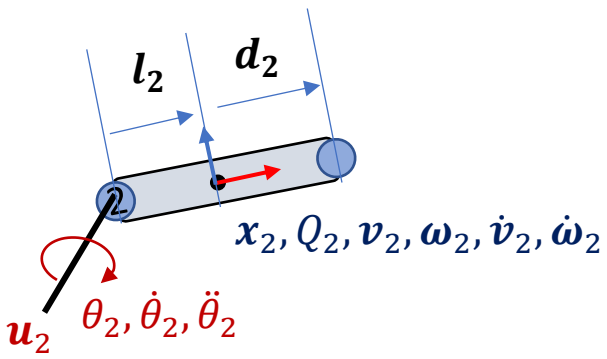
Rigid bodies:

- $m_i, I_i, \boldsymbol{x}_i, R_i$
- Geometries

Joints:

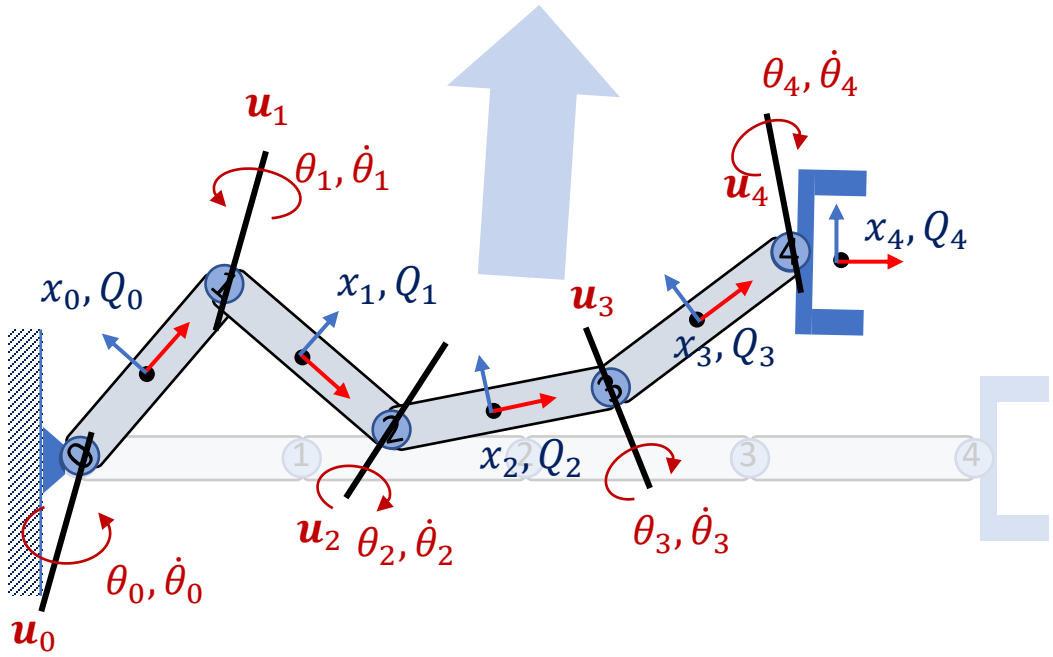
- Position
- Type
- Bodies

Maximized Coordinates & Generalized Coordinates



Maximized Coordinates:

$$\boldsymbol{x}_i, Q_i \qquad \boldsymbol{v}_i, \boldsymbol{\omega}_i \qquad \dot{\boldsymbol{v}}_i, \dot{\boldsymbol{\omega}}_i$$

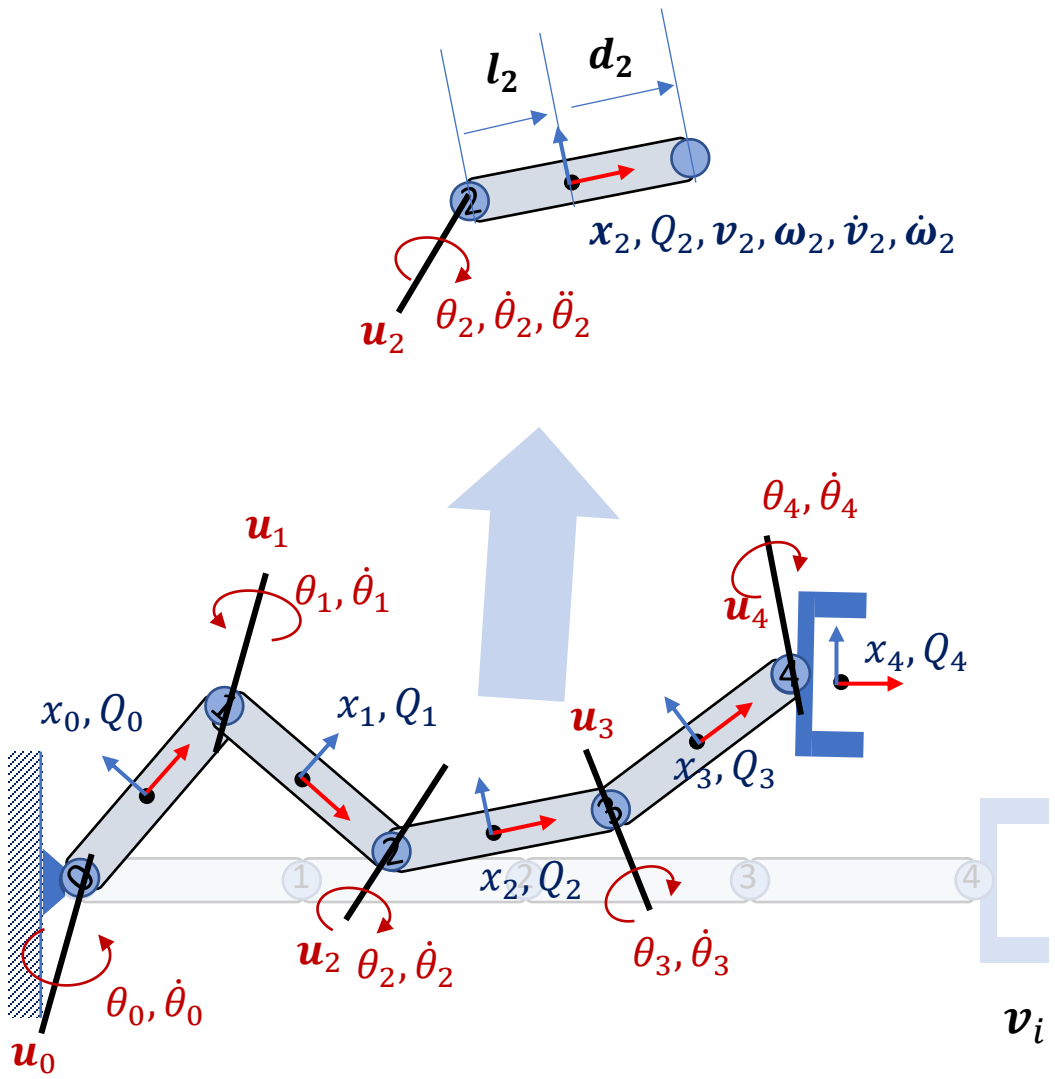


Generalized Coordinates:

$$\boldsymbol{\theta} = (\theta_0, \theta_1, \dots)$$

$$\boldsymbol{\theta}, \dot{\boldsymbol{\theta}}, \ddot{\boldsymbol{\theta}}$$

Maximized Coordinates & Generalized Coordinates



Generalized Coordinates

$$\boldsymbol{\theta}, \dot{\boldsymbol{\theta}}, \ddot{\boldsymbol{\theta}}$$

Maximized Coordinates:

$$\boldsymbol{x}_i, Q_i \quad \boldsymbol{v}_i, \boldsymbol{\omega}_i \quad \dot{\boldsymbol{v}}_i, \dot{\boldsymbol{\omega}}_i$$

Forward kinematics:

$$Q_i = \prod_{j=0}^{i} R(\theta_j \boldsymbol{u}_j)$$

$$Q_0 = R(\theta_0 \boldsymbol{u}_0)$$

$$Q_1 = R(\theta_0 \boldsymbol{u}_0) R(\theta_1 \boldsymbol{u}_1)$$

$$\boldsymbol{x}_i = \sum_{j=0}^{i-1} Q_j(\boldsymbol{l}_j + \boldsymbol{d}_j) + Q_i \boldsymbol{l}_i$$

$$\boldsymbol{x}_0 = Q_0 \boldsymbol{l}_0$$

$$\boldsymbol{x}_1 = Q_0(\boldsymbol{l}_0 + \boldsymbol{d}_0) + Q_1 \boldsymbol{l}_1$$

$$\boldsymbol{\omega}_i = \sum_{j=0}^{i} Q_{j-1} \dot{\theta}_j \boldsymbol{u}_j$$

$$\boldsymbol{\omega}_0 = \dot{\theta}_0 \boldsymbol{u}_0$$

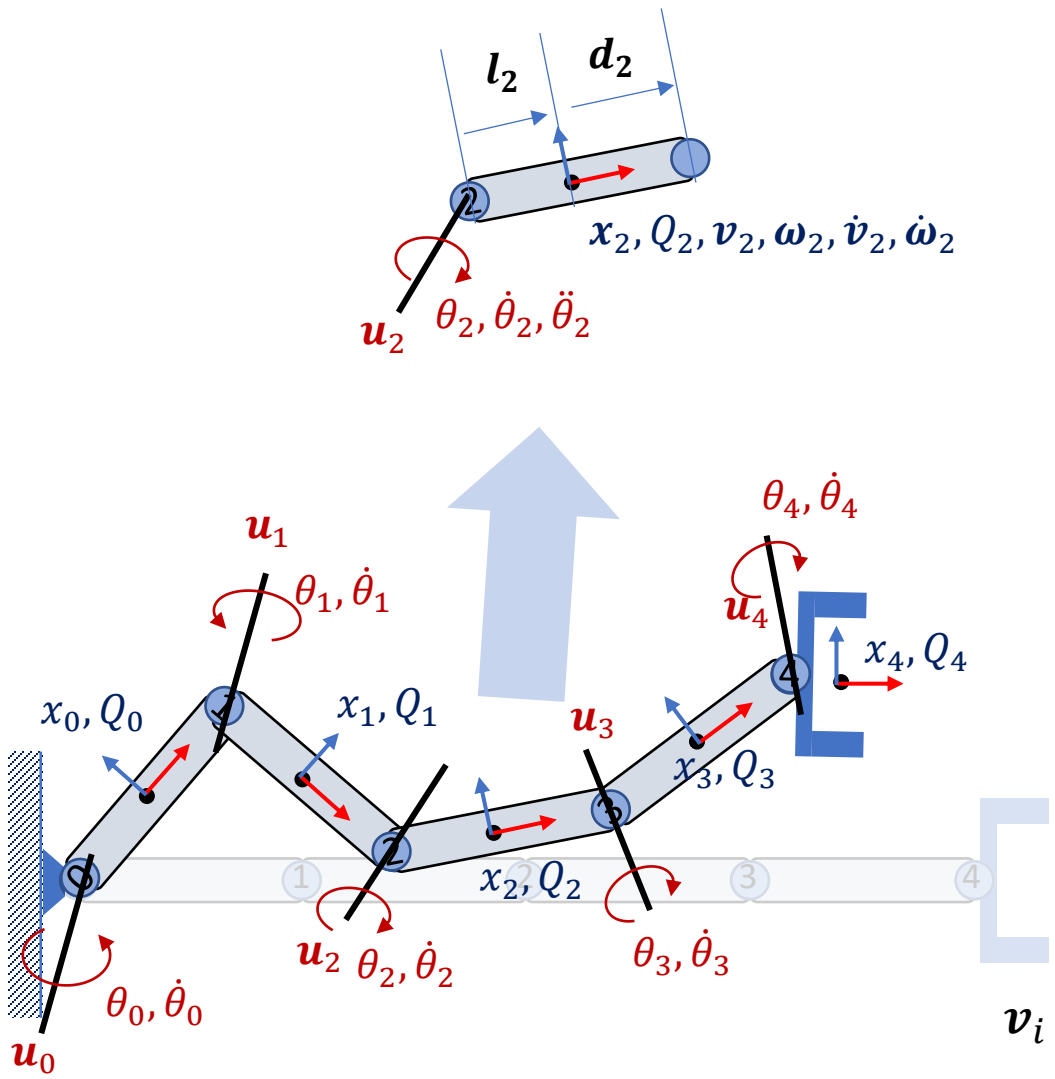
$$\boldsymbol{\omega}_1 = \dot{\theta}_0 \boldsymbol{u}_0 + Q_0 \dot{\theta}_1 \boldsymbol{u}_1$$

$$\boldsymbol{v}_i = \sum_{j=0}^{i-1} \boldsymbol{\omega}_j \times \left(Q_j(\boldsymbol{l}_j + \boldsymbol{d}_j)\right) + \boldsymbol{\omega}_i \times \boldsymbol{l}_i$$

$$\boldsymbol{v}_0 = \boldsymbol{\omega}_0 \times \boldsymbol{l}_0$$

$$\boldsymbol{v}_1 = \boldsymbol{\omega}_0 \times \left(Q_0(\boldsymbol{l}_0 + \boldsymbol{d}_0)\right) + \boldsymbol{\omega}_1 \times Q_1 \boldsymbol{l}_1$$

Maximized Coordinates & Generalized Coordinates



Generalized Coordinates

$$\boldsymbol{\theta}, \dot{\boldsymbol{\theta}}, \ddot{\boldsymbol{\theta}}$$

Maximized Coordinates:

$$\boldsymbol{x}_i, Q_i \quad \boldsymbol{v}_i, \boldsymbol{\omega}_i \quad \dot{\boldsymbol{v}}_i, \dot{\boldsymbol{\omega}}_i$$

Forward kinematics:

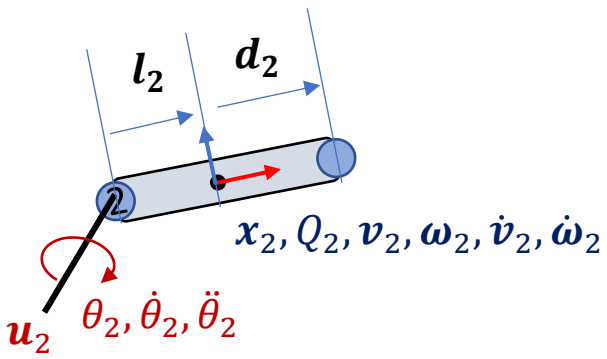
$$Q_i = \prod_{j=0}^{i} R(\theta_j \boldsymbol{u}_j) \qquad Q_i = Q_{i-1} R(\theta_i \boldsymbol{u}_i)$$

$$\boldsymbol{x}_i = \sum_{j=0}^{i-1} Q_j(\boldsymbol{l}_j + \boldsymbol{d}_j) + Q_i \boldsymbol{l}_i \qquad \boldsymbol{x}_i = \boldsymbol{x}_{i-1} + Q_{i-1}\boldsymbol{d}_{i-1} + Q_i \boldsymbol{l}_i$$

$$\boldsymbol{\omega}_i = \sum_{j=0}^{i} Q_{j-1} \dot{\theta}_j \boldsymbol{u}_j \qquad \boldsymbol{\omega}_i = \boldsymbol{\omega}_{i-1} + Q_{i-1} \dot{\theta}_i \boldsymbol{u}_i$$

$$\boldsymbol{v}_i = \sum_{j=0}^{i-1} \boldsymbol{\omega}_j \times \left(Q_j(\boldsymbol{l}_j + \boldsymbol{d}_j)\right) + \boldsymbol{\omega}_i \times \boldsymbol{l}_i \qquad \boldsymbol{v}_i = \boldsymbol{v}_{i-1} + \boldsymbol{\omega}_{i-1} \times (Q_{i-1}\boldsymbol{d}_{i-1}) + \boldsymbol{\omega}_i \times (Q_i \boldsymbol{l}_i)$$

Maximized Coordinates & Generalized Coordinates



Generalized Coordinates

$$\boldsymbol{\theta}, \dot{\boldsymbol{\theta}}, \ddot{\boldsymbol{\theta}}$$

Maximized Coordinates:

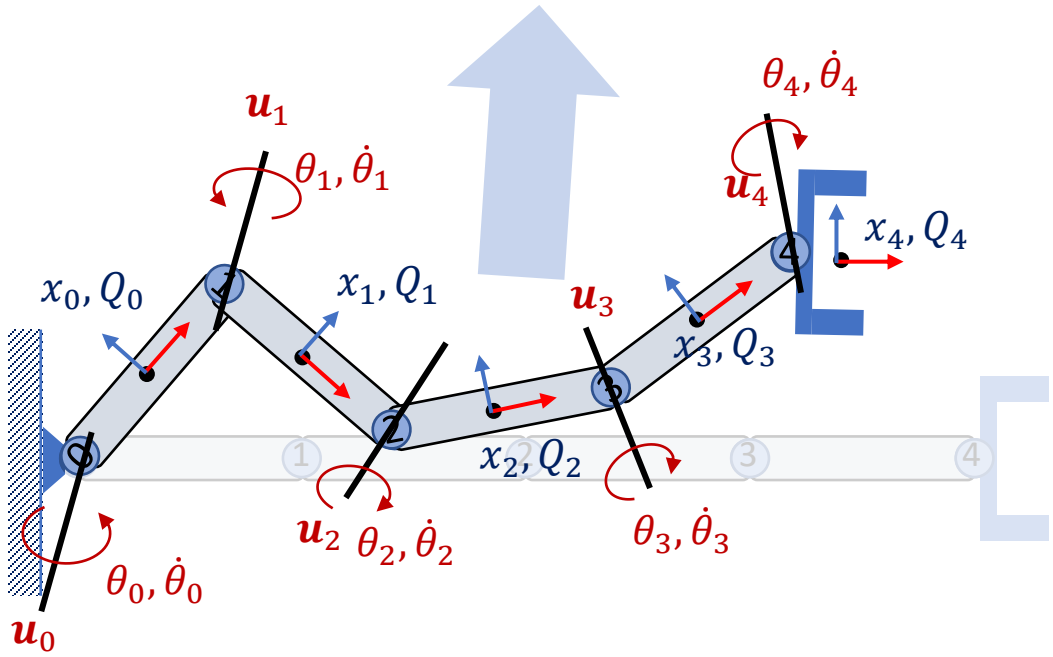
$$\boldsymbol{x}_i, Q_i \quad \boldsymbol{v}_i, \boldsymbol{\omega}_i \quad \dot{\boldsymbol{v}}_i, \dot{\boldsymbol{\omega}}_i$$

Forward kinematics:

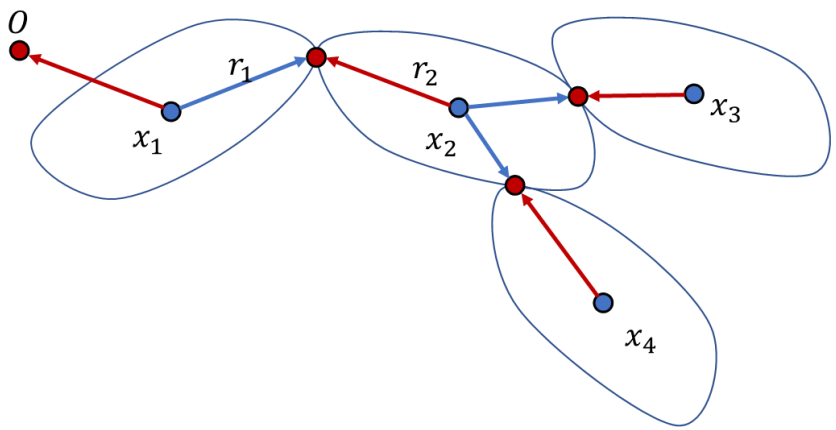
$$\dot{\boldsymbol{\omega}}_i = \dot{\boldsymbol{\omega}}_{i-1} + \boldsymbol{\omega}_i \times \left(Q_i \dot{\theta}_i \boldsymbol{u}_i\right) + Q_i \ddot{\theta}_i \boldsymbol{u}_i$$

$$\dot{\boldsymbol{v}}_i = \dot{\boldsymbol{v}}_{i-1} + \dot{\boldsymbol{\omega}}_{i-1} \times (Q_{i-1}\boldsymbol{d}_{i-1}) + \boldsymbol{\omega}_{i-1} \times \left(\boldsymbol{\omega}_{i-1} \times (Q_{i-1}\boldsymbol{d}_{i-1})\right)$$

$$+\dot{\boldsymbol{\omega}}_i \times (Q_i \boldsymbol{l}_i) + \boldsymbol{\omega}_i \times \left(\boldsymbol{\omega}_i \times (Q_i \boldsymbol{l}_i)\right)$$



Maximized Coordinates & Generalized Coordinates



Maximized Coordinates:

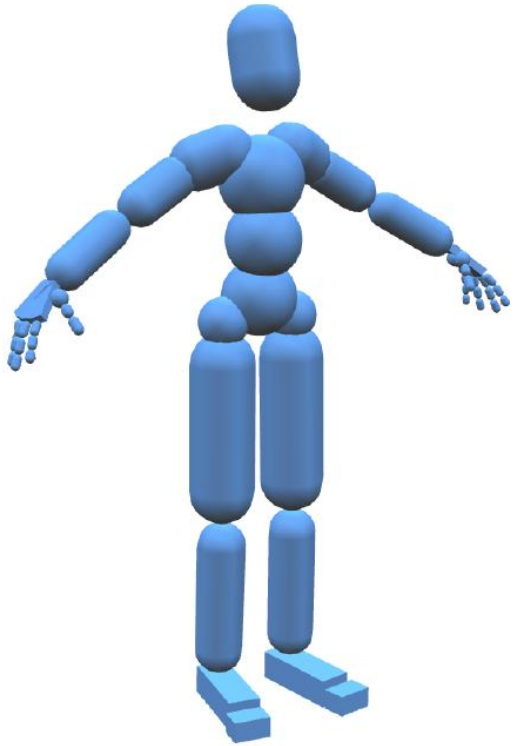
$$\boldsymbol{x}_i, R_i \qquad \boldsymbol{v}_i, \boldsymbol{\omega}_i \qquad \dot{\boldsymbol{v}}_i, \dot{\boldsymbol{\omega}}_i$$

Generalized Coordinates:

$$\boldsymbol{\theta} = (\boldsymbol{t}_0, \boldsymbol{\theta}_0, \boldsymbol{\theta}_1, \boldsymbol{\theta}_2, \dots\dots)$$

$$\boldsymbol{\theta}, \dot{\boldsymbol{\theta}}, \ddot{\boldsymbol{\theta}}$$

Character Kinematics



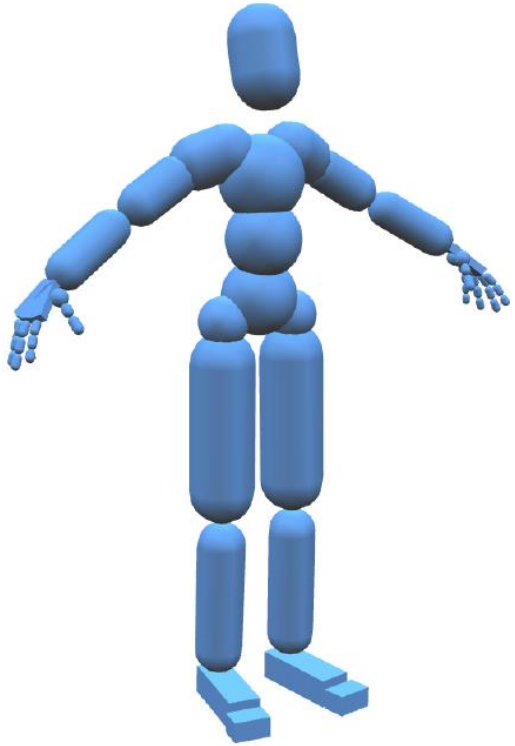
Forward Kinematics

$$\boldsymbol{\theta} = (\boldsymbol{t}_0, \boldsymbol{\theta}_0, \boldsymbol{\theta}_1, \boldsymbol{\theta}_2, \dots\dots) \longrightarrow \boldsymbol{x}_i, R_i$$

$$\dot{\boldsymbol{\theta}} = (v_0, \omega_0, \omega_1, \omega_2, \dots\dots) \longrightarrow \boldsymbol{v}_i, \boldsymbol{\omega}_i$$

$$\ddot{\boldsymbol{\theta}} = (\dot{v}_0, \dot{\omega}_0, \dot{\omega}_1, \dot{\omega}_2, \dots\dots) \longrightarrow \dot{\boldsymbol{v}}_i, \dot{\boldsymbol{\omega}}_i$$

Simulating a Character

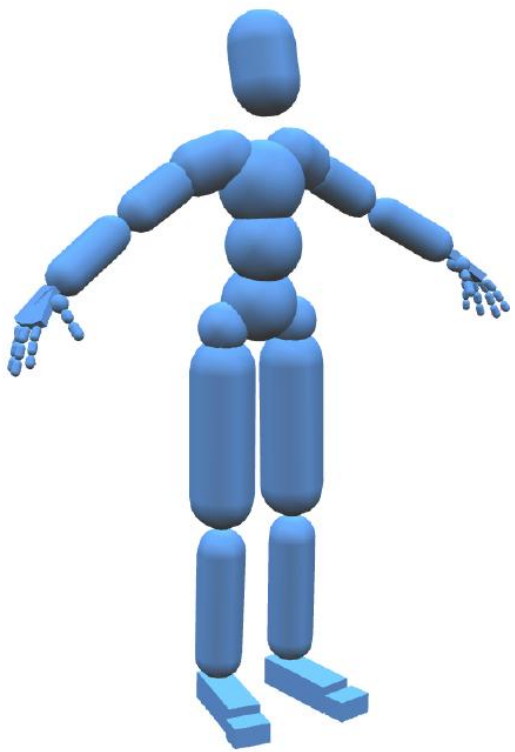


Current state:
$\boldsymbol{x}_t, \boldsymbol{v}_t, R_t, \boldsymbol{\omega}_t$

Dynamic Simulator

Next state:
$\boldsymbol{x}_{t+1}, \boldsymbol{v}_{t+1}, R_{t+1}, \boldsymbol{\omega}_{t+1}$

Simulating a Character

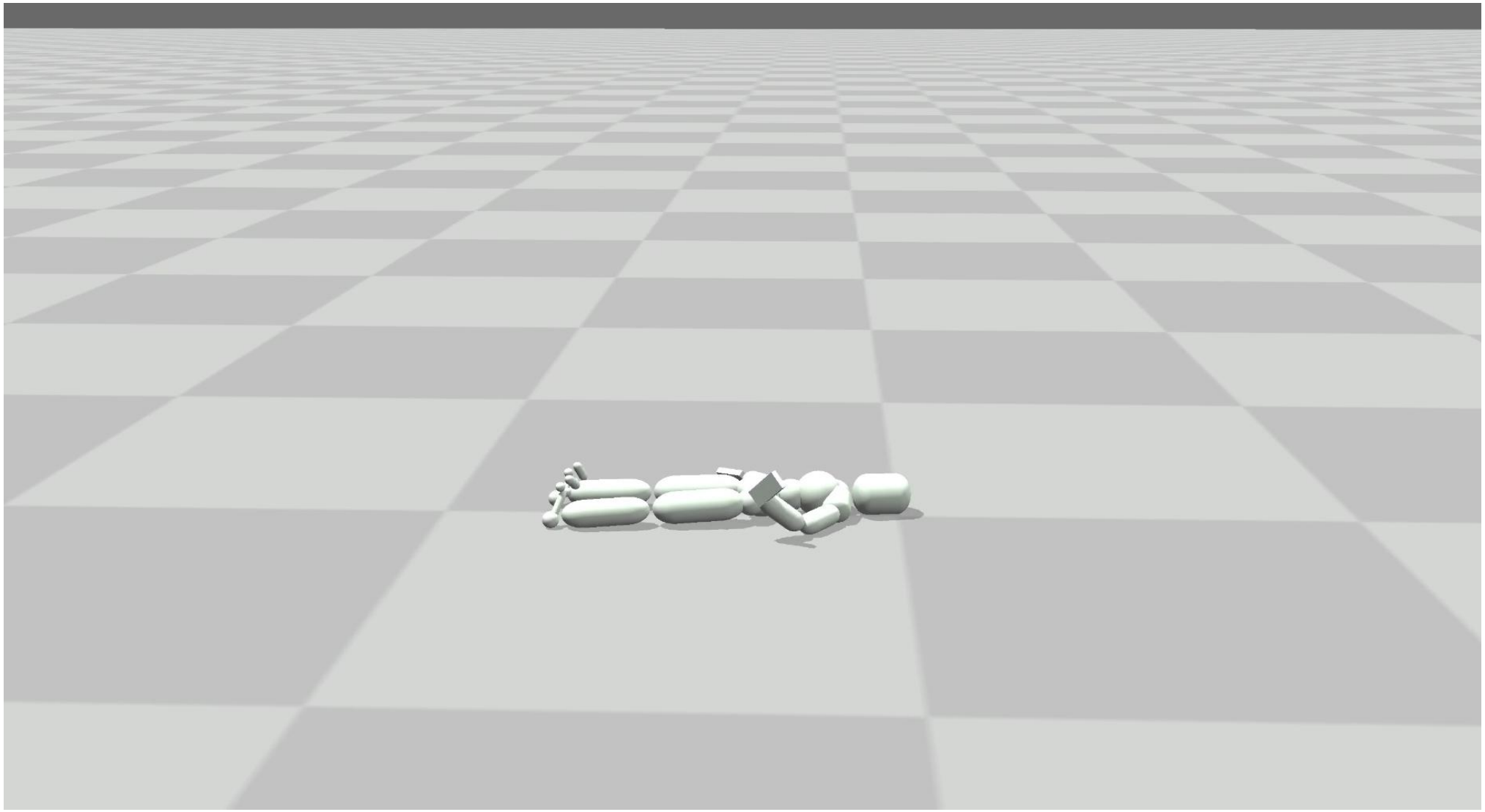


Current state:
$\boldsymbol{x}_t, \boldsymbol{v}_t, R_t, \boldsymbol{\omega}_t$

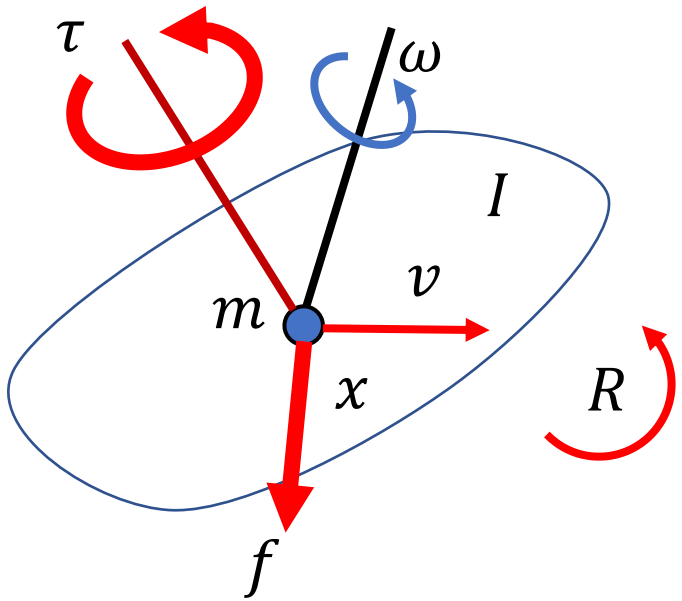
- Collision Detection
- Contact Handling
- Constraint Handling
- Construct Equations of Motion
- Solve Equations of Motion
- Update Velocity $\boldsymbol{v}_{t+1}, \boldsymbol{\omega}_{t+1}$
- Update Position $\boldsymbol{x}_{t+1}, \boldsymbol{R}_{t+1}$

Next state:
$\boldsymbol{x}_{t+1}, \boldsymbol{v}_{t+1}, R_{t+1}, \boldsymbol{\omega}_{t+1}$

Ragdoll Simulation



Actuating a Rigid Body



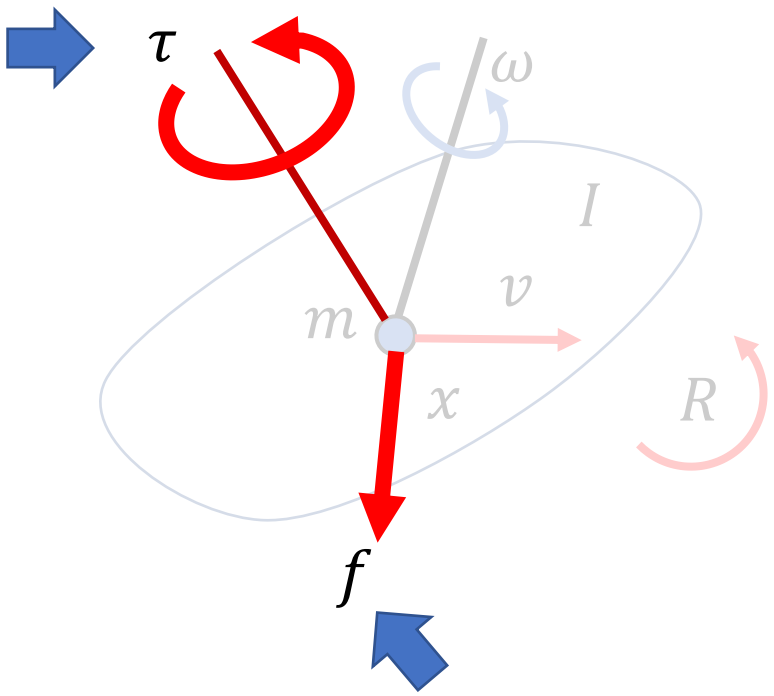
Linear	Angular
x, v	R, ω
$p = mv$	$L = I\omega$

Newton's Second Law:

Euler's laws of motion:

$$\begin{bmatrix} m\mathrm{I}_3 & 0 \\ 0 & I \end{bmatrix} \begin{bmatrix} \dot{v} \\ \dot{\omega} \end{bmatrix} + \begin{bmatrix} 0 \\ \omega \times I\omega \end{bmatrix} = \begin{bmatrix} \boldsymbol{f} \\ \boldsymbol{\tau} \end{bmatrix}$$

Actuating a Rigid Body



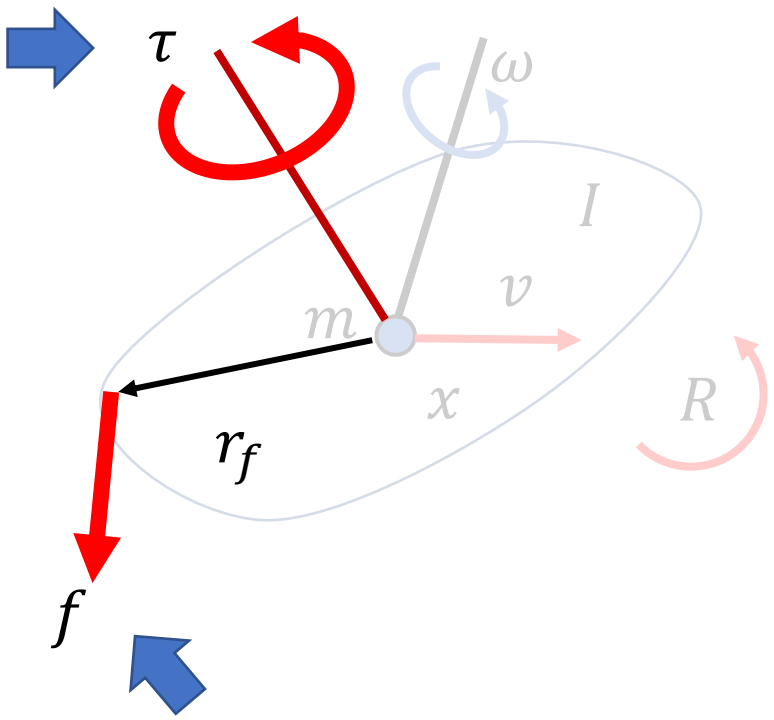
Linear	Angular
x, v	R, ω
$p = mv$	$L = I\omega$

Newton's Second Law:

Euler's laws of motion:

$$\begin{bmatrix} m\mathrm{I}_3 & 0 \\ 0 & I \end{bmatrix} \begin{bmatrix} \dot{v} \\ \dot{\omega} \end{bmatrix} + \begin{bmatrix} 0 \\ \omega \times I\omega \end{bmatrix} = \begin{bmatrix} \boldsymbol{f} \\ \boldsymbol{\tau} \end{bmatrix}$$

Actuating a Rigid Body



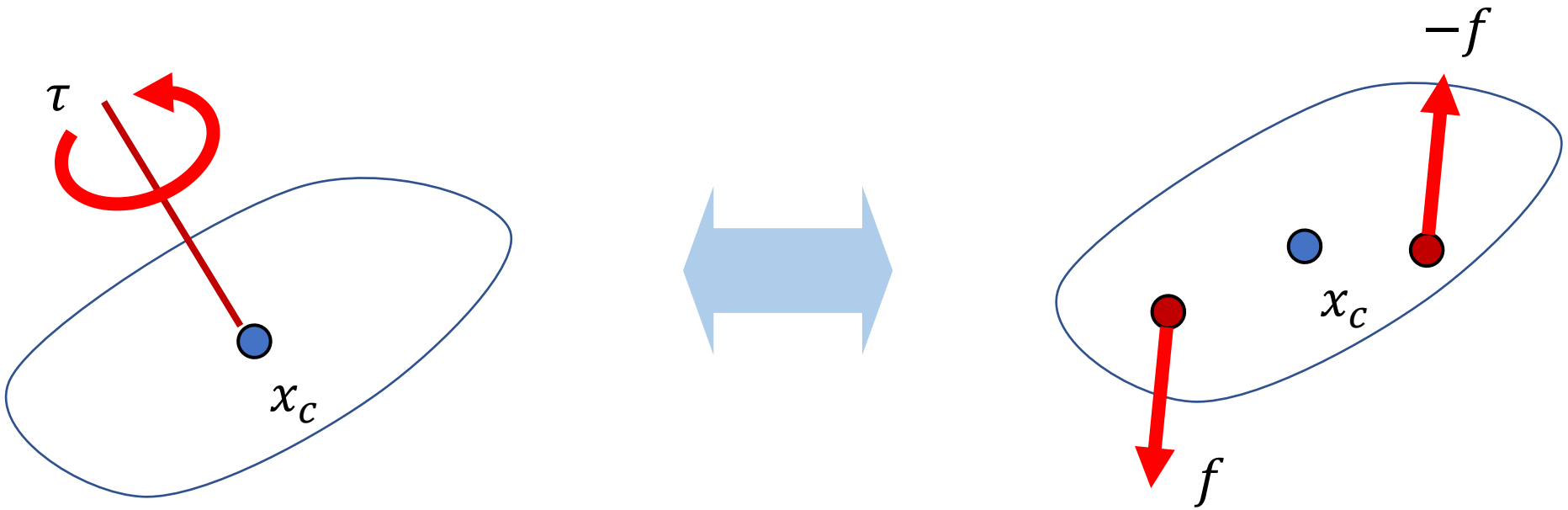
Linear	Angular
x, v	R, ω
$p = mv$	$L = I\omega$

Newton's Second Law:

Euler's laws of motion:

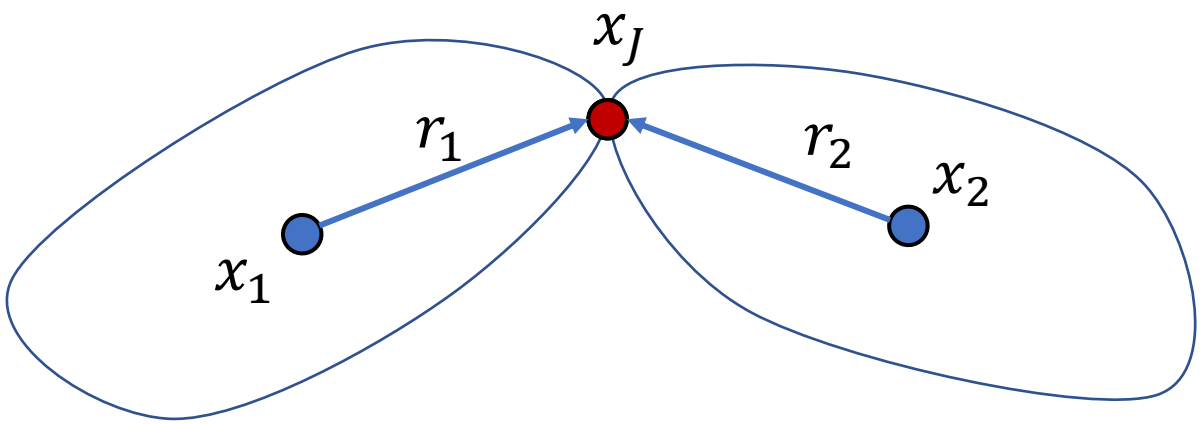
$$\begin{bmatrix} m\mathrm{I}_3 & 0 \\ 0 & I \end{bmatrix} \begin{bmatrix} \dot{v} \\ \dot{\omega} \end{bmatrix} + \begin{bmatrix} 0 \\ \omega \times I\omega \end{bmatrix} = \begin{bmatrix} \boldsymbol{f} \\ \boldsymbol{\tau} + \boldsymbol{r}_f \times \boldsymbol{f} \end{bmatrix}$$

Actuating a Rigid Body



Actuating Articulated Rigid Bodies

$m_i, I_i, x_i, R_i, v_i, \omega_i$

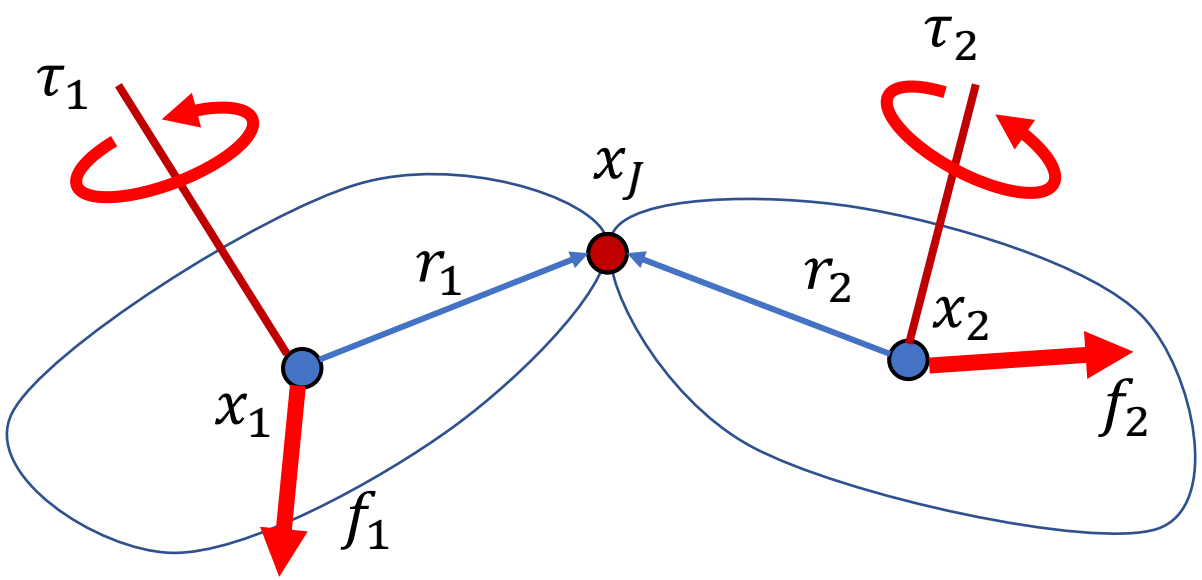


$$M\dot{\boldsymbol{v}} + C(\boldsymbol{x}, \boldsymbol{v}) = \boldsymbol{f} + \boldsymbol{J}^{\boldsymbol{T}}\boldsymbol{\lambda}$$

$$Jv = 0$$

Actuating Articulated Rigid Bodies

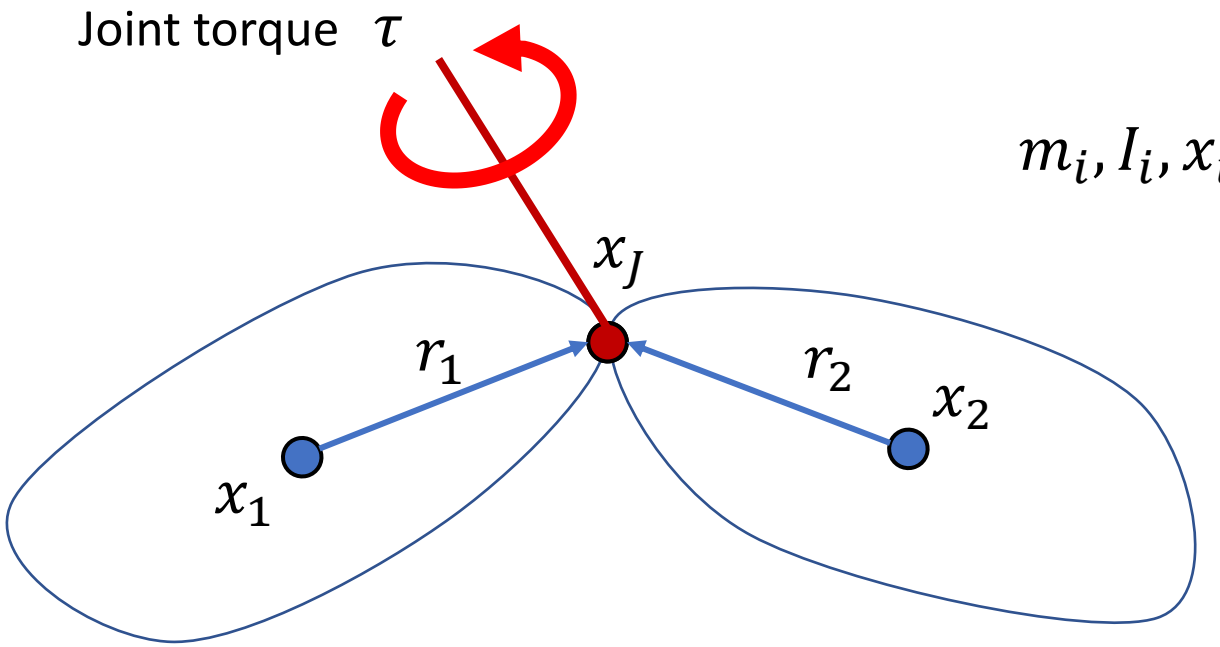
$m_i, I_i, x_i, R_i, v_i, \omega_i$



$$M\dot{\boldsymbol{v}} + C(\boldsymbol{x}, \boldsymbol{v}) = \boldsymbol{f} + \boldsymbol{J}^{\boldsymbol{T}}\boldsymbol{\lambda}$$

$$Jv = 0$$

Actuating Articulated Rigid Bodies

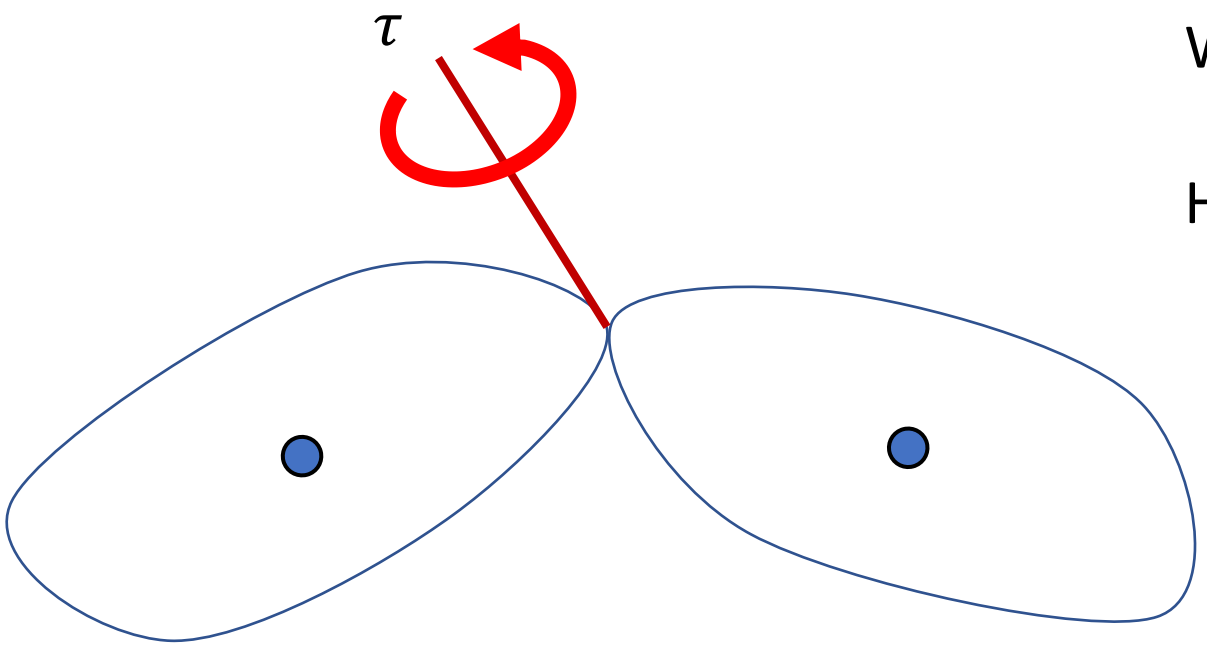


$$m_i, I_i, x_i, R_i, v_i, \omega_i$$

$$M\dot{\boldsymbol{v}} + C(\boldsymbol{x}, \boldsymbol{v}) = \boldsymbol{f} + \boldsymbol{J}^{\boldsymbol{T}}\boldsymbol{\lambda}$$

$$Jv = 0$$

Joint Torques



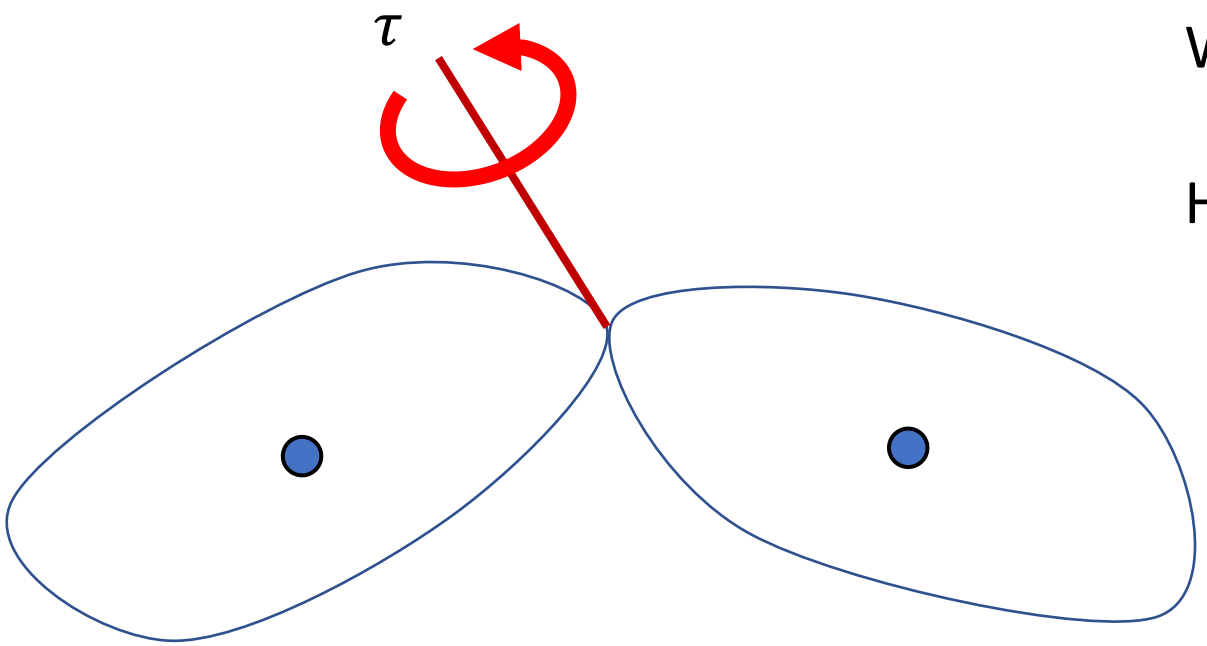
What is a joint torque?

How to apply a joint torque?

$$M\dot{\boldsymbol{v}} + C(\boldsymbol{x}, \boldsymbol{v}) = \boldsymbol{f} + \boldsymbol{J}^{\boldsymbol{T}}\boldsymbol{\lambda}$$

$$Jv = 0$$

Joint Torques



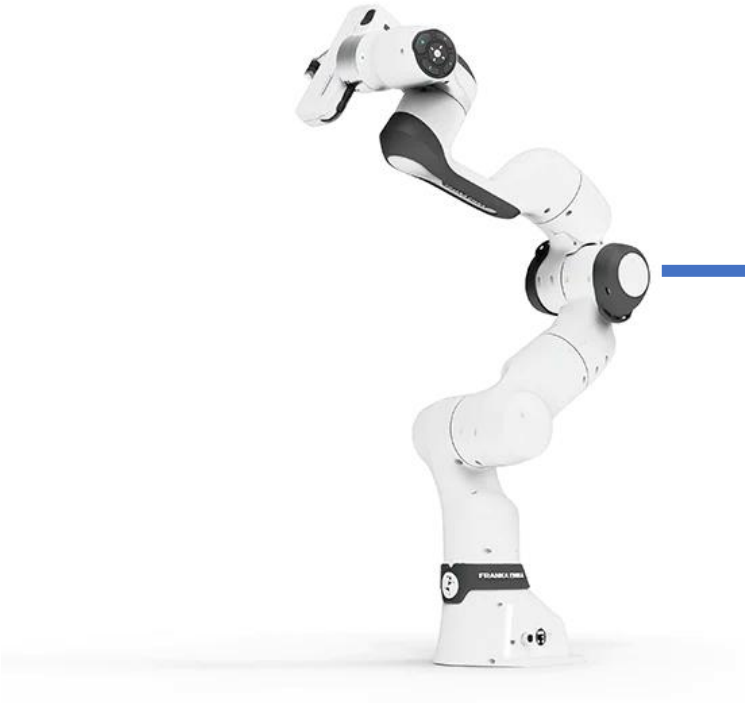
What is a joint torque?

How to apply a joint torque?

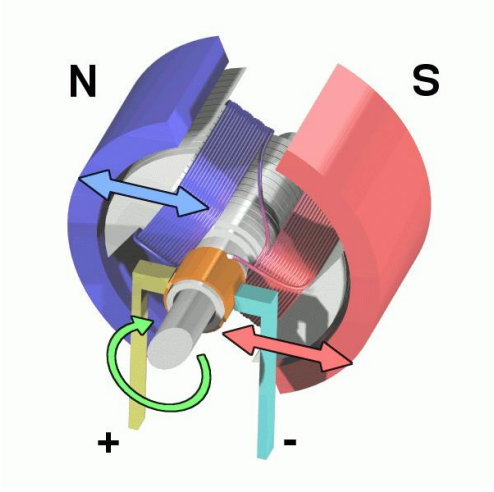
$$\begin{bmatrix} m_1 \mathrm{I}_3 & & & \\ & I_1 & & \\ & & m_2 \mathrm{I}_3 & \\ & & & I_2 \end{bmatrix} \begin{bmatrix} \dot{v}_1 \\ \dot{\omega}_1 \\ \dot{v}_2 \\ \dot{\omega}_2 \end{bmatrix} + \begin{bmatrix} 0 \\ \omega_1 \times I_1 \omega_1 \\ 0 \\ \omega_2 \times I_2 \omega_2 \end{bmatrix} = \begin{bmatrix} f_1 \\ \tau_1 \\ f_2 \\ \tau_2 \end{bmatrix} + J^T \lambda$$

$$Jv = 0$$

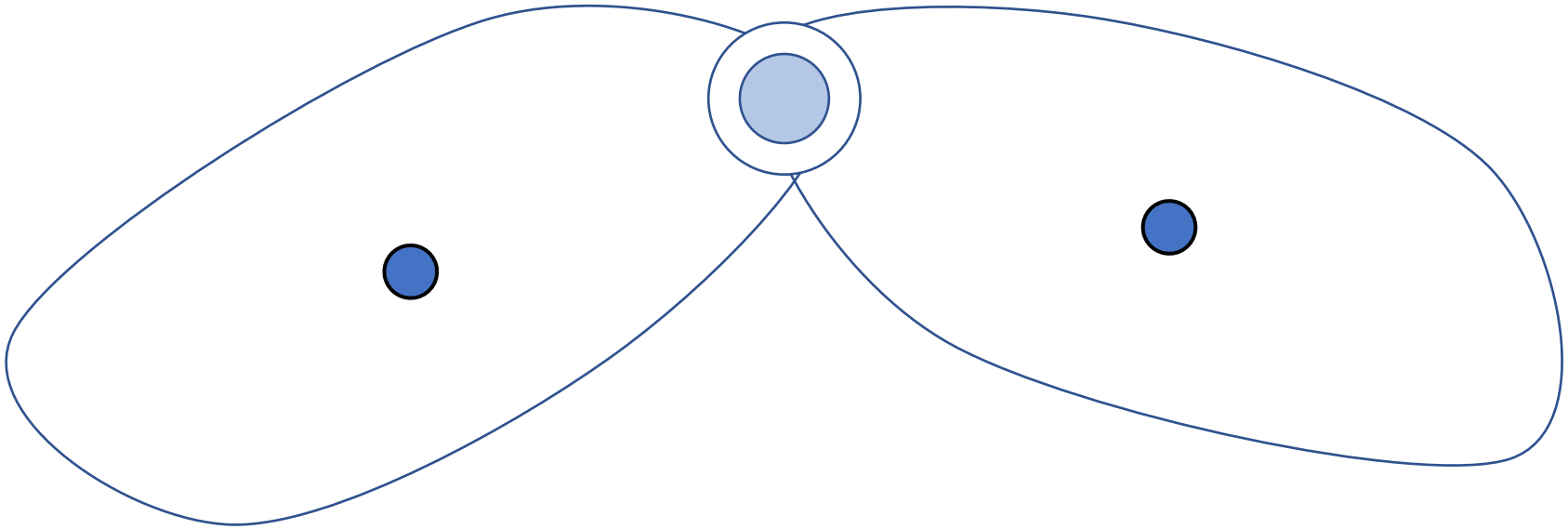
Joint Torques



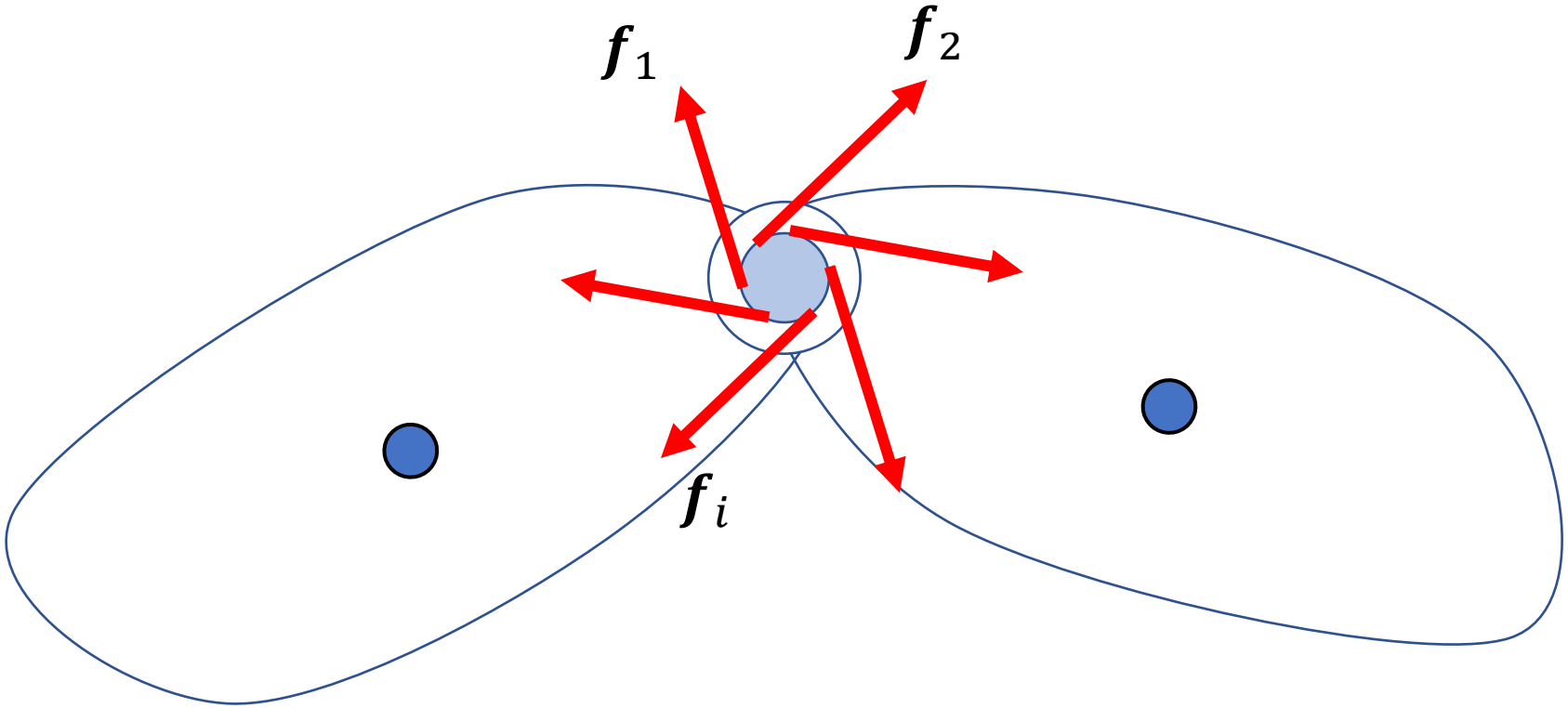
Franka



Joint Torques



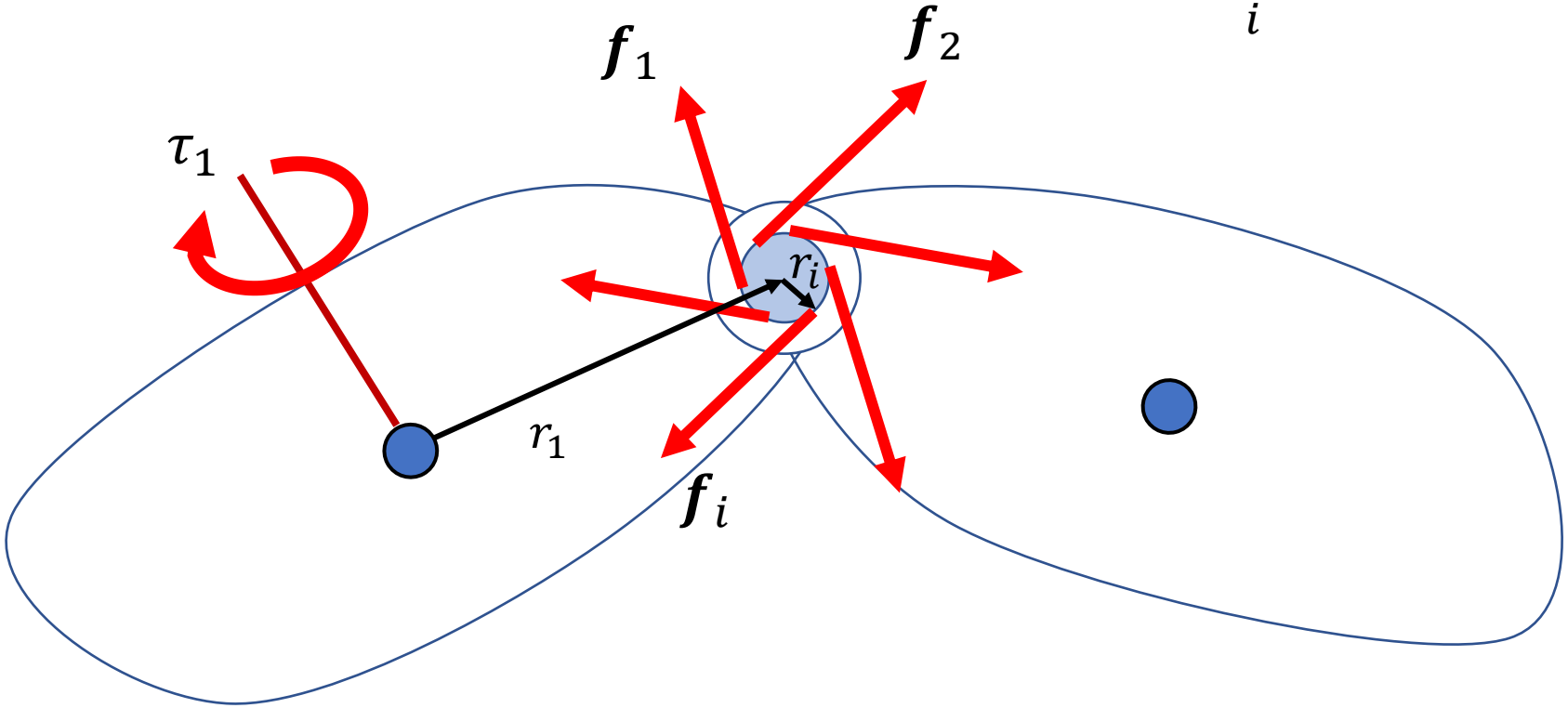
Joint Torques



$$\sum_i f_i = 0$$

Joint Torques

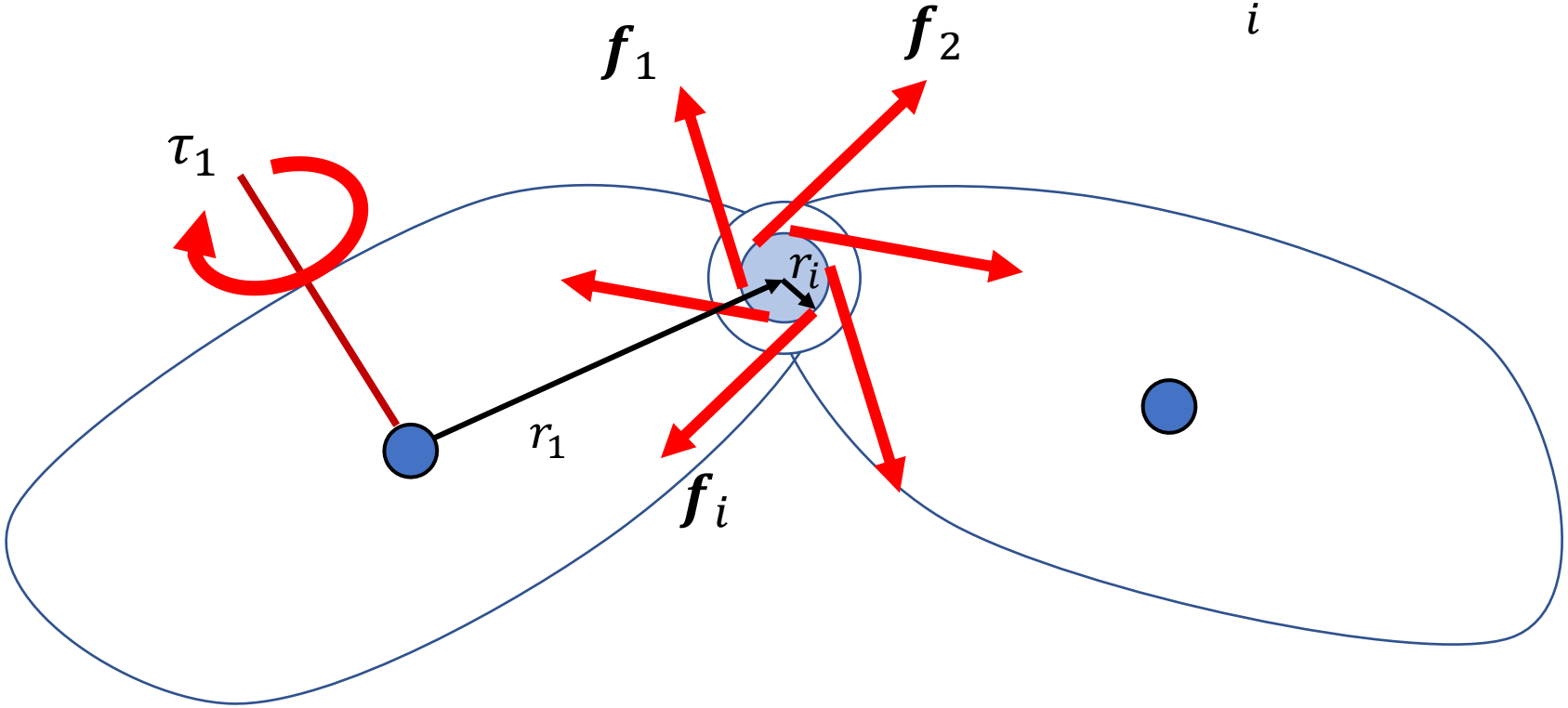
$$\sum_i f_i = 0$$



$$\tau_1 = \sum_i (r_1 + r_i) \times f_i = r_1 \times \sum_i f_i + \sum_i r_i \times f_i$$

Joint Torques

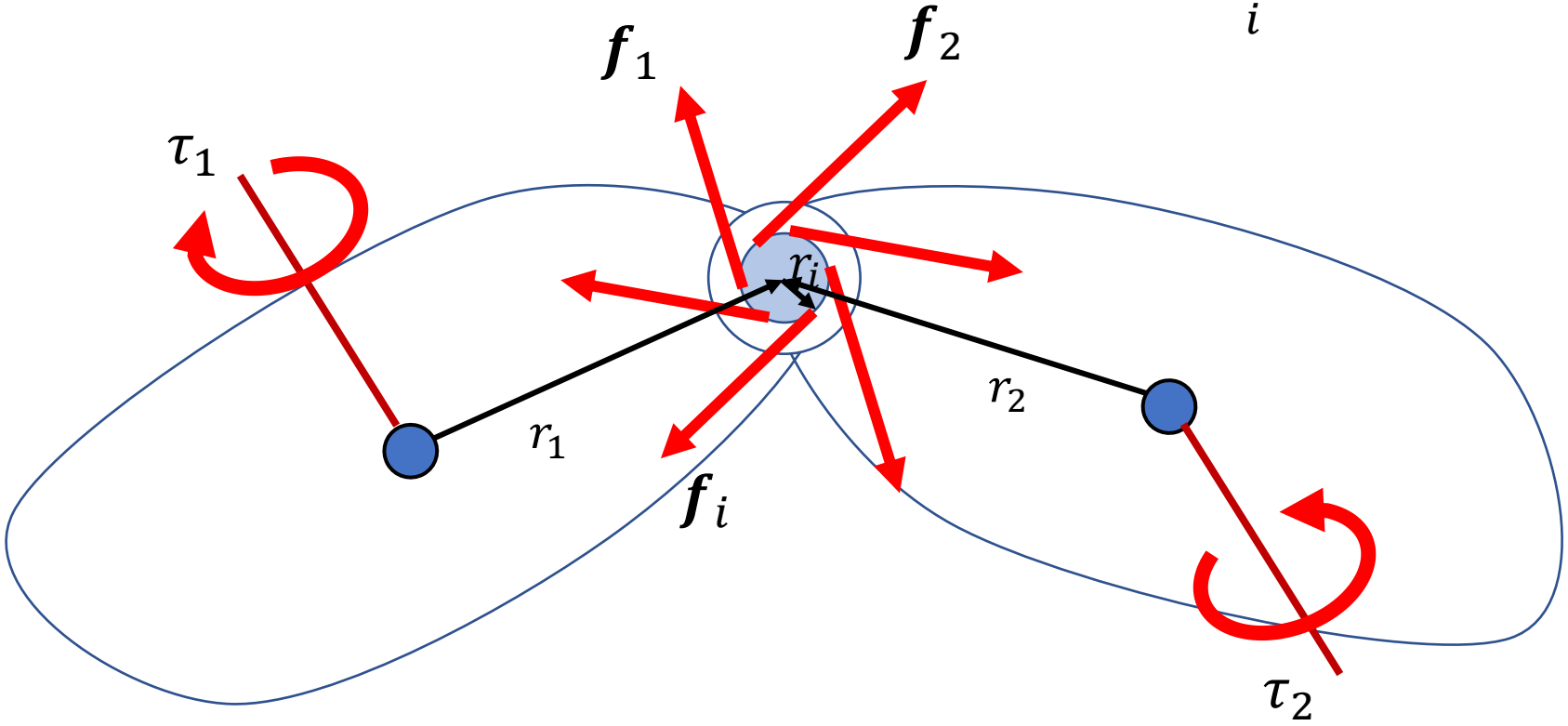
$$\sum_i f_i = 0$$



$$\tau_1 = \sum_i r_i \times f_i$$

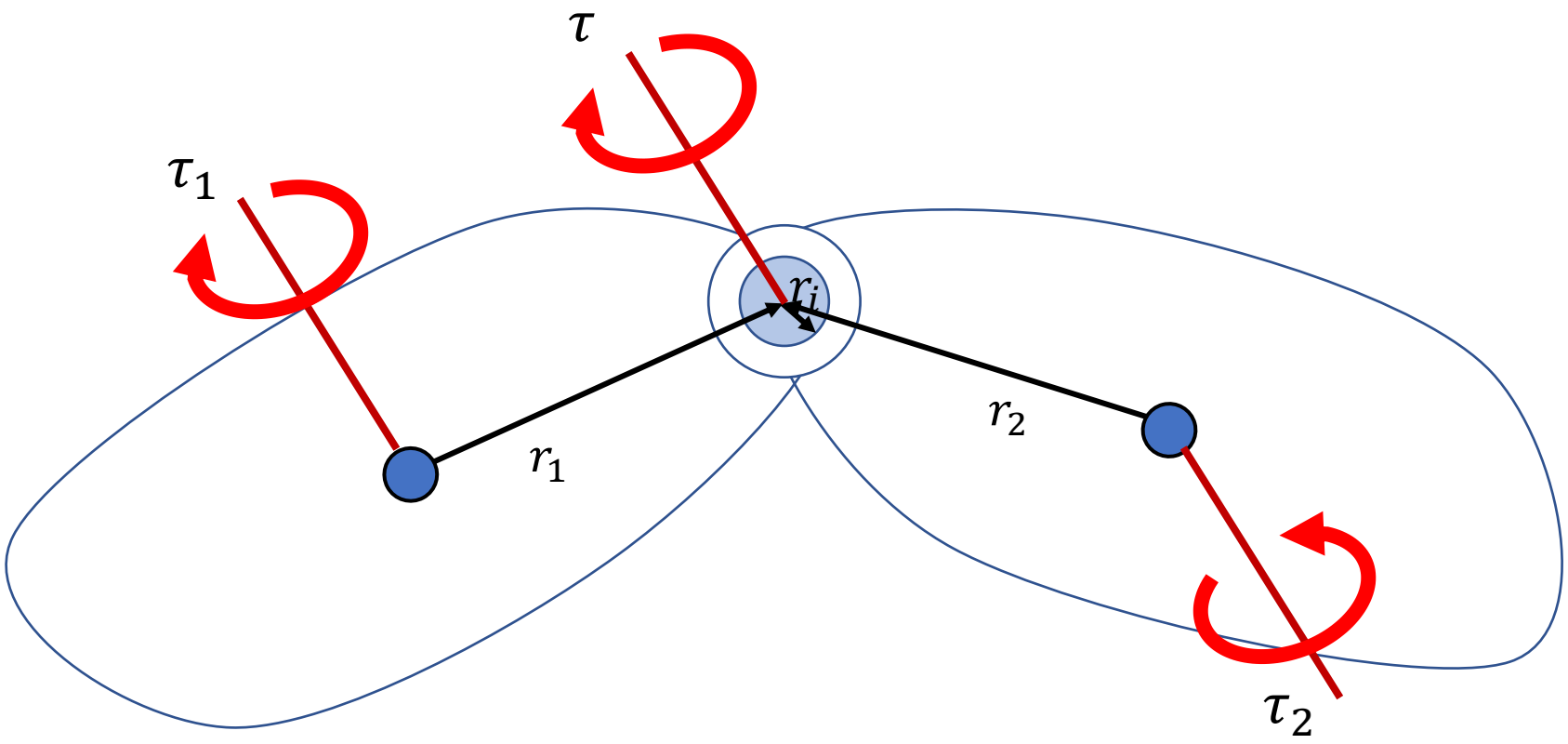
Joint Torques

$$\sum_i f_i = 0$$



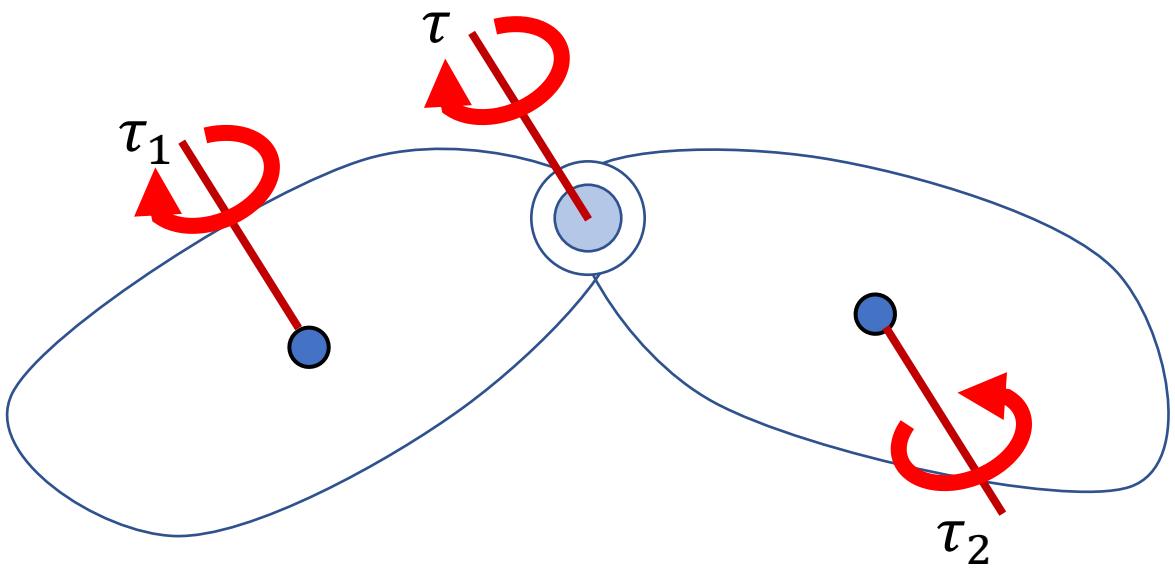
$$\tau_1 = \sum_i r_i \times f_i \qquad \tau_2 = -\sum_i r_i \times f_i$$

Joint Torques



$\tau_1 = \tau$ $\tau_2 = -\tau$

Joint Torques



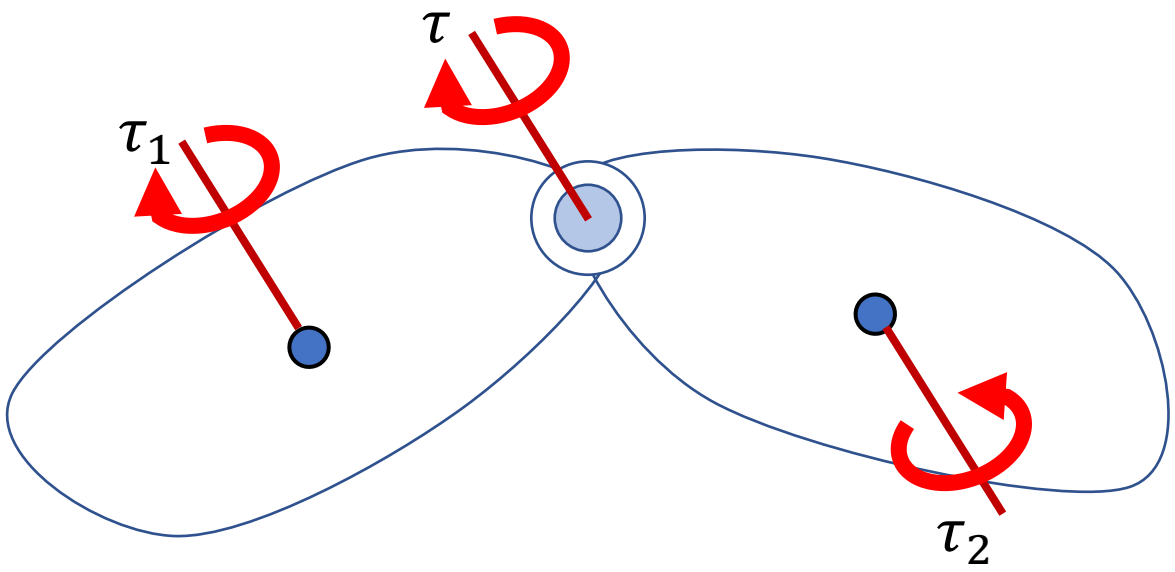
$\tau_1 = \tau$

$\tau_2 = -\tau$

Applying a joint torque τ:

- Add τ to one attached body
- Add $-\tau$ to the other attached body

Joint Torques



$\tau_1 = \tau$

$\tau_2 = -\tau$

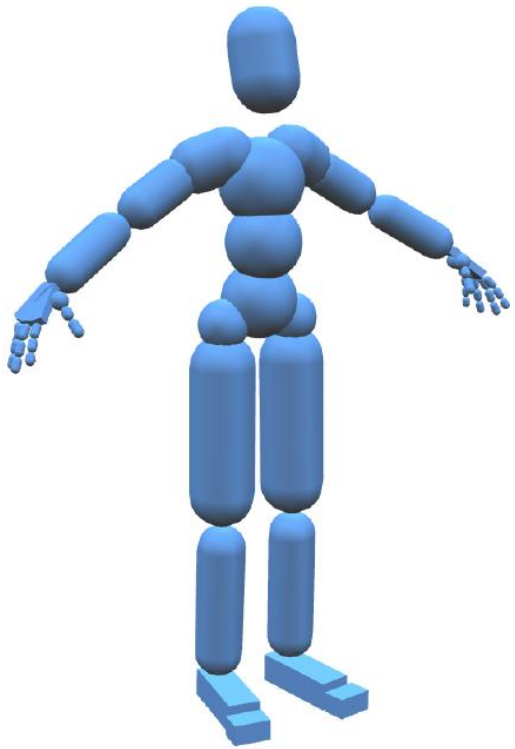
Applying a joint torque τ:

- Add τ to one attached body
- Add $-\tau$ to the other attached body

$$M \begin{bmatrix} \dot{v}_1 \\ \dot{\omega}_1 \\ \dot{v}_2 \\ \dot{\omega}_2 \end{bmatrix} + \begin{bmatrix} 0 \\ \omega_1 \times I_1 \omega_1 \\ 0 \\ \omega_2 \times I_2 \omega_2 \end{bmatrix} = \begin{bmatrix} 0 \\ \tau \\ 0 \\ -\tau \end{bmatrix} + J^T \lambda$$

$$Jv = 0$$

Simulating a Character

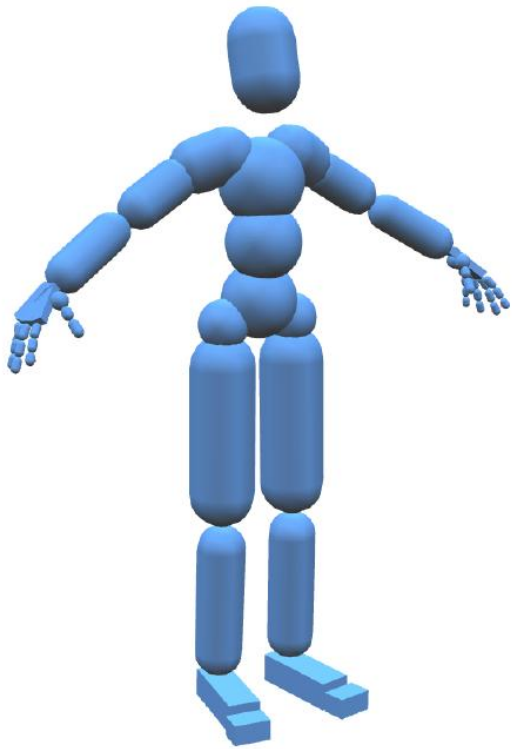


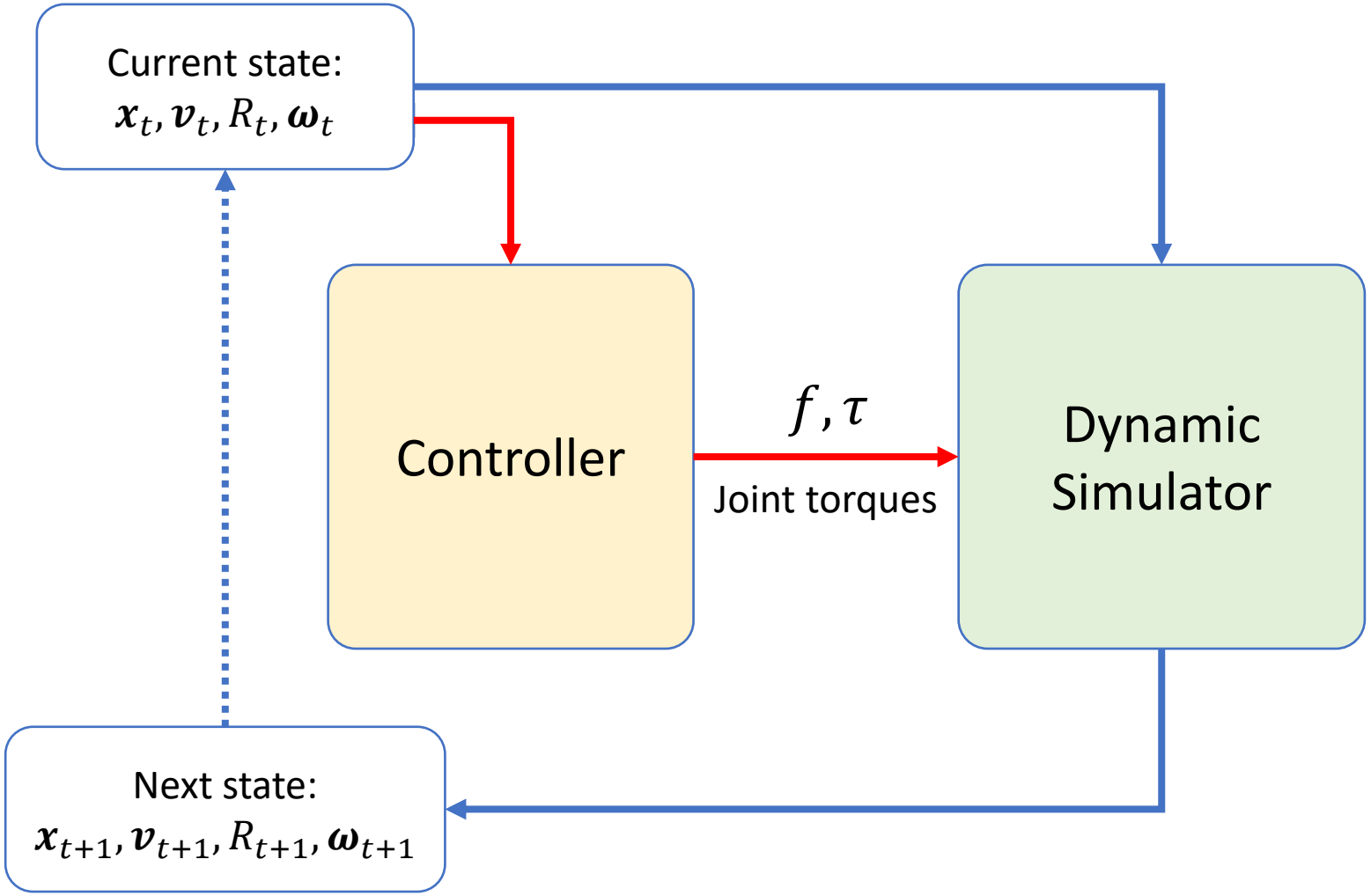
Current state:
$\boldsymbol{x}_t, \boldsymbol{v}_t, R_t, \boldsymbol{\omega}_t$

Dynamic Simulator

Next state:
$\boldsymbol{x}_{t+1}, \boldsymbol{v}_{t+1}, R_{t+1}, \boldsymbol{\omega}_{t+1}$

Simulating + Controlling a Character





Forward Dynamics vs. Inverse Dynamics

Equations of motion of the system:

$$M\dot{\boldsymbol{v}} + C(\boldsymbol{x}, \boldsymbol{v}) = \boldsymbol{f} + J^T\lambda \qquad Jv = 0$$

Forward Dynamics vs. Inverse Dynamics

Equations of motion of the system:

$$M\dot{\boldsymbol{v}} + C(\boldsymbol{x}, \boldsymbol{v}) = \boldsymbol{f} + J^T\lambda \qquad Jv = 0$$

Forward dynamics: $[\boldsymbol{x}, \boldsymbol{v}, \boldsymbol{f}] \mapsto \dot{\boldsymbol{v}}$

given a set of force/torques, compute the motion

Forward Dynamics vs. Inverse Dynamics

Equations of motion of the system:

$$M\dot{\boldsymbol{v}} + C(\boldsymbol{x}, \boldsymbol{v}) = \boldsymbol{f} + J^T\lambda \qquad Jv = 0$$

Forward dynamics: $[\boldsymbol{x}, \boldsymbol{v}, \boldsymbol{f}] \mapsto \dot{\boldsymbol{v}}$

given a set of force/torques, compute the motion

Inverse dynamics: $[\boldsymbol{x}, \boldsymbol{v}, \dot{\boldsymbol{v}}] \mapsto \boldsymbol{f}$

given a motion, compute the forces/torques that give rise to it

Forward Dynamics vs. Inverse Dynamics

Equations of motion of the system:

$$M\dot{\boldsymbol{v}} + C(\boldsymbol{x}, \boldsymbol{v}) = \boldsymbol{f} + J^T\lambda \qquad Jv = 0$$

Forward dynamics: $[\boldsymbol{x}, \boldsymbol{v}, \boldsymbol{f}] \mapsto \dot{\boldsymbol{v}}$

dynamic simulator

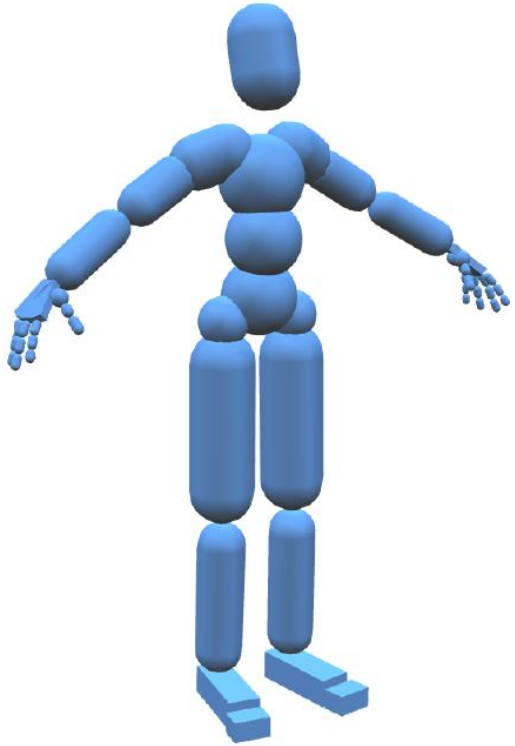
given a set of force/torques, compute the motion

Inverse dynamics: $[\boldsymbol{x}, \boldsymbol{v}, \dot{\boldsymbol{v}}] \mapsto \boldsymbol{f}$

controller

given a motion, compute the forces/torques that give rise to it

Simulating + Control



If #actuators < #dofs,
the system is **underactuated**

Current state:
$\boldsymbol{x}_t, \boldsymbol{v}_t, R_t, \boldsymbol{\omega}_t$

Controller

f, τ

Joint torques

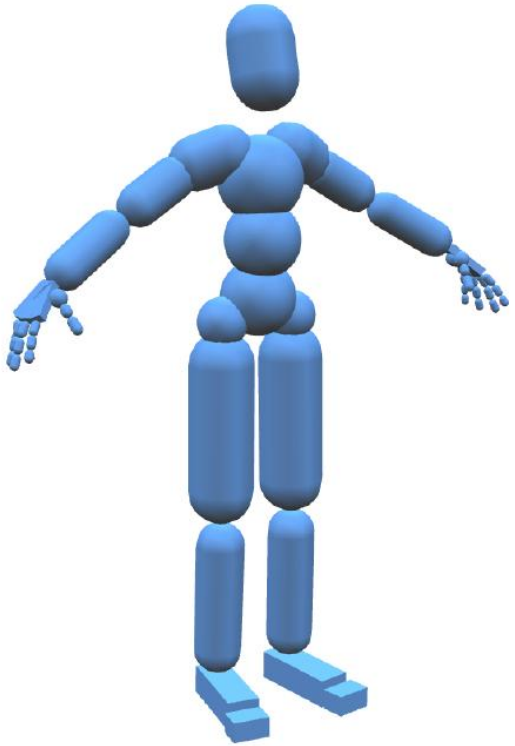
Dynamic Simulator

Next state:
$\boldsymbol{x}_{t+1}, \boldsymbol{v}_{t+1}, R_{t+1}, \boldsymbol{\omega}_{t+1}$

Fully-Actuated vs. Underactuated

Fully-Actuated

Underactuated



欠驱动系统

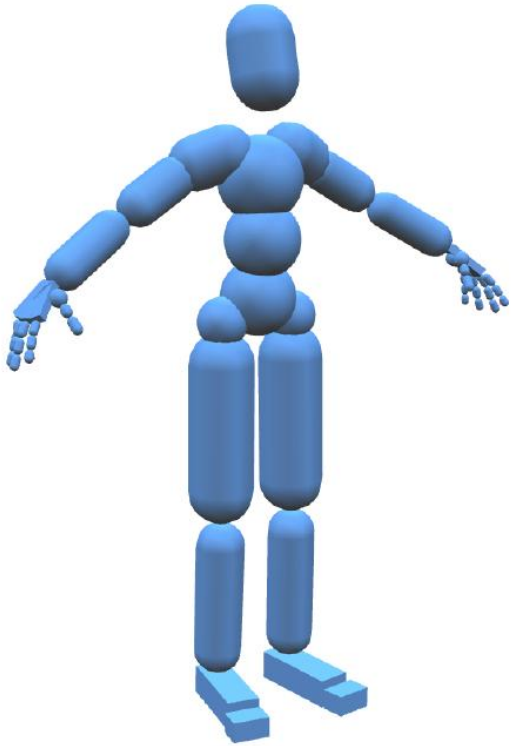
If #actuators ≥ #dofs, the system is **fully-actuated**

If #actuators < #dofs, the system is **underactuated**

Fully-Actuated vs. Underactuated

Fully-Actuated

Underactuated

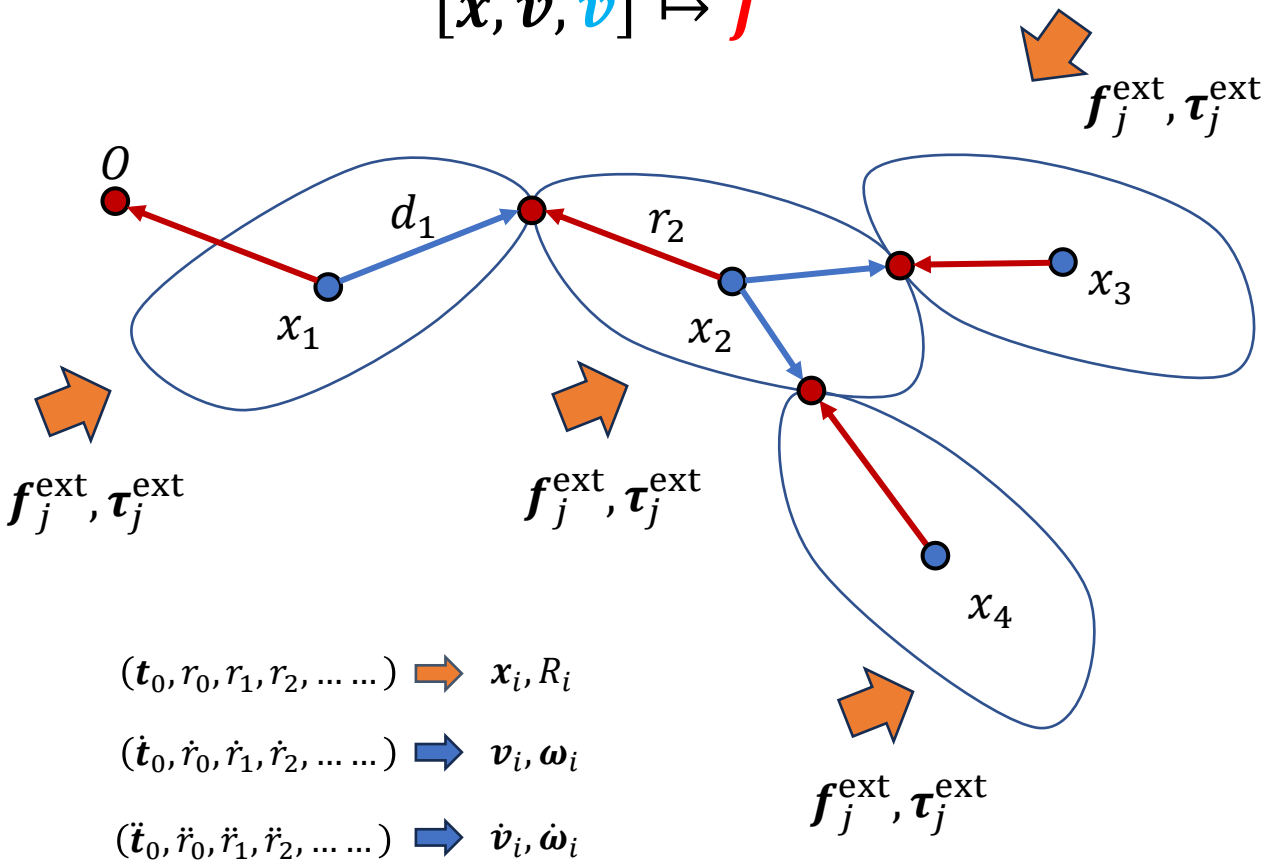


For any $[\boldsymbol{x}, \boldsymbol{v}, \dot{\boldsymbol{v}}]$, there exists an $\boldsymbol{f}$ that produces the motion

For many $[\boldsymbol{x}, \boldsymbol{v}, \dot{\boldsymbol{v}}]$, there is no such $\boldsymbol{f}$ that produces the motion

Inverse Dynamics

$$[\boldsymbol{x}, \boldsymbol{v}, \dot{\boldsymbol{v}}] \mapsto \boldsymbol{f}$$



$(\boldsymbol{t}_0, r_0, r_1, r_2, \dots\dots) \Rightarrow \boldsymbol{x}_i, R_i$

$(\dot{\boldsymbol{t}}_0, \dot{r}_0, \dot{r}_1, \dot{r}_2, \dots\dots) \Rightarrow \boldsymbol{v}_i, \boldsymbol{\omega}_i$

$(\ddot{\boldsymbol{t}}_0, \ddot{r}_0, \ddot{r}_1, \ddot{r}_2, \dots\dots) \Rightarrow \dot{\boldsymbol{v}}_i, \dot{\boldsymbol{\omega}}_i$

Given: $m_i, I_i, \boldsymbol{x}_i, R_i, \boldsymbol{v}_i, \boldsymbol{\omega}_i$

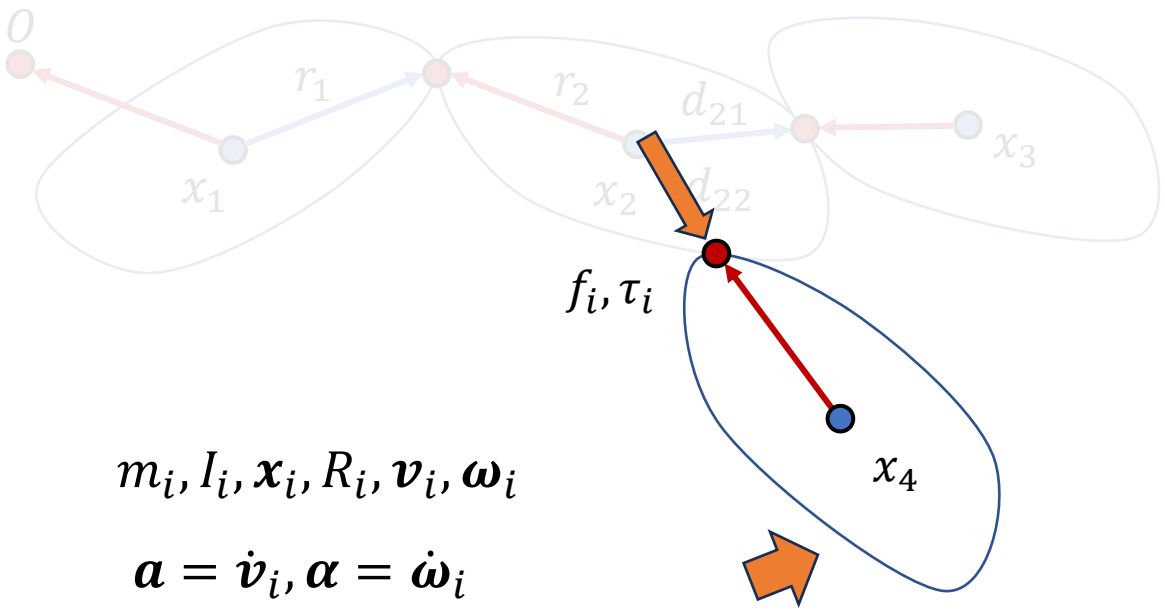
$\boldsymbol{a} = \dot{\boldsymbol{v}}_i, \boldsymbol{\alpha} = \dot{\boldsymbol{\omega}}_i$

$\boldsymbol{f}_j^{\text{ext}}, \boldsymbol{\tau}_j^{\text{ext}}$

Compute: $\boldsymbol{f}_k, \boldsymbol{\tau}_k$

Inverse Dynamics

$$[\boldsymbol{x}, \boldsymbol{v}, \dot{\boldsymbol{v}}] \mapsto \boldsymbol{f}$$



$m_i, I_i, \boldsymbol{x}_i, R_i, \boldsymbol{v}_i, \boldsymbol{\omega}_i$

$\boldsymbol{a} = \dot{\boldsymbol{v}}_i, \boldsymbol{\alpha} = \dot{\boldsymbol{\omega}}_i$

$\boldsymbol{f}_j^{\text{ext}}, \boldsymbol{\tau}_j^{\text{ext}}$

$\boldsymbol{f}_j^{\text{ext}}, \boldsymbol{\tau}_j^{\text{ext}}$

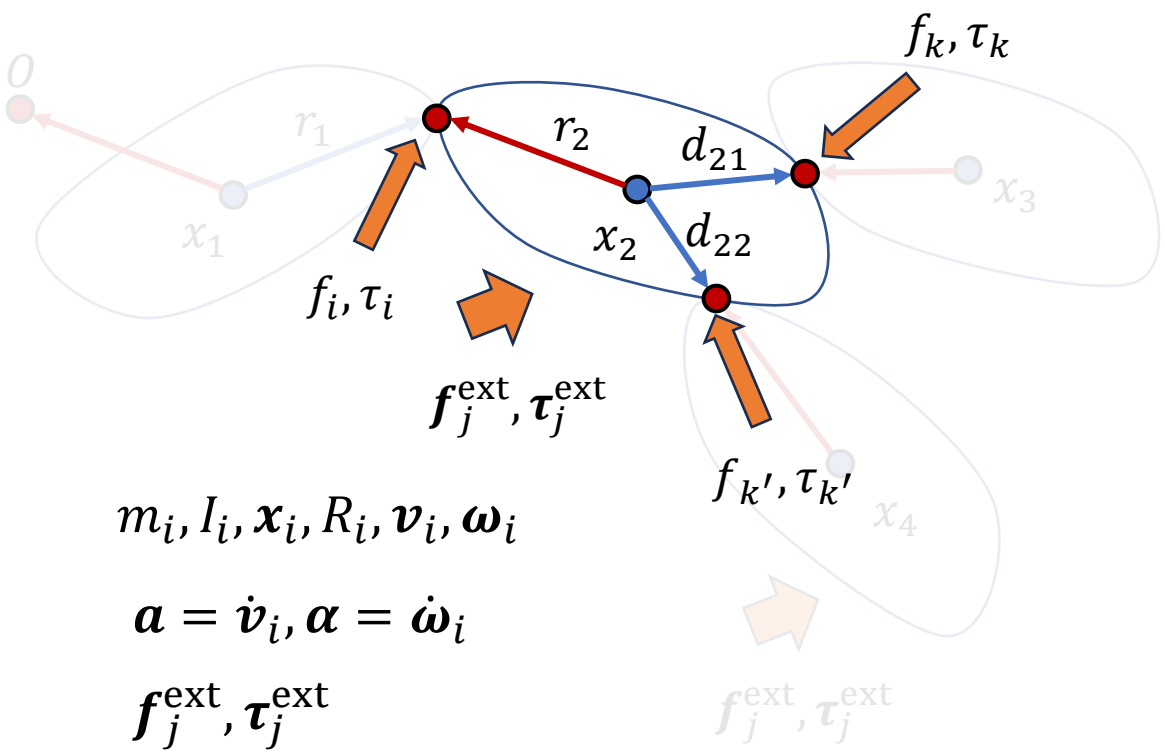
Recursive Newton Euler Algorithm (RNEA)

$$m_i \dot{\boldsymbol{v}}_i = \boldsymbol{f}_i + \sum_j \boldsymbol{f}_j^{\text{ext}}$$

$$I_i \dot{\boldsymbol{\omega}}_i + \boldsymbol{\omega}_i \times I_i \boldsymbol{\omega}_i = \boldsymbol{\tau}_i + \boldsymbol{r}_i \times \boldsymbol{f}_i + \sum_j \boldsymbol{\tau}_j^{\text{ext}}$$

Inverse Dynamics

$$[\boldsymbol{x}, \boldsymbol{v}, \dot{\boldsymbol{v}}] \mapsto \boldsymbol{f}$$



$m_i, I_i, \boldsymbol{x}_i, R_i, \boldsymbol{v}_i, \boldsymbol{\omega}_i$

$\boldsymbol{a} = \dot{\boldsymbol{v}}_i, \boldsymbol{\alpha} = \dot{\boldsymbol{\omega}}_i$

$\boldsymbol{f}_j^{\text{ext}}, \boldsymbol{\tau}_j^{\text{ext}}$

Recursive Newton Euler Algorithm (RNEA)

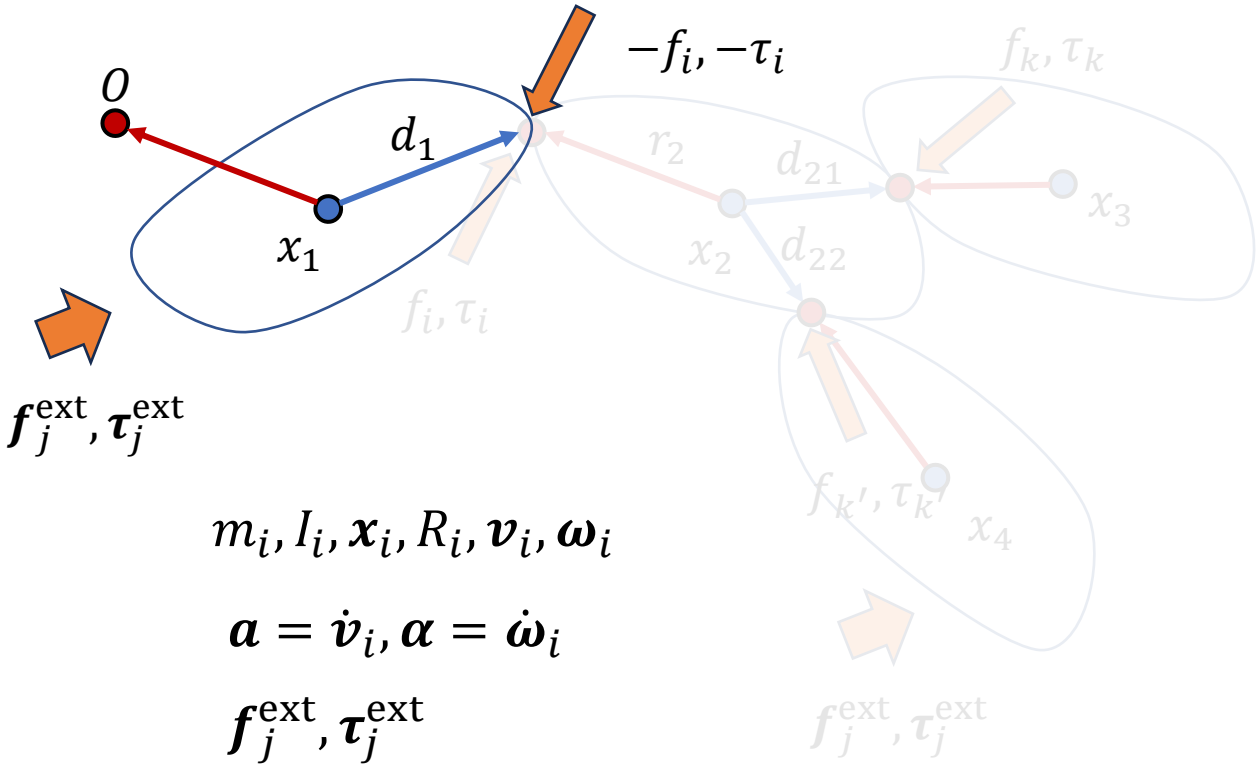
$$m_i \dot{\boldsymbol{v}}_i = \boldsymbol{f}_i - \sum_{k \in c(i)} \boldsymbol{f}_k + \sum_j \boldsymbol{f}_j^{\text{ext}}$$

$$I_i \dot{\boldsymbol{\omega}}_i + \boldsymbol{\omega}_i \times I_i \boldsymbol{\omega}_i = \boldsymbol{\tau}_i + \boldsymbol{r}_i \times \boldsymbol{f}_i$$

$$- \sum_{k \in c(i)} (\boldsymbol{\tau}_k + \boldsymbol{d}_{ik} \times \boldsymbol{f}_k) + \sum_j \boldsymbol{\tau}_j^{\text{ext}}$$

Inverse Dynamics

$$[\boldsymbol{x}, \boldsymbol{v}, \dot{\boldsymbol{v}}] \mapsto \boldsymbol{f}$$



$m_i, I_i, \boldsymbol{x}_i, R_i, \boldsymbol{v}_i, \boldsymbol{\omega}_i$

$\boldsymbol{a} = \dot{\boldsymbol{v}}_i, \boldsymbol{\alpha} = \dot{\boldsymbol{\omega}}_i$

$\boldsymbol{f}_j^{\text{ext}}, \boldsymbol{\tau}_j^{\text{ext}}$

Recursive Newton Euler Algorithm (RNEA)

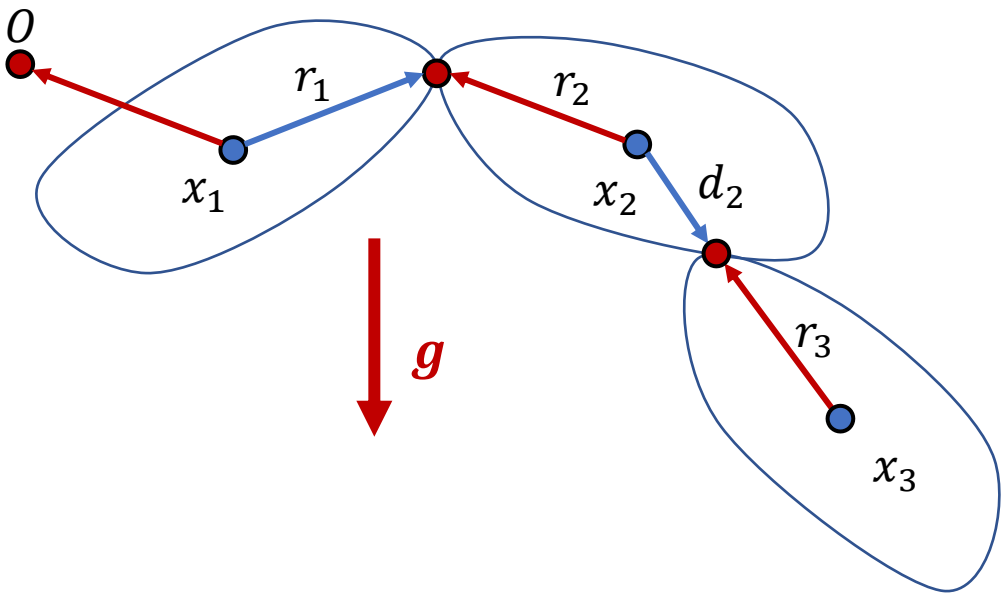
$$m_i \dot{\boldsymbol{v}}_i = \boldsymbol{f}_i - \sum_{k \in c(i)} \boldsymbol{f}_k + \sum_j \boldsymbol{f}_j^{\text{ext}}$$

$$I_i \dot{\boldsymbol{\omega}}_i + \boldsymbol{\omega}_i \times I_i \boldsymbol{\omega}_i = \boldsymbol{\tau}_i + \boldsymbol{r}_i \times \boldsymbol{f}_i$$

$$- \sum_{k \in c(i)} (\boldsymbol{\tau}_k + \boldsymbol{d}_{ik} \times \boldsymbol{f}_k) + \sum_j \boldsymbol{\tau}_j^{\text{ext}}$$

Example: Gravity Compensation

$$[\boldsymbol{x}, \mathbf{0}, \mathbf{0}] \mapsto \boldsymbol{f}$$



$(\boldsymbol{t}_0, r_0, r_1, r_2, \dots\dots) \Rightarrow \boldsymbol{x}_i, R_i$

$(\dot{\boldsymbol{t}}_0, \dot{r}_0, \dot{r}_1, \dot{r}_2, \dots\dots) \Rightarrow \boldsymbol{v}_i, \boldsymbol{\omega}_i$

Given: $m_i, I_i, \boldsymbol{x}_i, R_i, \boldsymbol{v}_i = 0, \boldsymbol{\omega}_i = 0$

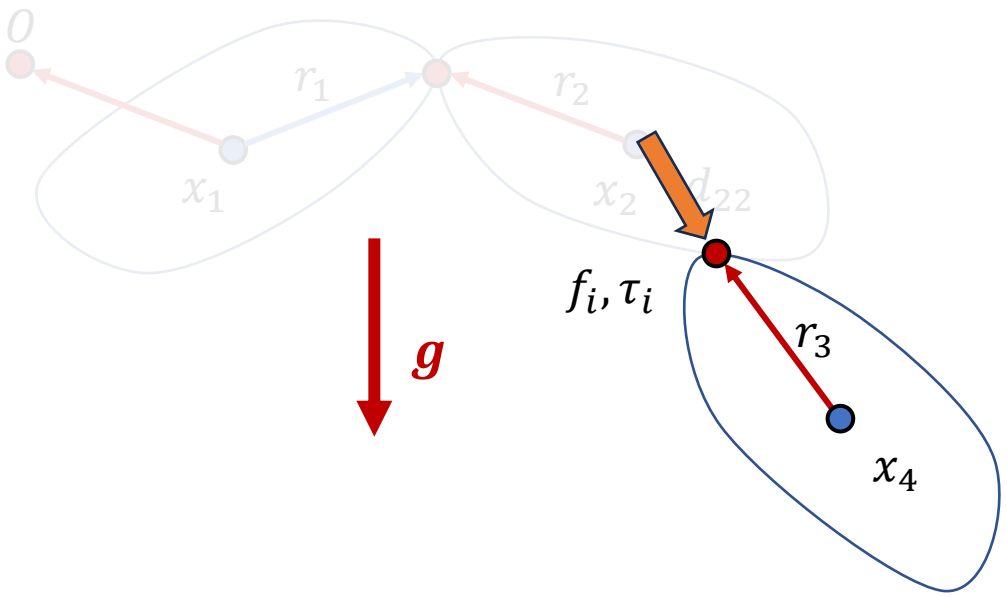
$\boldsymbol{a} = 0, \boldsymbol{\alpha} = 0$

Gravity: $\boldsymbol{g}$

Compute: $\boldsymbol{f}_k, \boldsymbol{\tau}_k$

Example: Gravity Compensation

$$[\boldsymbol{x}, \mathbf{0}, \mathbf{0}] \mapsto \boldsymbol{f}$$



$(\boldsymbol{t}_0, r_0, r_1, r_2, \dots\dots) \Rightarrow \boldsymbol{x}_i, R_i$

$(\dot{\boldsymbol{t}}_0, \dot{r}_0, \dot{r}_1, \dot{r}_2, \dots\dots) \Rightarrow \boldsymbol{v}_i, \boldsymbol{\omega}_i$

Given: $m_i, I_i, \boldsymbol{x}_i, R_i, \boldsymbol{v}_i = 0, \boldsymbol{\omega}_i = 0$

$\boldsymbol{a} = 0, \boldsymbol{\alpha} = 0$

Gravity: $\boldsymbol{g}$

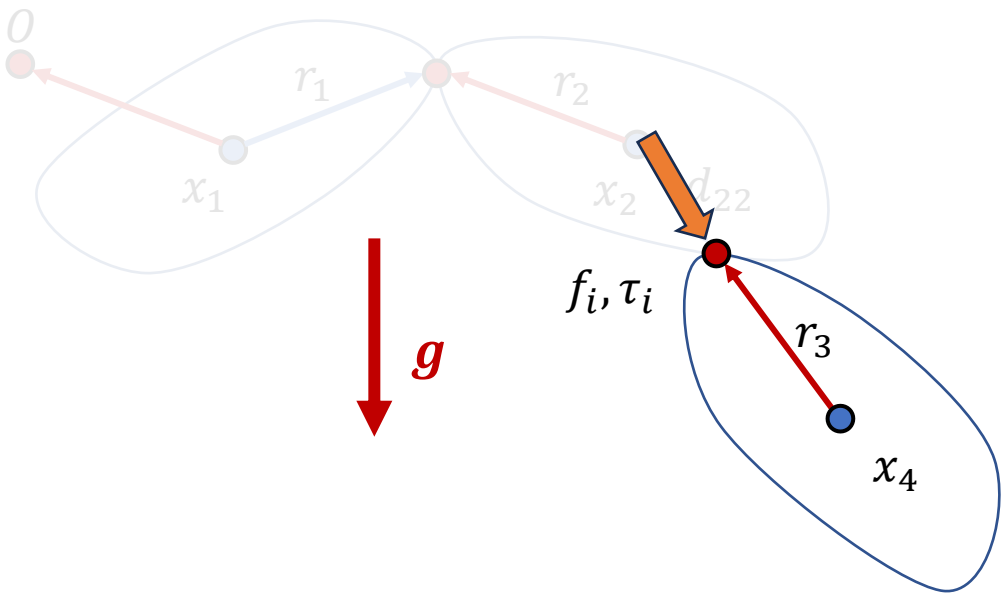
Compute: $\boldsymbol{f}_k, \boldsymbol{\tau}_k$

$$\boldsymbol{f}_3 = -m_3 \boldsymbol{g}$$

$$m_i \dot{\boldsymbol{v}}_i = \boldsymbol{f}_i + \sum_j \boldsymbol{f}_j^{\text{ext}}$$

Example: Gravity Compensation

$$[\boldsymbol{x}, \mathbf{0}, \mathbf{0}] \mapsto \boldsymbol{f}$$



$(\boldsymbol{t}_0, r_0, r_1, r_2, \dots\dots) \Rightarrow \boldsymbol{x}_i, R_i$

$(\dot{\boldsymbol{t}}_0, \dot{r}_0, \dot{r}_1, \dot{r}_2, \dots\dots) \Rightarrow \boldsymbol{v}_i, \boldsymbol{\omega}_i$

Given: $m_i, I_i, \boldsymbol{x}_i, R_i, \boldsymbol{v}_i = 0, \boldsymbol{\omega}_i = 0$

$\boldsymbol{a} = 0, \boldsymbol{\alpha} = 0$

Gravity: $\boldsymbol{g}$

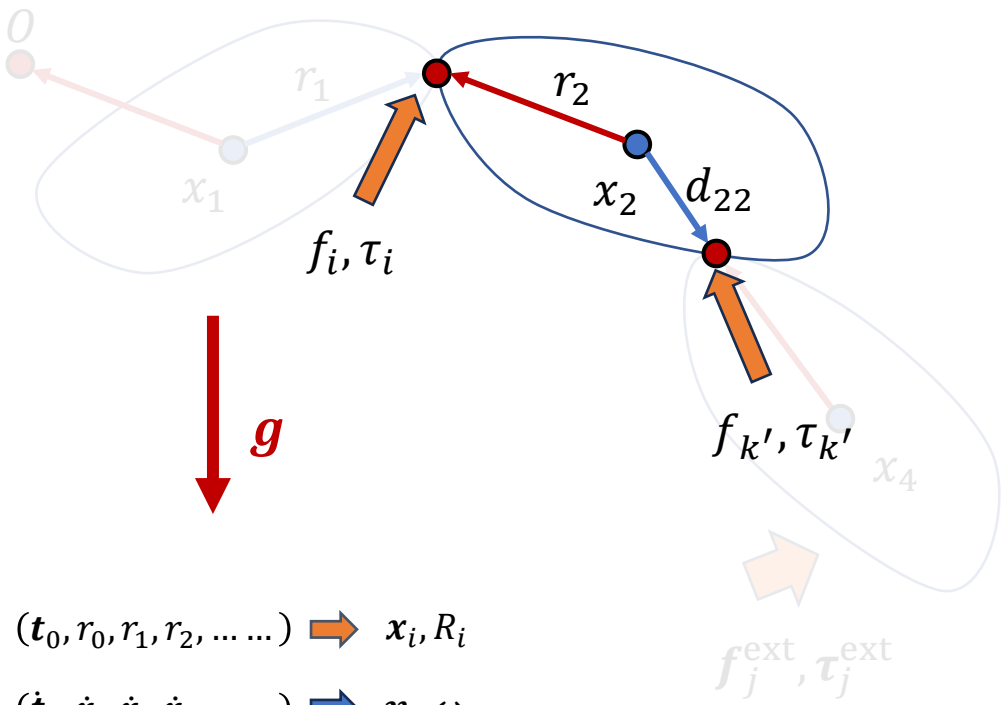
Compute: $\boldsymbol{f}_k, \boldsymbol{\tau}_k$

$$\boldsymbol{f}_3 = -m_3 \boldsymbol{g} \qquad \boldsymbol{\tau}_3 = \boldsymbol{r}_3 \times m_3 \boldsymbol{g}$$

$$I_i \dot{\boldsymbol{\omega}}_i + \boldsymbol{\omega}_i \times I_i \boldsymbol{\omega}_i = \boldsymbol{\tau}_i + \boldsymbol{r}_i \times \boldsymbol{f}_i + \sum_j \boldsymbol{\tau}_j^{\text{ext}}$$

Example: Gravity Compensation

$$[\boldsymbol{x}, \mathbf{0}, \mathbf{0}] \mapsto \boldsymbol{f}$$



$(\boldsymbol{t}_0, r_0, r_1, r_2, \dots\dots) \Rightarrow \boldsymbol{x}_i, R_i$

$(\dot{\boldsymbol{t}}_0, \dot{r}_0, \dot{r}_1, \dot{r}_2, \dots\dots) \Rightarrow \boldsymbol{v}_i, \boldsymbol{\omega}_i$

Given: $m_i, I_i, \boldsymbol{x}_i, R_i, \boldsymbol{v}_i = 0, \boldsymbol{\omega}_i = 0$

$\boldsymbol{a} = 0, \boldsymbol{\alpha} = 0$

Gravity: $\boldsymbol{g}$

Compute: $\boldsymbol{f}_k, \boldsymbol{\tau}_k$

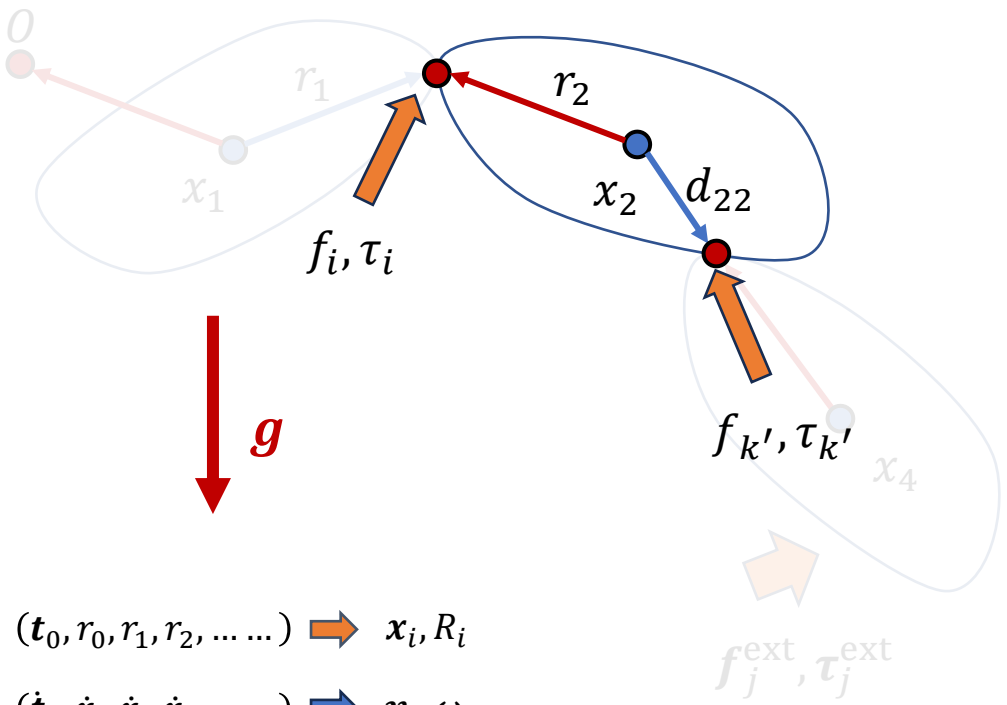
$$\boldsymbol{f}_3 = -m_3\boldsymbol{g} \qquad \boldsymbol{\tau}_3 = \boldsymbol{r}_3 \times m_3\boldsymbol{g}$$

$$\boldsymbol{f}_2 = -m_2\boldsymbol{g} - m_3\boldsymbol{g}$$

$$m_i\dot{\boldsymbol{v}}_i = \boldsymbol{f}_i - \sum_{k \in c(i)} \boldsymbol{f}_k + \sum_j \boldsymbol{f}_j^{\text{ext}}$$

Example: Gravity Compensation

$$[\boldsymbol{x}, \mathbf{0}, \mathbf{0}] \mapsto \boldsymbol{f}$$



$(\boldsymbol{t}_0, r_0, r_1, r_2, \dots\dots) \Rightarrow \boldsymbol{x}_i, R_i$

$(\dot{\boldsymbol{t}}_0, \dot{r}_0, \dot{r}_1, \dot{r}_2, \dots\dots) \Rightarrow \boldsymbol{v}_i, \boldsymbol{\omega}_i$

Given: $m_i, I_i, \boldsymbol{x}_i, R_i, \boldsymbol{v}_i = 0, \boldsymbol{\omega}_i = 0$

$\boldsymbol{a} = 0, \boldsymbol{\alpha} = 0$

Gravity: $\boldsymbol{g}$

Compute: $\boldsymbol{f}_k, \boldsymbol{\tau}_k$

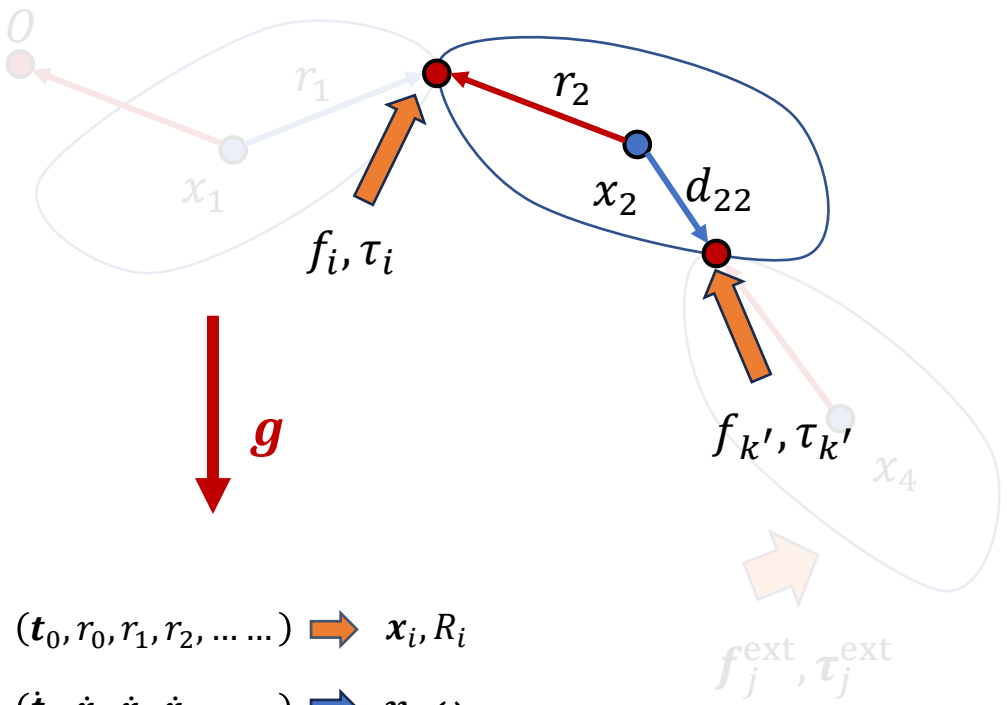
$$\boldsymbol{f}_3 = -m_3\boldsymbol{g} \qquad \boldsymbol{\tau}_3 = \boldsymbol{r}_3 \times m_3\boldsymbol{g}$$

$$\boldsymbol{f}_2 = -m_2\boldsymbol{g} - m_3\boldsymbol{g} \qquad \boldsymbol{\tau}_2 = ?$$

$$I_i\dot{\boldsymbol{\omega}}_i + \boldsymbol{\omega}_i \times I_i\boldsymbol{\omega}_i = \boldsymbol{\tau}_i + \boldsymbol{r}_i \times \boldsymbol{f}_i - \sum_{k \in c(i)} (\boldsymbol{\tau}_k + \boldsymbol{d}_{ik} \times \boldsymbol{f}_k) + \sum_j \boldsymbol{\tau}_j^{\text{ext}}$$

Example: Gravity Compensation

$$[\boldsymbol{x}, \mathbf{0}, \mathbf{0}] \mapsto \boldsymbol{f}$$



$(\boldsymbol{t}_0, r_0, r_1, r_2, \dots\dots) \Rightarrow \boldsymbol{x}_i, R_i$

$(\dot{\boldsymbol{t}}_0, \dot{r}_0, \dot{r}_1, \dot{r}_2, \dots\dots) \Rightarrow \boldsymbol{v}_i, \boldsymbol{\omega}_i$

Given: $m_i, I_i, \boldsymbol{x}_i, R_i, \boldsymbol{v}_i = 0, \boldsymbol{\omega}_i = 0$

$\boldsymbol{a} = 0, \boldsymbol{\alpha} = 0$

Gravity: $\boldsymbol{g}$

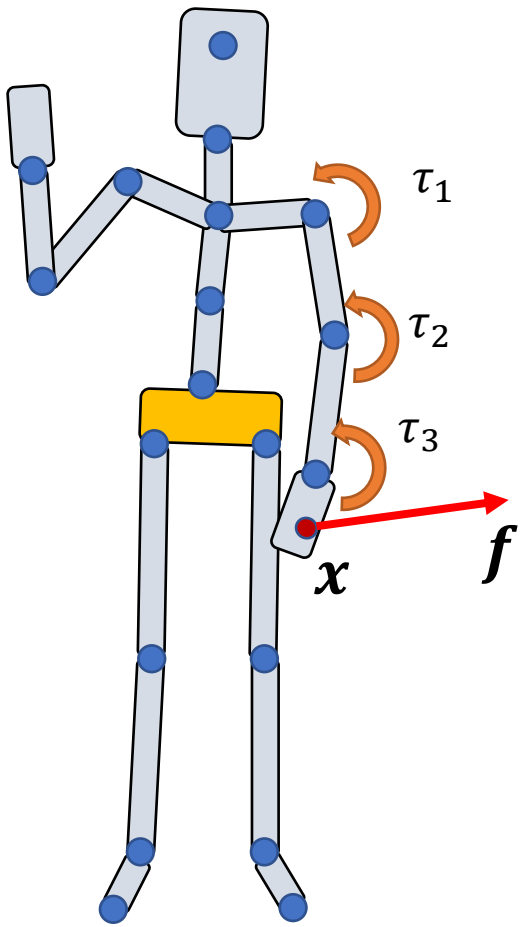
Compute: $\boldsymbol{f}_k, \boldsymbol{\tau}_k$

$$\boldsymbol{f}_3 = -m_3\boldsymbol{g} \qquad \boldsymbol{\tau}_3 = \boldsymbol{r}_3 \times m_3\boldsymbol{g}$$

$$\boldsymbol{f}_2 = -m_2\boldsymbol{g} - m_3\boldsymbol{g} \qquad \boldsymbol{\tau}_2 = \boldsymbol{r}_2 \times m_2\boldsymbol{g} + (\boldsymbol{r}_2 - \boldsymbol{d}_2 + \boldsymbol{r}_3) \times m_3\boldsymbol{g}$$

$$I_i\dot{\boldsymbol{\omega}}_i + \boldsymbol{\omega}_i \times I_i\boldsymbol{\omega}_i = \boldsymbol{\tau}_i + \boldsymbol{r}_i \times \boldsymbol{f}_i - \sum_{k \in c(i)} (\boldsymbol{\tau}_k + \boldsymbol{d}_{ik} \times \boldsymbol{f}_k) + \sum_j \boldsymbol{\tau}_j^{\text{ext}}$$

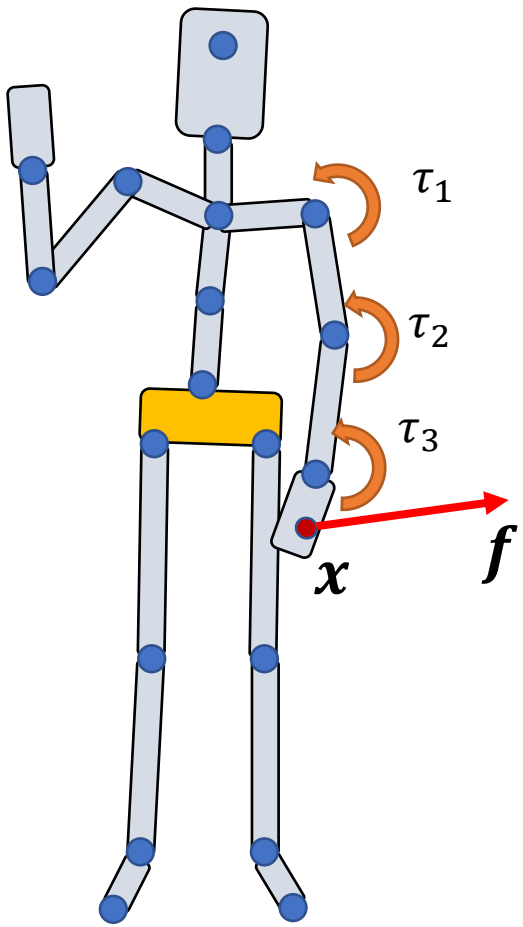
Jacobian Transpose Control



Can we use joint torques $\boldsymbol{\tau}_i$ to mimic the effect of a force $\boldsymbol{f}$ applied at $\boldsymbol{x}$

- Note that the desired force $\boldsymbol{f}$ is not actually applied
- Also called "virtual force"

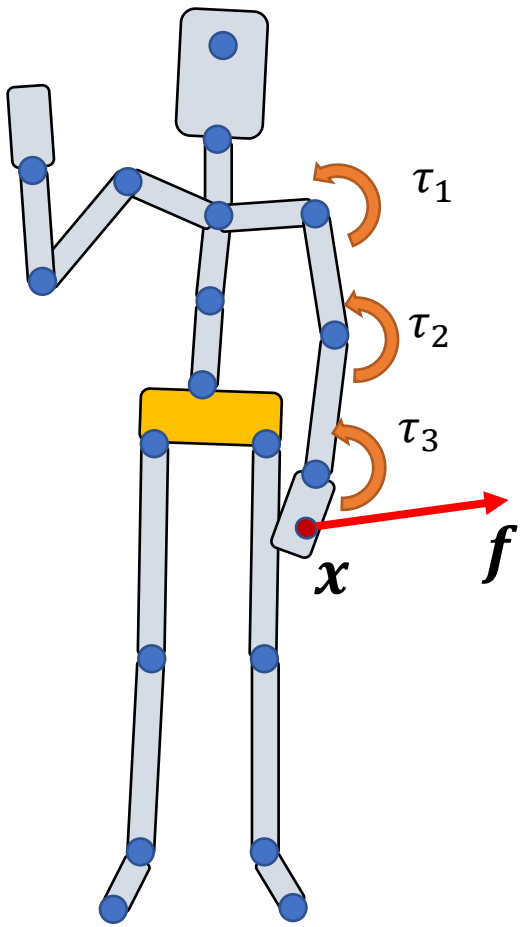
Jacobian Transpose Control



Can we use joint torques $\boldsymbol{\tau}_i$ to mimic the **effect** of a force $\boldsymbol{f}$ applied at $\boldsymbol{x}$

Make $\boldsymbol{f}$ and $\boldsymbol{\tau}_i$ done the same work

Jacobian Transpose Control

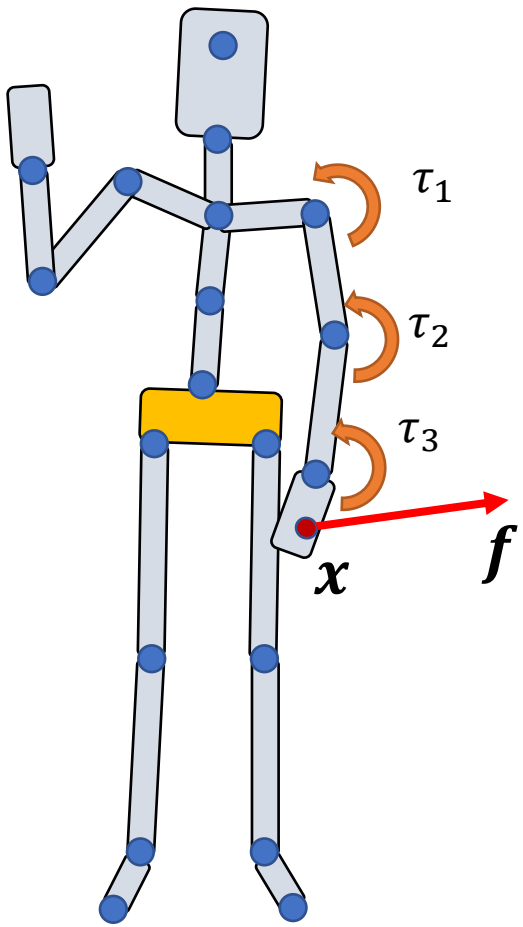


Can we use joint torques $\boldsymbol{\tau}_i$ to mimic the **effect** of a force $\boldsymbol{f}$ applied at $\boldsymbol{x}$

Make $\boldsymbol{f}$ and $\boldsymbol{\tau}_i$ done the same power

$$P = \boldsymbol{f}^T \dot{\boldsymbol{x}} = \boldsymbol{\tau}^T \dot{\boldsymbol{\theta}}$$

Jacobian Transpose Control



Can we use joint torques $\boldsymbol{\tau}_i$ to mimic the **effect** of a force $\boldsymbol{f}$ applied at $\boldsymbol{x}$

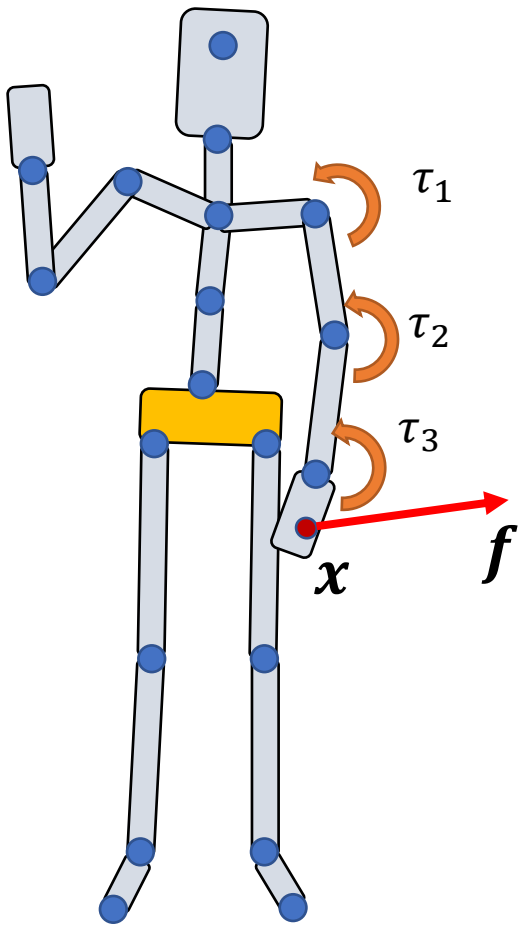
Make $\boldsymbol{f}$ and $\boldsymbol{\tau}_i$ done the same power

$$P = \boldsymbol{f}^T \dot{\boldsymbol{x}} = \boldsymbol{\tau}^T \dot{\boldsymbol{\theta}}$$

Forward kinematics $\boldsymbol{x} = g(\boldsymbol{\theta}) \Rightarrow \dot{\boldsymbol{x}} = J\dot{\boldsymbol{\theta}}$

$$J = \frac{\partial g}{\partial \boldsymbol{\theta}}$$

Jacobian Transpose Control



Can we use joint torques $\boldsymbol{\tau}_i$ to mimic the **effect** of a force $\boldsymbol{f}$ applied at $\boldsymbol{x}$

Make $\boldsymbol{f}$ and $\boldsymbol{\tau}_i$ done the same power

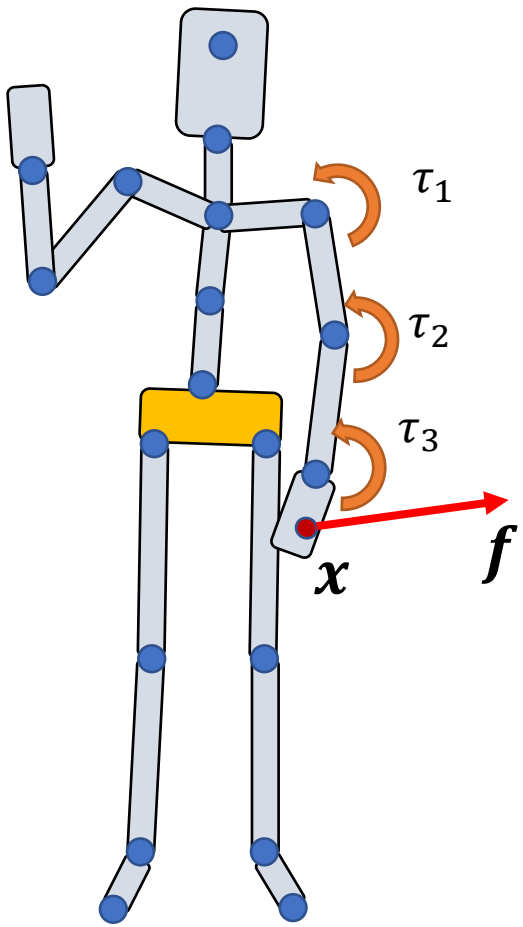
$$P = \boldsymbol{f}^T \dot{\boldsymbol{x}} = \boldsymbol{\tau}^T \dot{\boldsymbol{\theta}}$$

Forward kinematics $\boldsymbol{x} = g(\boldsymbol{\theta}) \Rightarrow \dot{\boldsymbol{x}} = J\dot{\boldsymbol{\theta}}$

$$\boldsymbol{f}^T J \dot{\boldsymbol{\theta}} = \boldsymbol{\tau}^T \dot{\boldsymbol{\theta}}$$

$$J = \frac{\partial g}{\partial \boldsymbol{\theta}}$$

Jacobian Transpose Control



Can we use joint torques $\boldsymbol{\tau}_i$ to mimic the **effect** of a force $\boldsymbol{f}$ applied at $\boldsymbol{x}$

Make $\boldsymbol{f}$ and $\boldsymbol{\tau}_i$ done the same power

$$P = \boldsymbol{f}^T \dot{\boldsymbol{x}} = \boldsymbol{\tau}^T \dot{\boldsymbol{\theta}}$$

Forward kinematics $\boldsymbol{x} = g(\boldsymbol{\theta}) \Rightarrow \dot{\boldsymbol{x}} = J\dot{\boldsymbol{\theta}}$

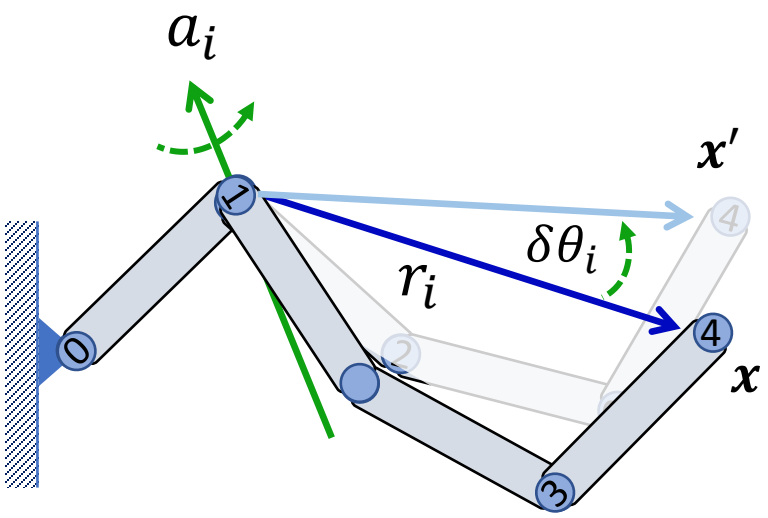
$$\boldsymbol{\tau} = J^T \boldsymbol{f}$$

$$J = \frac{\partial g}{\partial \boldsymbol{\theta}}$$

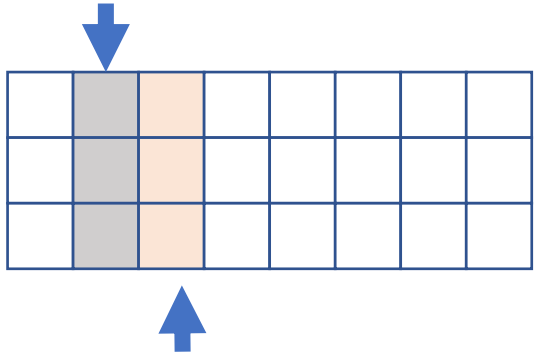
Recall: Geometric Approach

$$J = \frac{\partial g}{\partial \boldsymbol{\theta}} = \begin{pmatrix} \frac{\partial g}{\partial \theta_0} & \frac{\partial g}{\partial \theta_1} & \cdots & \frac{\partial g}{\partial \theta_n} \end{pmatrix}$$

Assuming all joints are hinge joint

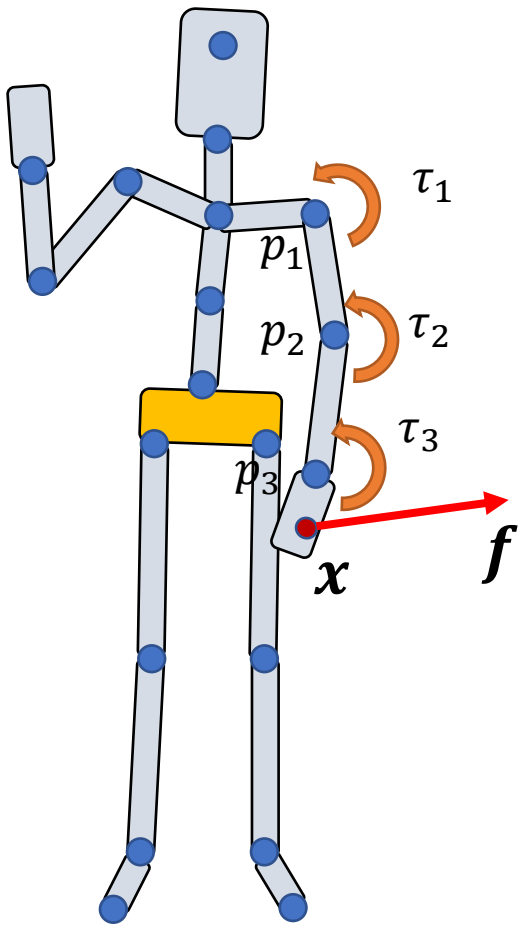


$$\frac{\partial g}{\partial \theta_1} = \boldsymbol{a}_1 \times \boldsymbol{r}_1$$



$$\frac{\partial f}{\partial \theta_2} = \boldsymbol{a}_2 \times \boldsymbol{r}_2$$

Jacobian Transpose Control



Can we use joint torques $\boldsymbol{\tau}_i$ to mimic the **effect** of a force $\boldsymbol{f}$ applied at $\boldsymbol{x}$

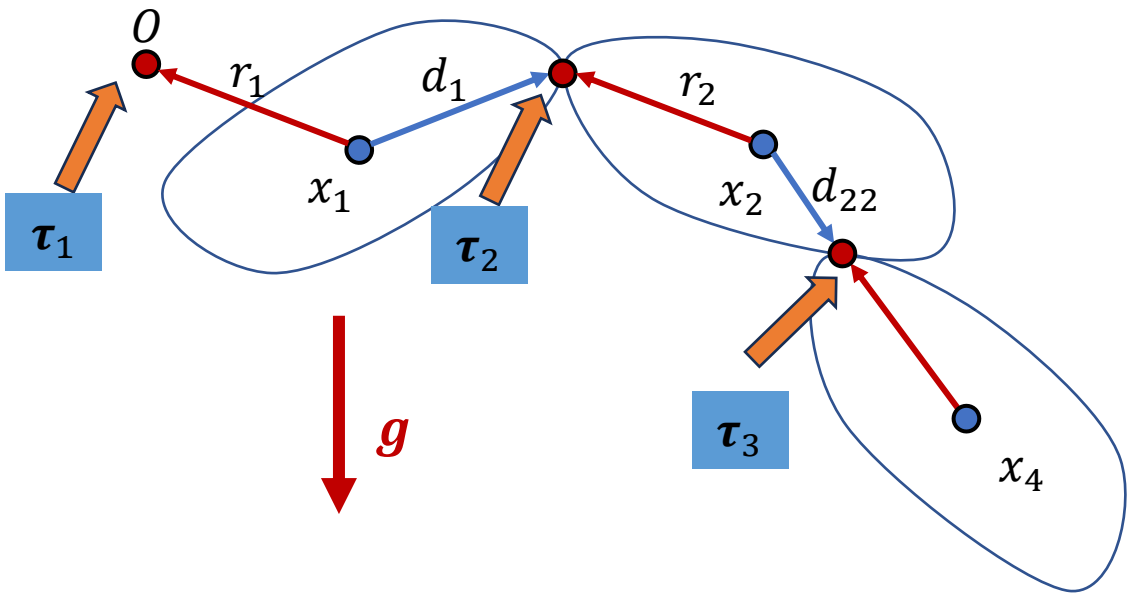
Make $\boldsymbol{f}$ and $\boldsymbol{\tau}_i$ done the same power

$$\boldsymbol{\tau} = J^T \boldsymbol{f} \qquad J = \frac{\partial g}{\partial \boldsymbol{\theta}}$$

$$\boldsymbol{\tau}_i = (\boldsymbol{x} - \boldsymbol{p}_i) \times \boldsymbol{f}$$

Example: Gravity Compensation

$$[\boldsymbol{x}, \mathbf{0}, \mathbf{0}] \mapsto \boldsymbol{f}$$



$(\boldsymbol{t}_0, r_0, r_1, r_2, \dots\dots) \Rightarrow \boldsymbol{x}_i, R_i$

$(\dot{\boldsymbol{t}}_0, \dot{r}_0, \dot{r}_1, \dot{r}_2, \dots\dots) \Rightarrow \boldsymbol{v}_i, \boldsymbol{\omega}_i$

Given: $m_i, I_i, \boldsymbol{x}_i, R_i, \boldsymbol{v}_i = 0, \boldsymbol{\omega}_i = 0$

$\boldsymbol{a} = 0, \boldsymbol{\alpha} = 0$

Gravity: $\boldsymbol{g}$

Compute: $\boldsymbol{\tau}_k$

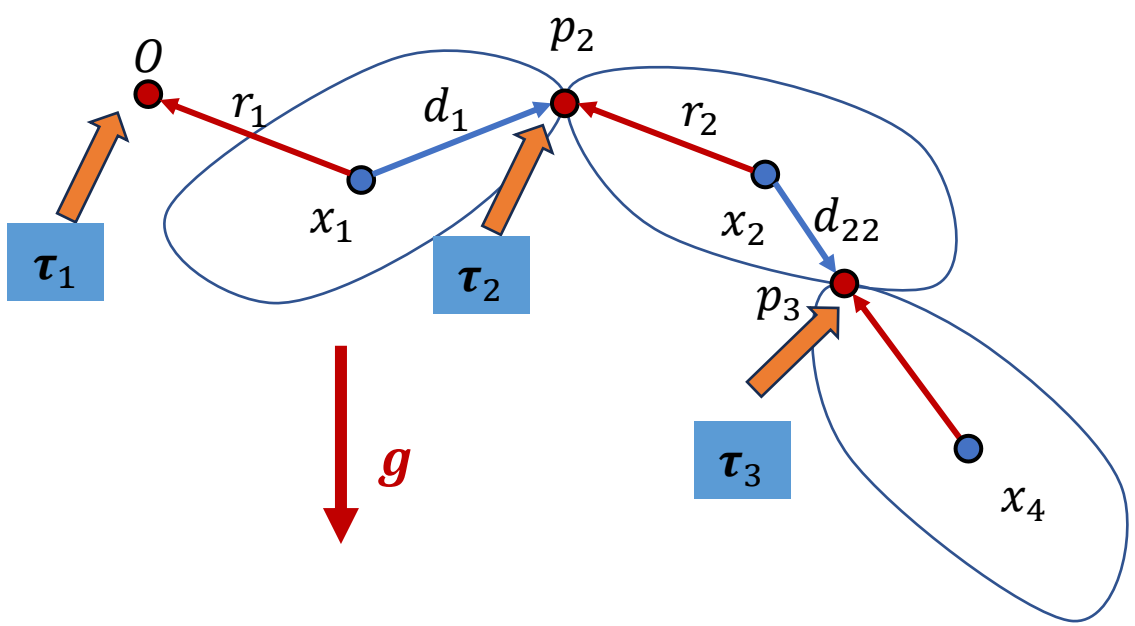
$$\boldsymbol{\tau}_3 = \boldsymbol{r}_3 \times m_3 \boldsymbol{g}$$

$$\boldsymbol{\tau}_2 = \boldsymbol{r}_2 \times m_2 \boldsymbol{g} + (\boldsymbol{r}_2 - \boldsymbol{d}_2 + \boldsymbol{r}_3) \times m_3 \boldsymbol{g}$$

$$\boldsymbol{\tau}_1 = \boldsymbol{r}_1 \times m_1 \boldsymbol{g} + (\boldsymbol{r}_1 - \boldsymbol{d}_1 + \boldsymbol{r}_2) \times m_2 \boldsymbol{g} + (\boldsymbol{r}_1 - \boldsymbol{d}_1 + \boldsymbol{r}_2 - \boldsymbol{d}_2 + \boldsymbol{r}_3) \times m_3 \boldsymbol{g}$$

Example: Gravity Compensation

$$[\boldsymbol{x}, \mathbf{0}, \mathbf{0}] \mapsto \boldsymbol{f}$$



$(\boldsymbol{t}_0, r_0, r_1, r_2, \dots\dots) \Rightarrow \boldsymbol{x}_i, R_i$

$(\dot{\boldsymbol{t}}_0, \dot{r}_0, \dot{r}_1, \dot{r}_2, \dots\dots) \Rightarrow \boldsymbol{v}_i, \boldsymbol{\omega}_i$

Given: $m_i, I_i, \boldsymbol{x}_i, R_i, \boldsymbol{v}_i = 0, \boldsymbol{\omega}_i = 0$

$\boldsymbol{a} = 0, \boldsymbol{\alpha} = 0$

Gravity: $\boldsymbol{g}$

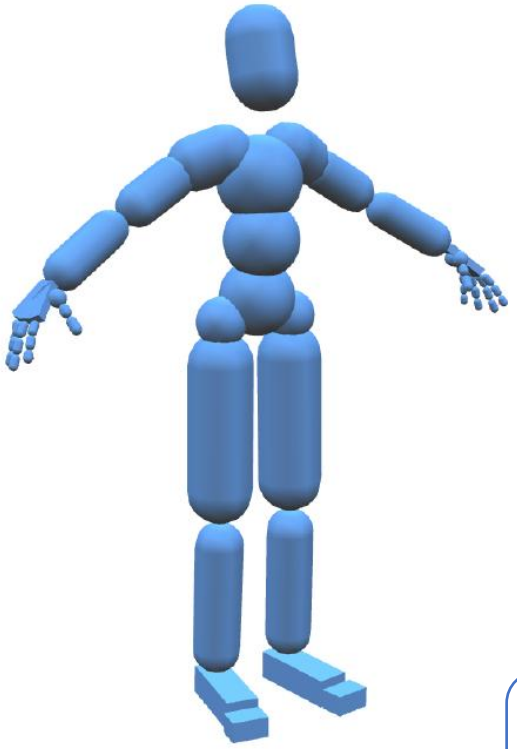
Compute: $\boldsymbol{\tau}_k$

$$\boldsymbol{\tau}_3 = (\boldsymbol{x}_4 - \boldsymbol{p}_3) \times (-m_3 \boldsymbol{g})$$

$$\boldsymbol{\tau}_2 = (\boldsymbol{x}_2 - \boldsymbol{p}_2) \times (-m_2 \boldsymbol{g}) + (\boldsymbol{x}_4 - \boldsymbol{p}_2) \times (-m_3 \boldsymbol{g})$$

$$\boldsymbol{\tau}_1 = (\boldsymbol{x}_1 - \boldsymbol{o}) \times (-m_1 \boldsymbol{g}) + (\boldsymbol{x}_2 - \boldsymbol{o}) \times (-m_2 \boldsymbol{g}) + (\boldsymbol{x}_4 - \boldsymbol{o}) \times (-m_3 \boldsymbol{g})$$

Simulating + Control



Current state:
$\boldsymbol{s}_t = (\boldsymbol{x}_t, \boldsymbol{v}_t, R_t, \boldsymbol{\omega}_t)$

Controller
π
$\pi: s_t, t \mapsto f, \tau$

f, τ
Joint torques

Dynamic Simulator

Next state:
$\boldsymbol{s}_{t+1} = (\boldsymbol{x}_{t+1}, \boldsymbol{v}_{t+1}, R_{t+1}, \boldsymbol{\omega}_{t+1})$

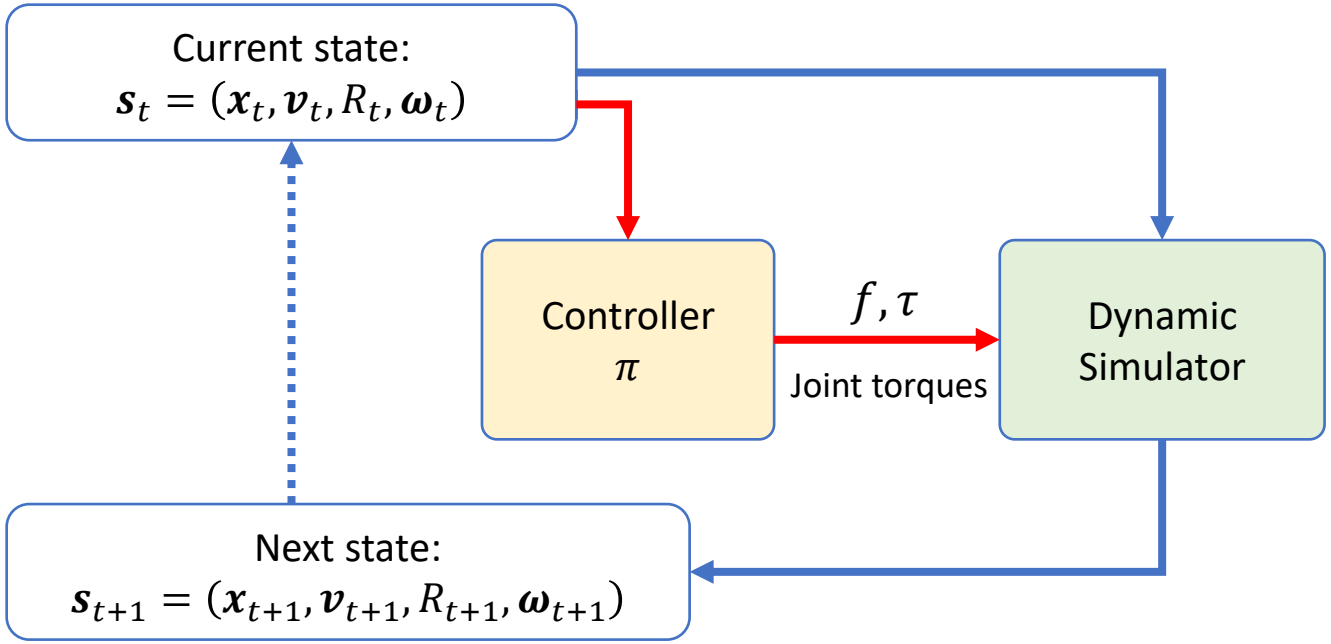
Feedforward vs. Feedback

Feedforward (Open-loop) control:

$$f, \tau = \pi(t)$$

- Apply predefined control signals without considering the current state of the system
- Assuming unchanging system. Perturbations may lead to unpredicted results

$\pi: s_t, t \mapsto f, \tau$



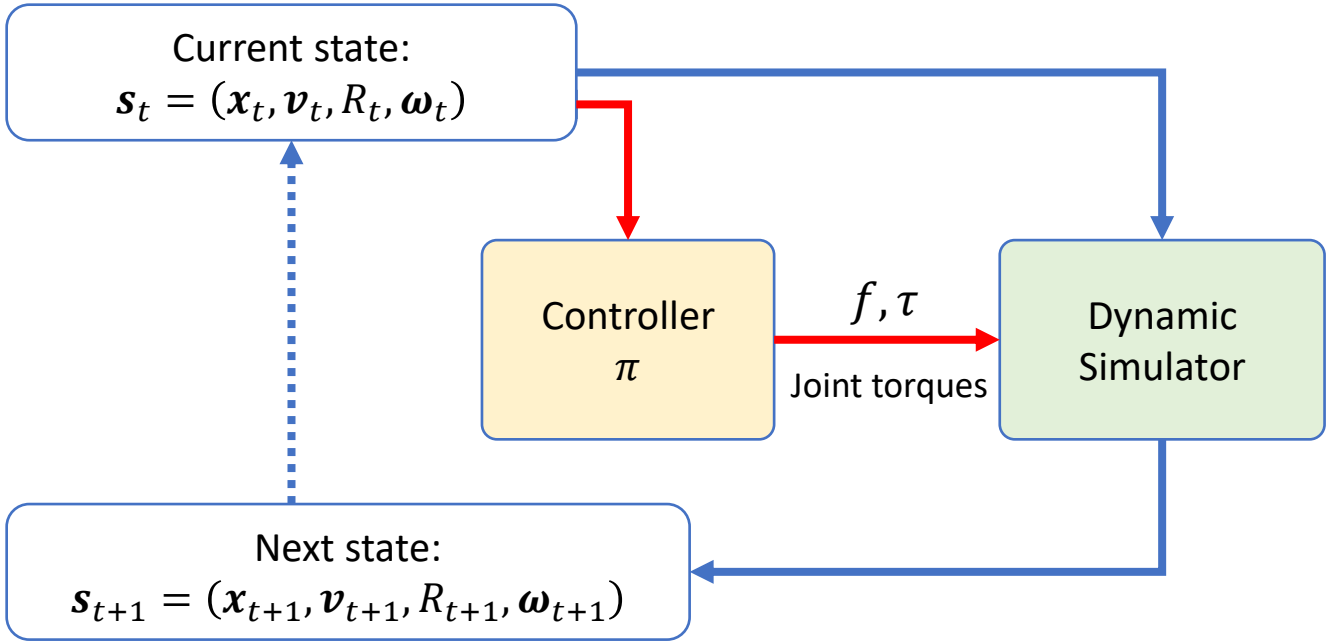
Feedforward vs. Feedback

Feedback control:

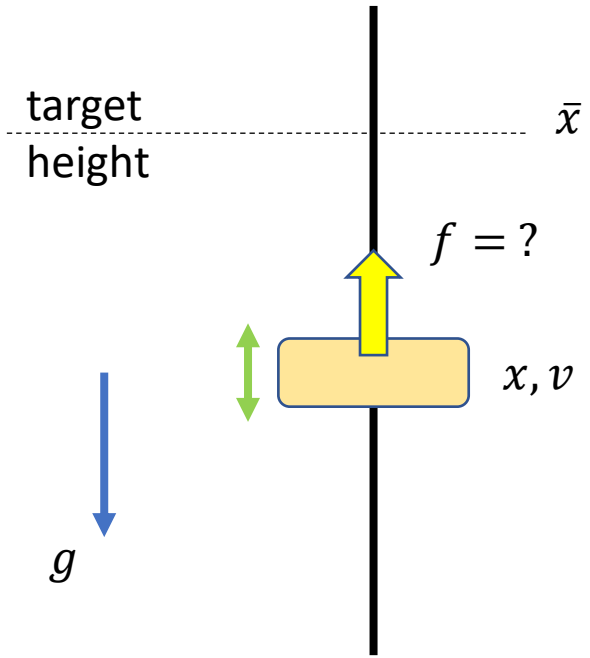
$$f, \tau = \pi(s_t, t)$$

- Adjust control signals based on the current state of the system
- Certain perturbations are expected. The feedback signal will be used to improves the performance at the next state.

$\pi: s_t, t \mapsto f, \tau$

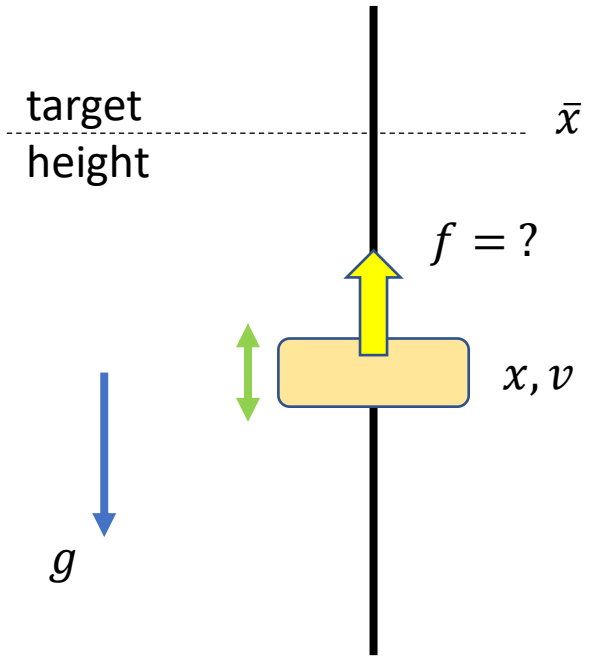


Proportional-Derivative Control



Compute force f to move
the object to the target height

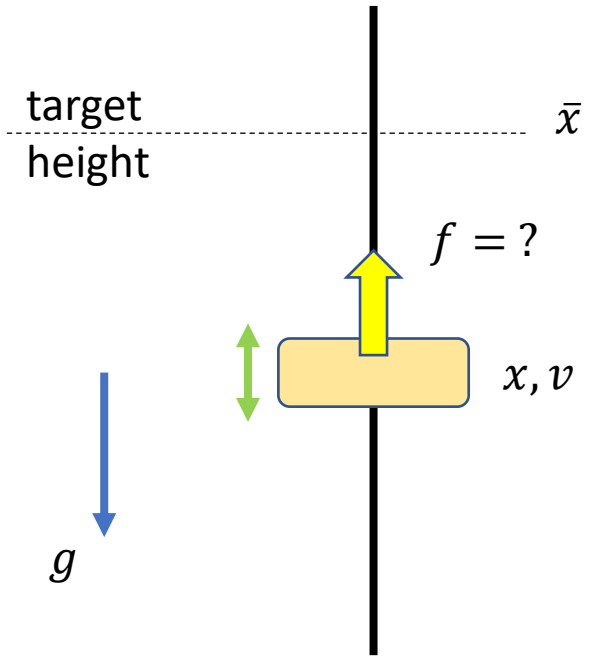
Proportional-Derivative Control



$$f = k_p(\bar{x} - x)$$

Compute force f to move
the object to the target height

Proportional-Derivative Control



Compute force f to move
the object to the target height

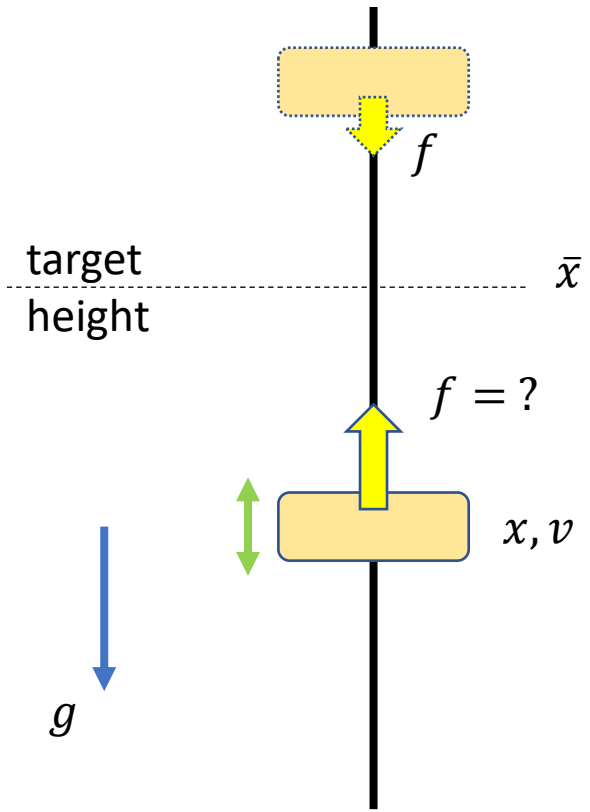
target
state

$$f = k_p(\bar{x} - x)$$

stiffness
(gain)

current
state

Proportional-Derivative Control



Compute force f to move
the object to the target height

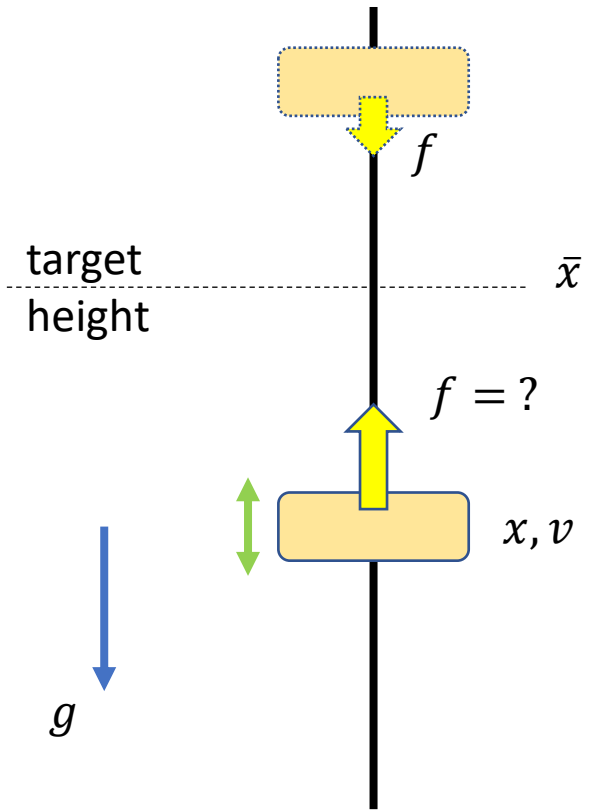
target
state

$$f = k_p(\bar{x} - x)$$

stiffness
(gain)

current
state

Proportional-Derivative Control



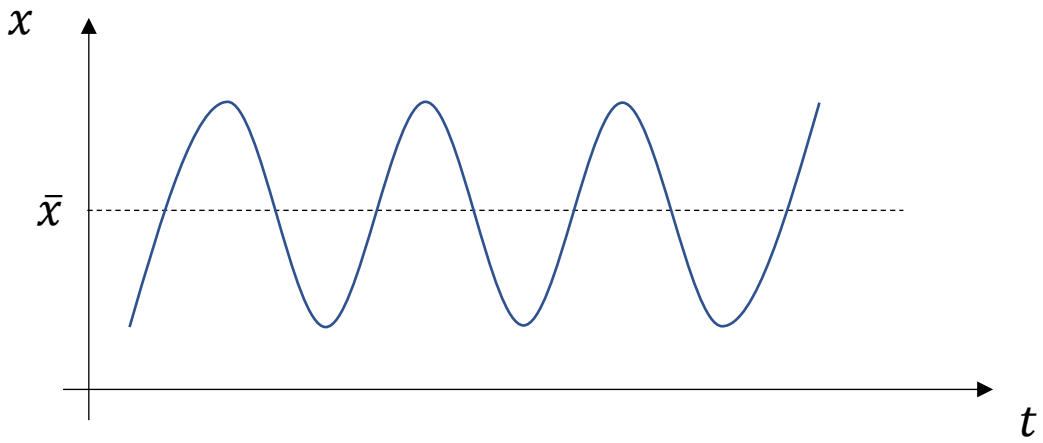
Compute force f to move
the object to the target height

target
state

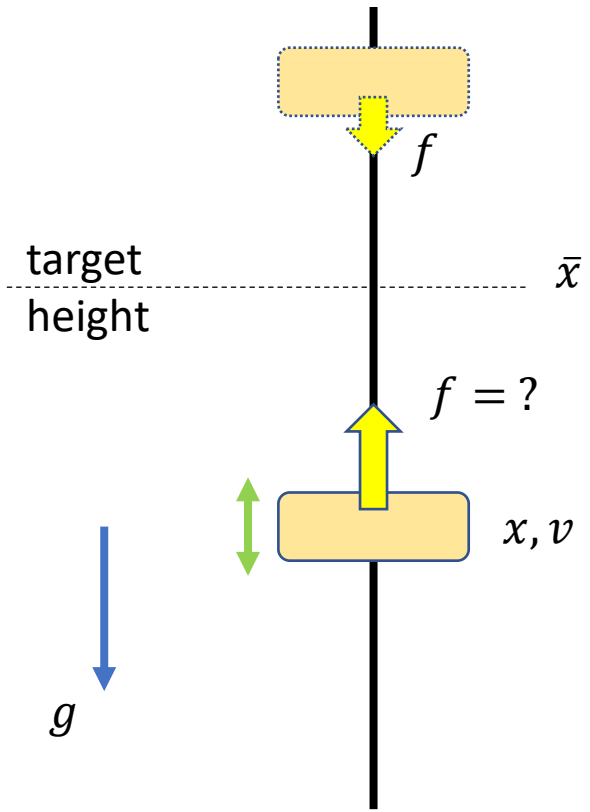
$$f = k_p(\bar{x} - x)$$

stiffness
(gain)

current
state



Proportional-Derivative Control



Compute force f to move
the object to the target height

target state

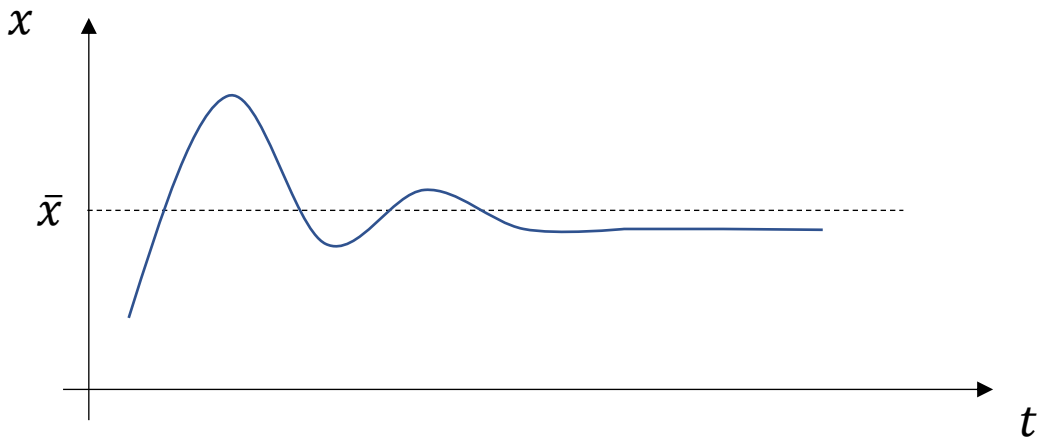
damping

$$f = k_p(\bar{x} - x) - k_d v$$

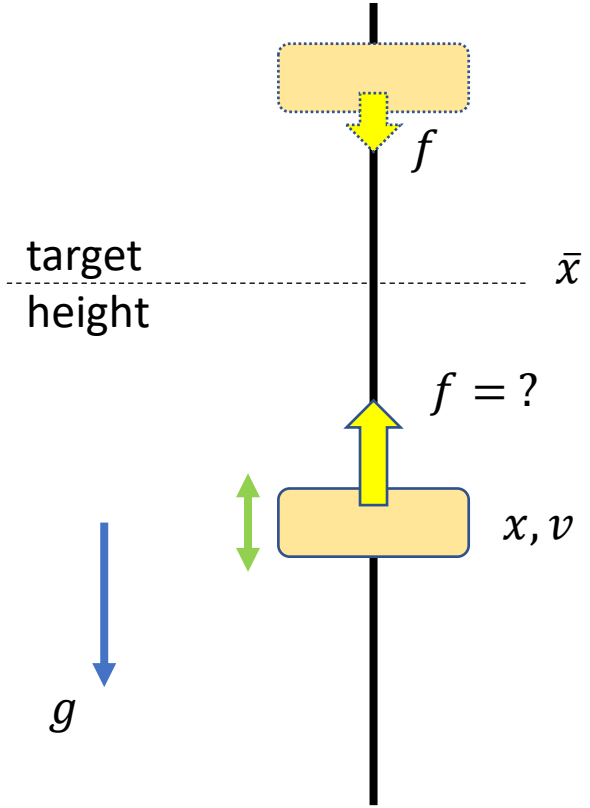
stiffness (gain)

current state

current velocity



Proportional-Derivative Control

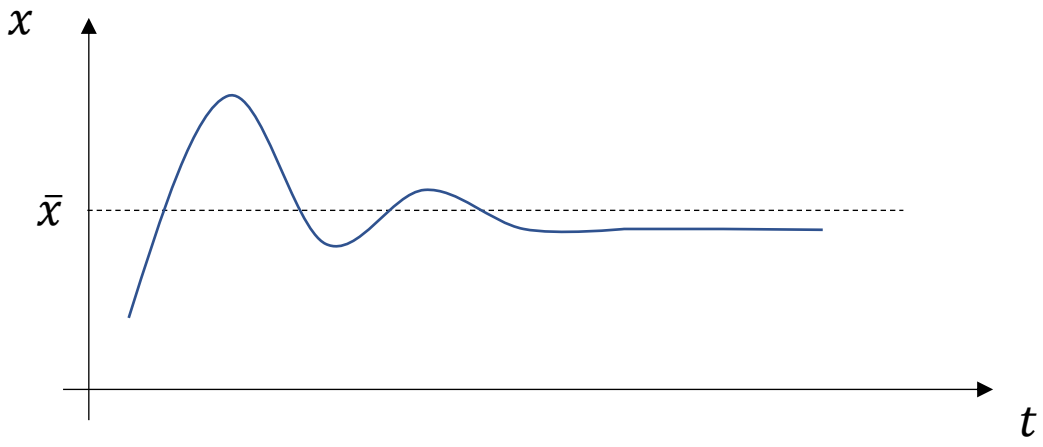


Compute force f to move
the object to the target height

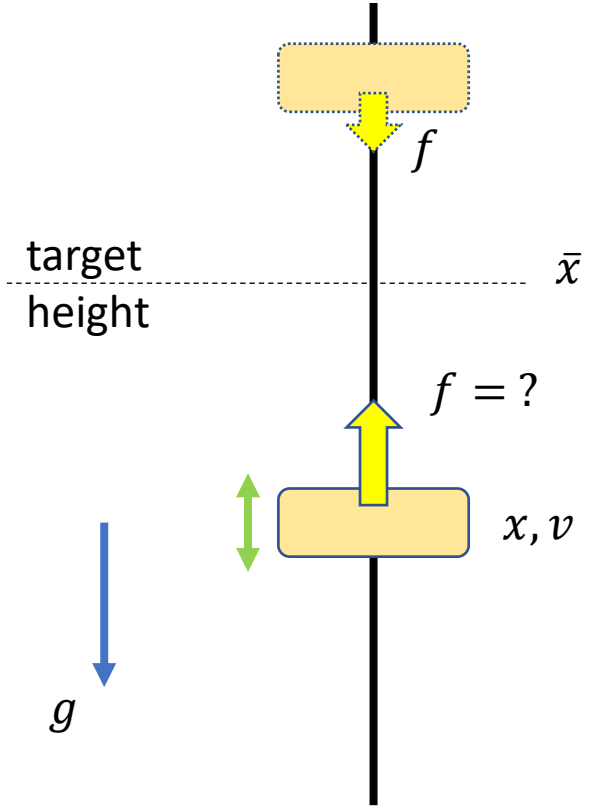
Let $e = \bar{x} - x$ be the error

$$f = k_p e + k_d \dot{e}$$

proportional control — derivative control



Proportional-Derivative Control

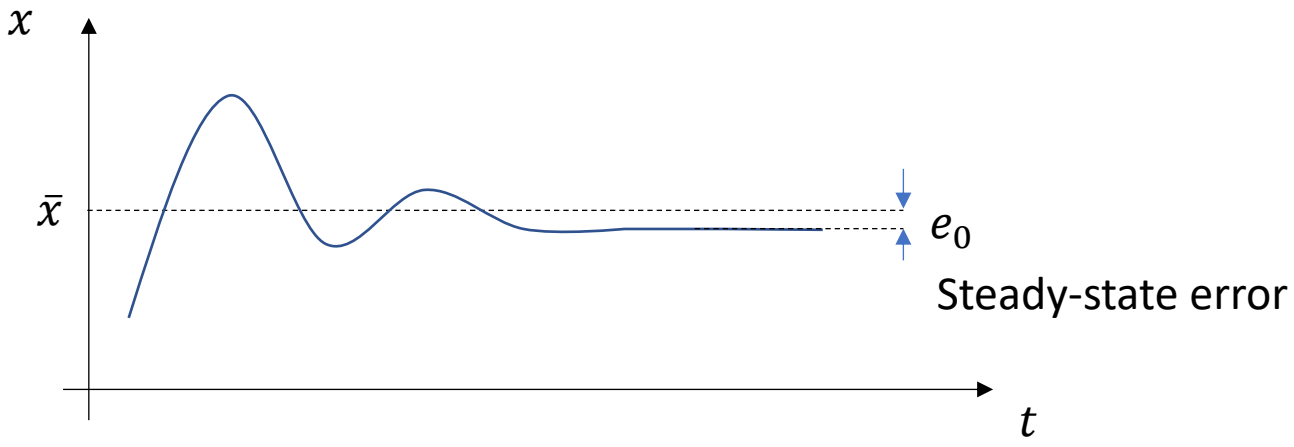


Compute force f to move
the object to the target height

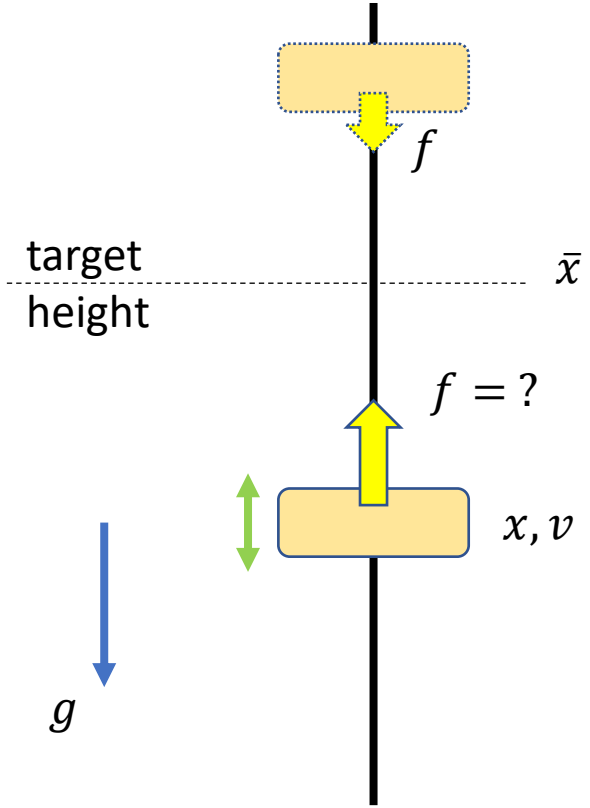
Let $e = \bar{x} - x$ be the error

$$f = k_p e + k_d \dot{e}$$

proportional control — derivative control



Proportional-Derivative Control

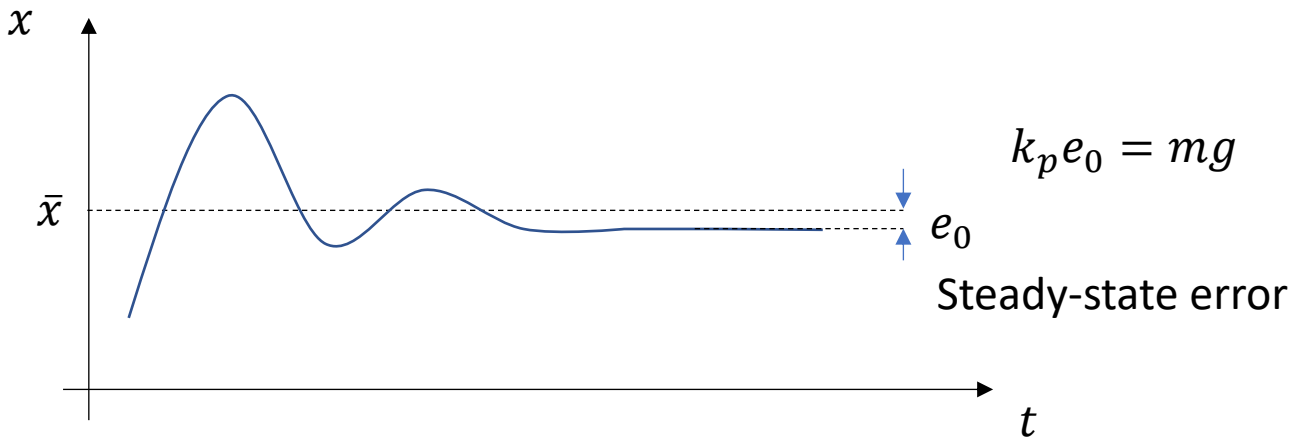


Compute force f to move
the object to the target height

Let $e = \bar{x} - x$ be the error

$$f = k_p e + k_d \dot{e}$$

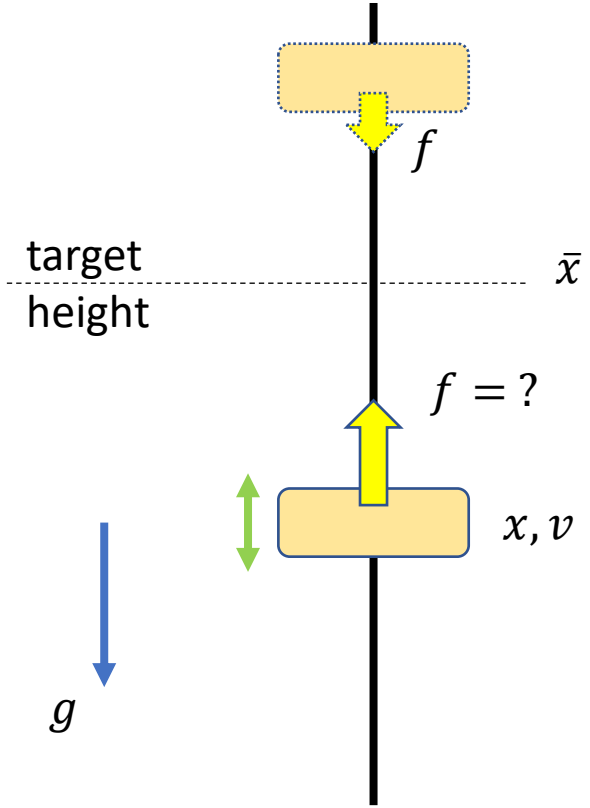
proportional control — derivative control



$k_p e_0 = mg$

Steady-state error

Proportional-Derivative Control



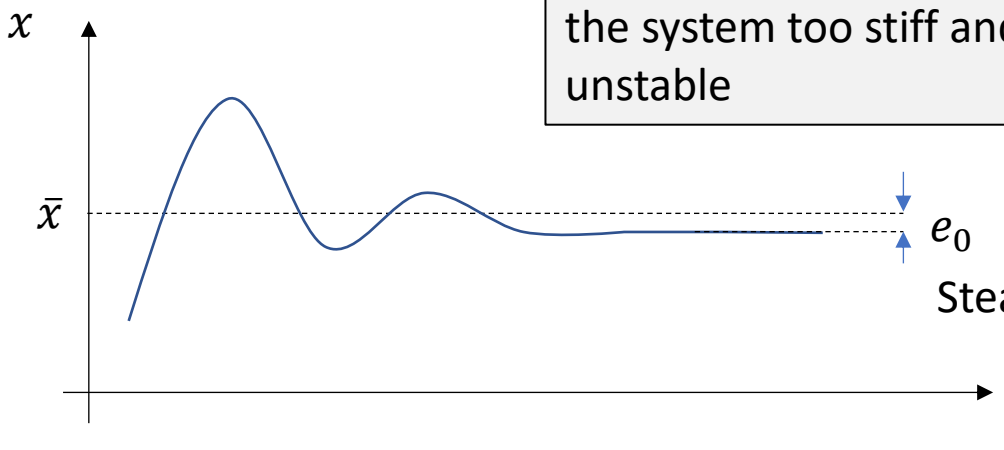
Compute force f to move the object to the target height

Let $e = \bar{x} - x$ be the error

$$f = k_p e + k_d \dot{e}$$

proportional control — derivative control

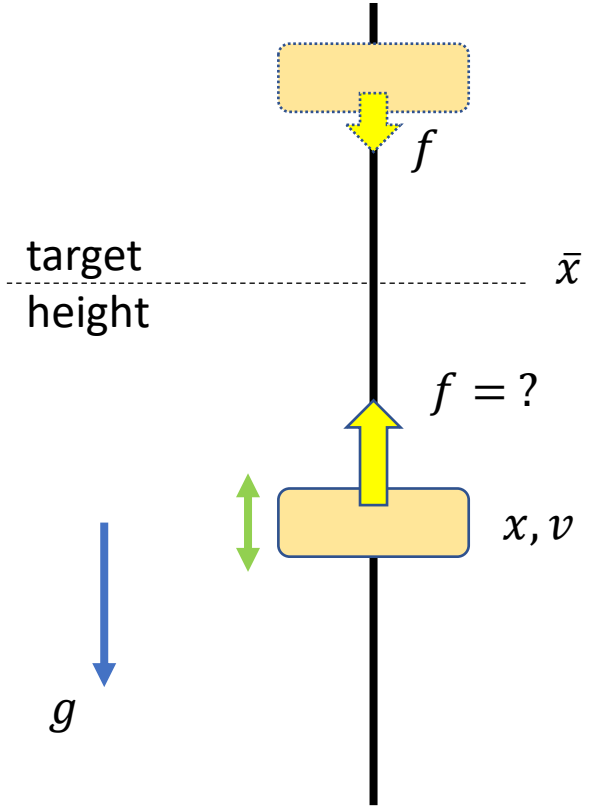
Increase stiffness k_p reduces the steady-state error, but can make the system too stiff and numerically unstable



$k_p e_0 = mg$

Steady-state error

Proportional-Integral-Derivative controller



Compute force f to move the object to the target height

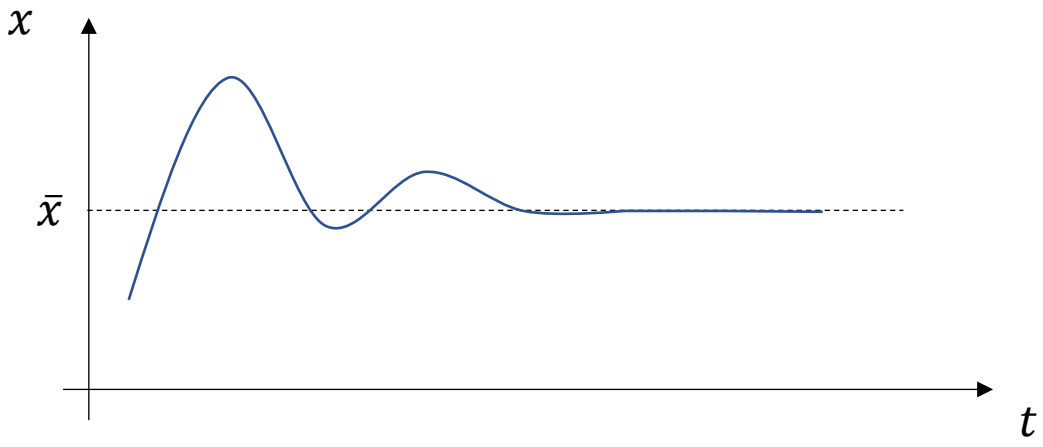
Let $e = \bar{x} - x$ be the error

$$f = k_p e + k_d \dot{e} + k_i \int_t e dt$$

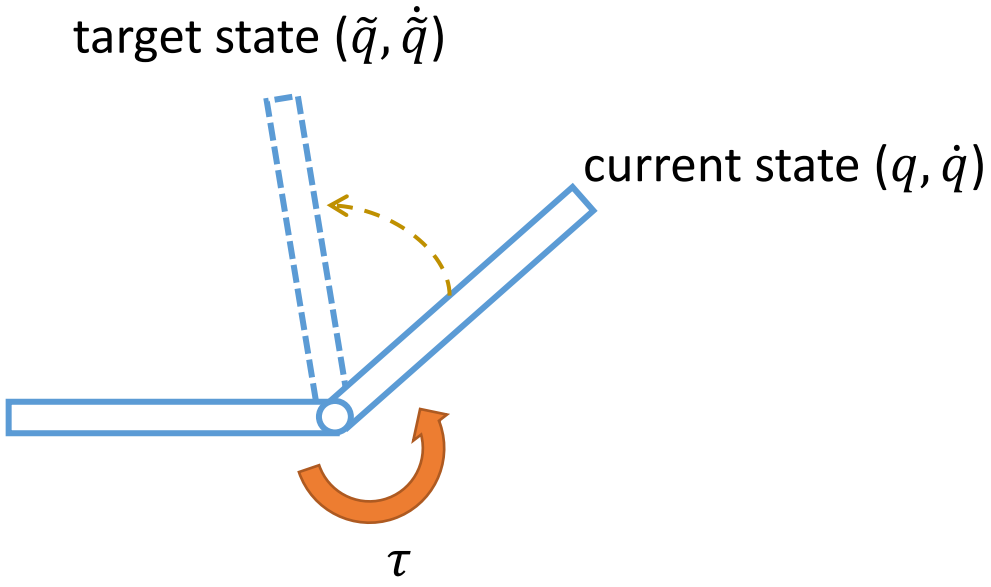
proportional control

derivative control

Integral control (rarely used in animation)



PD Control for Characters



target velocity
often zero

target pose

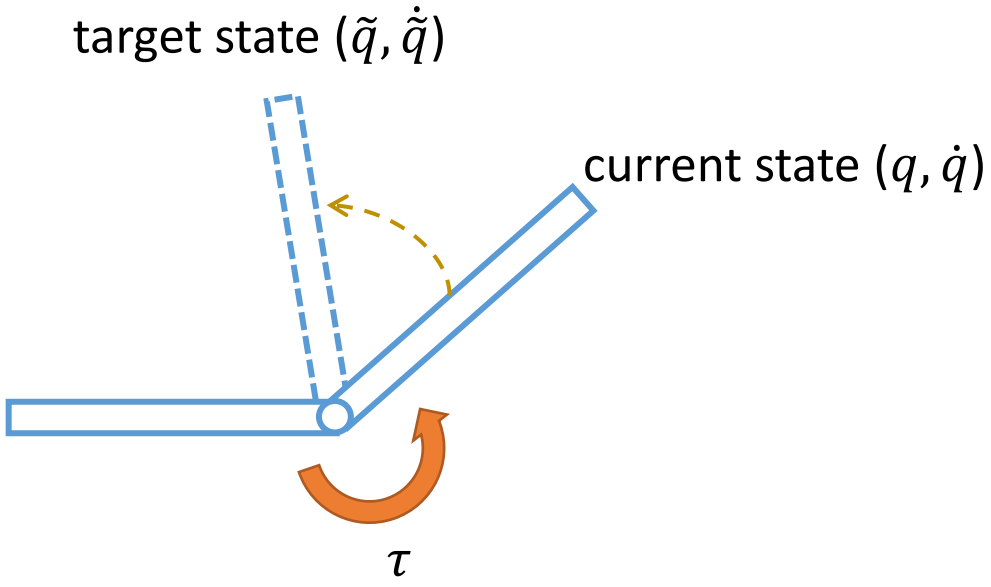
$$\tau = k_p(\bar{q} - q) + k_d(\dot{\bar{q}} - \dot{q})$$

joint torque

proportional control

derivative control

PD Control for Characters



target pose

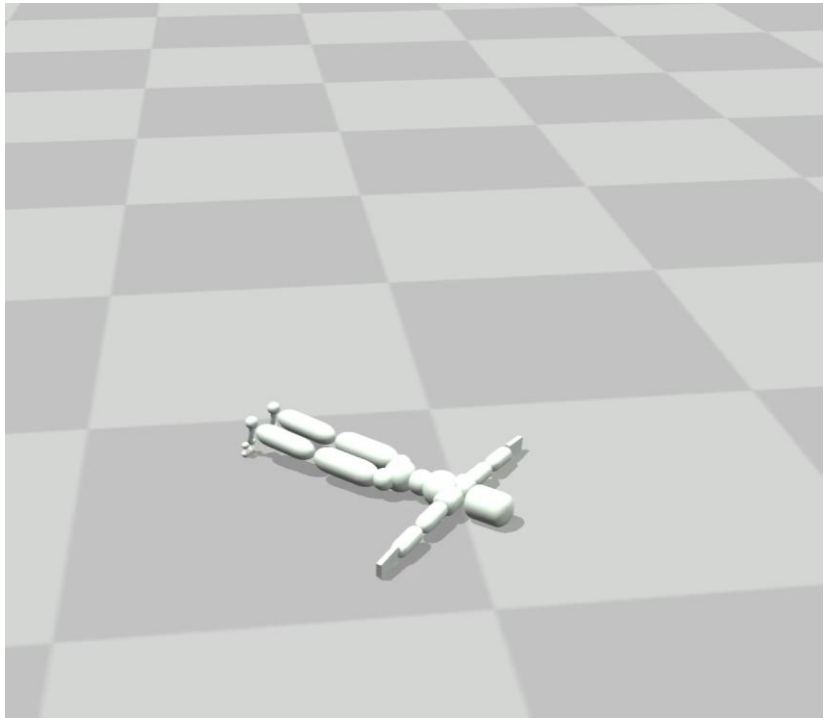
$$\tau = k_p(\bar{q} - q) - k_d\dot{q}$$

joint torque

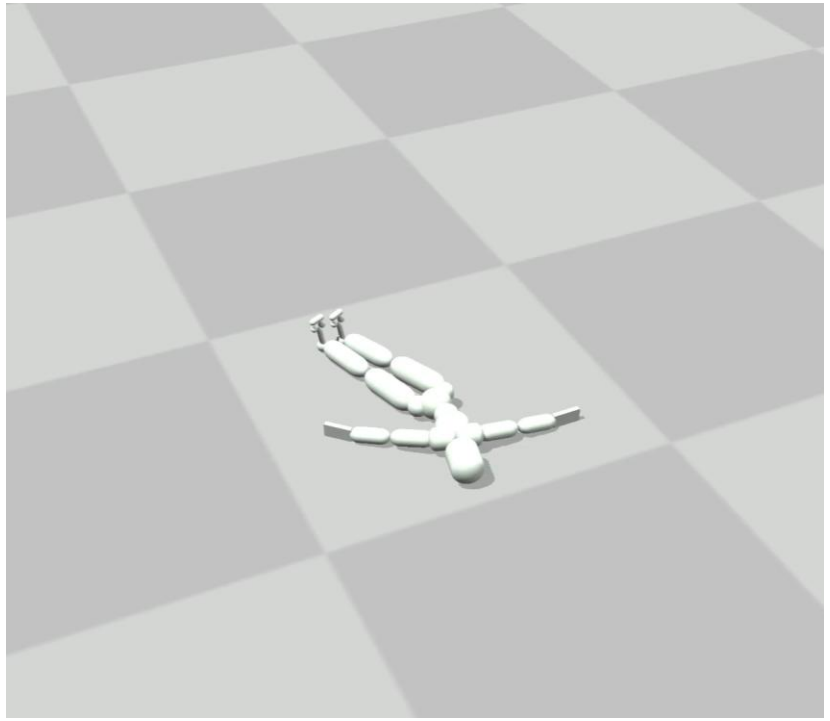
proportional control

derivative control

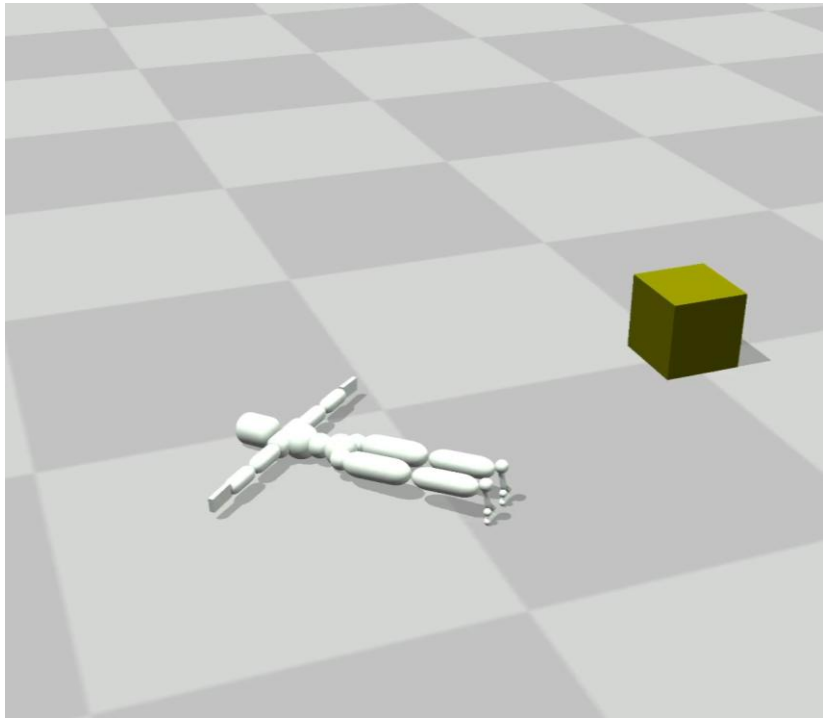
PD Control for Characters



Normal stiffness k_p

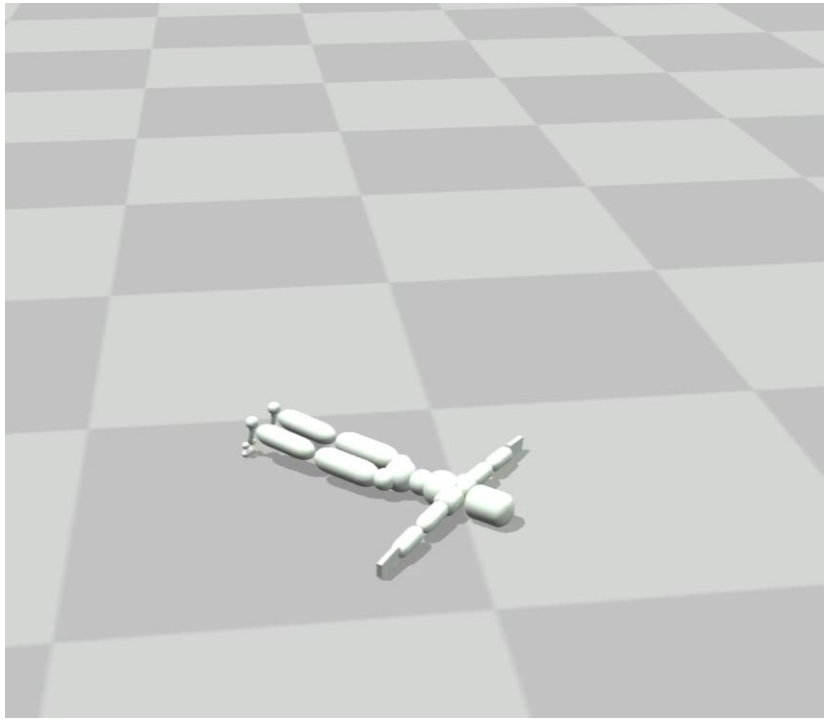


Small stiffness k_p

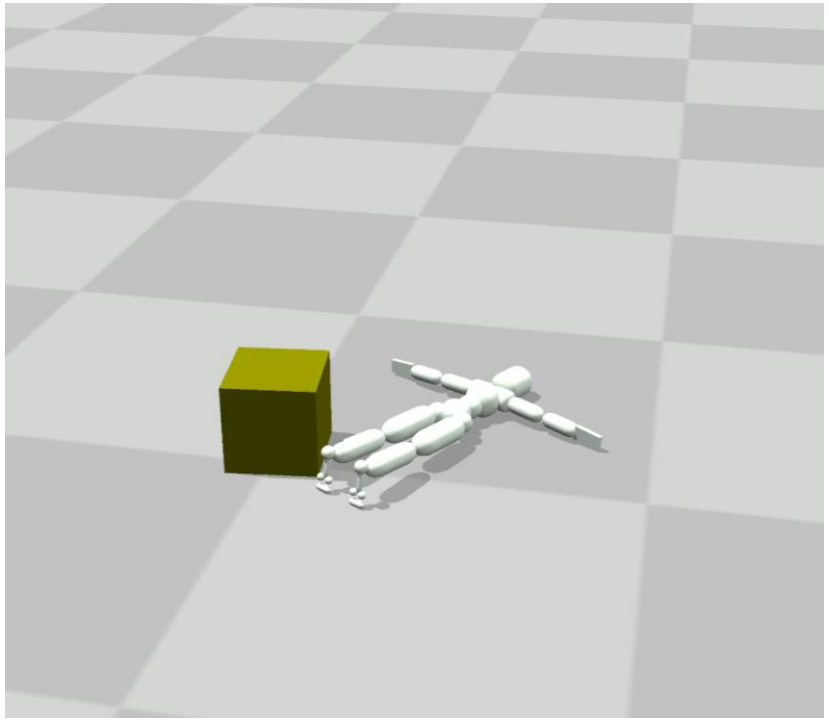


Large stiffness k_p

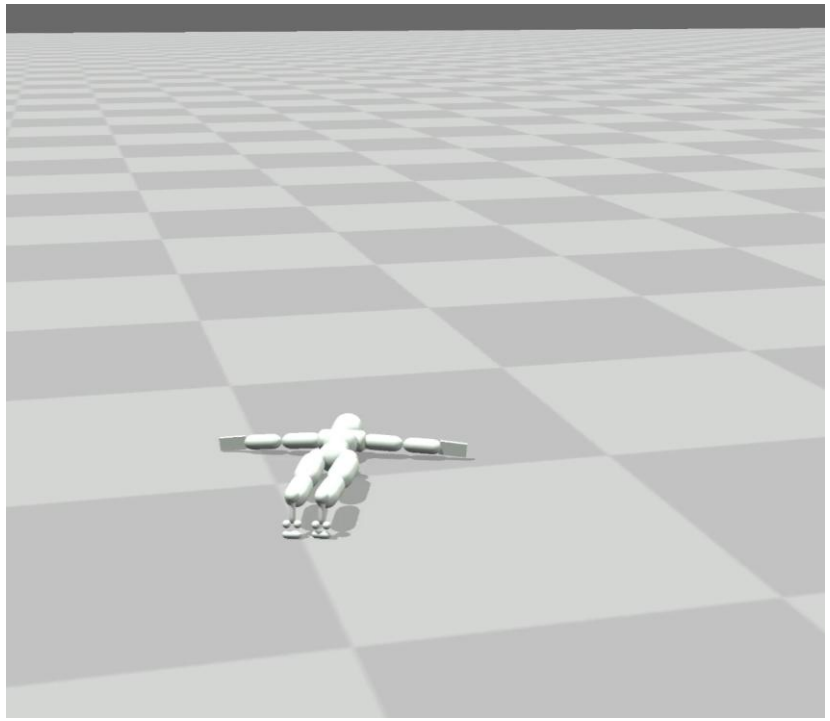
PD Control for Characters



Normal damping k_d

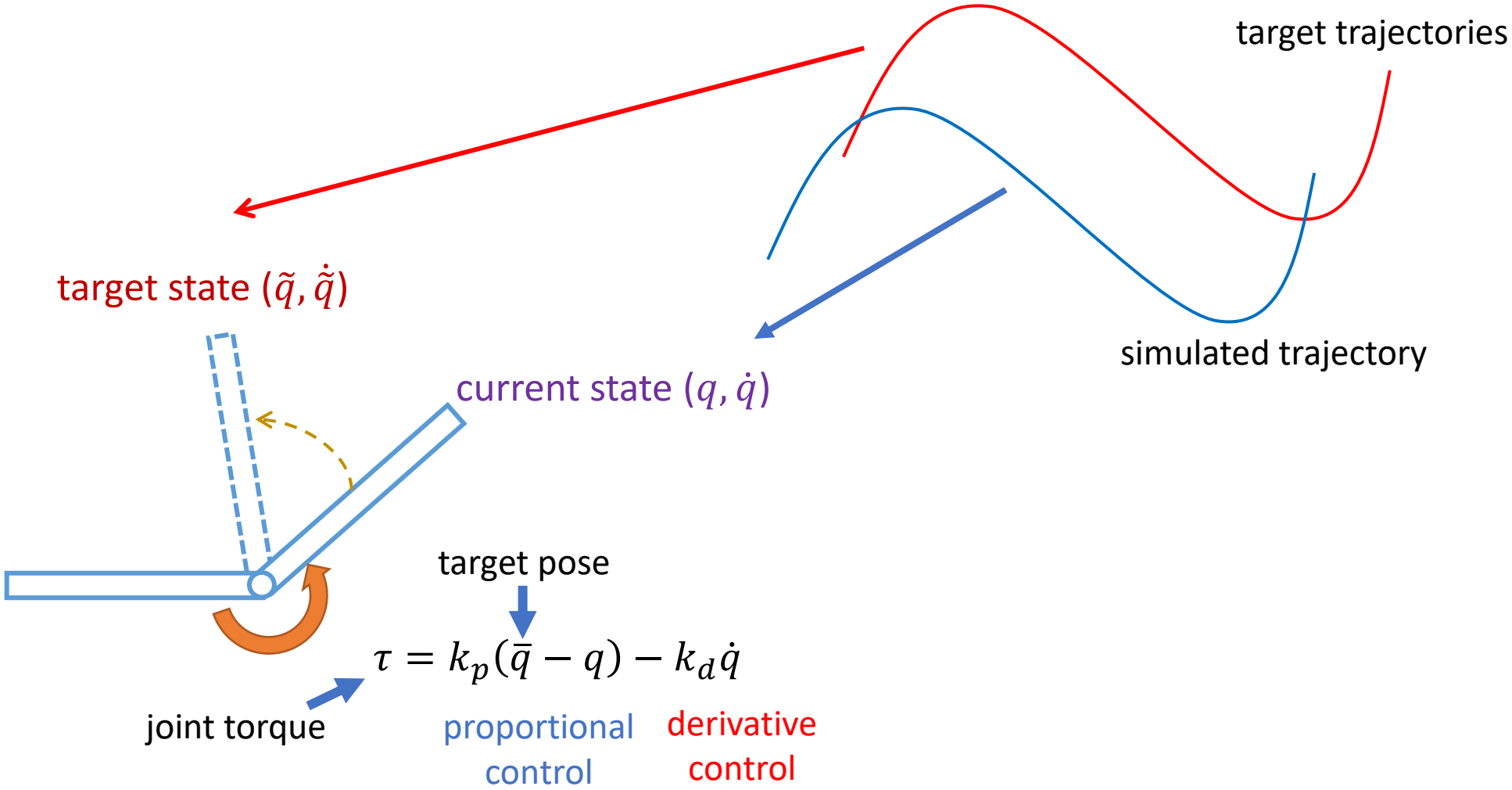


Small damping k_d

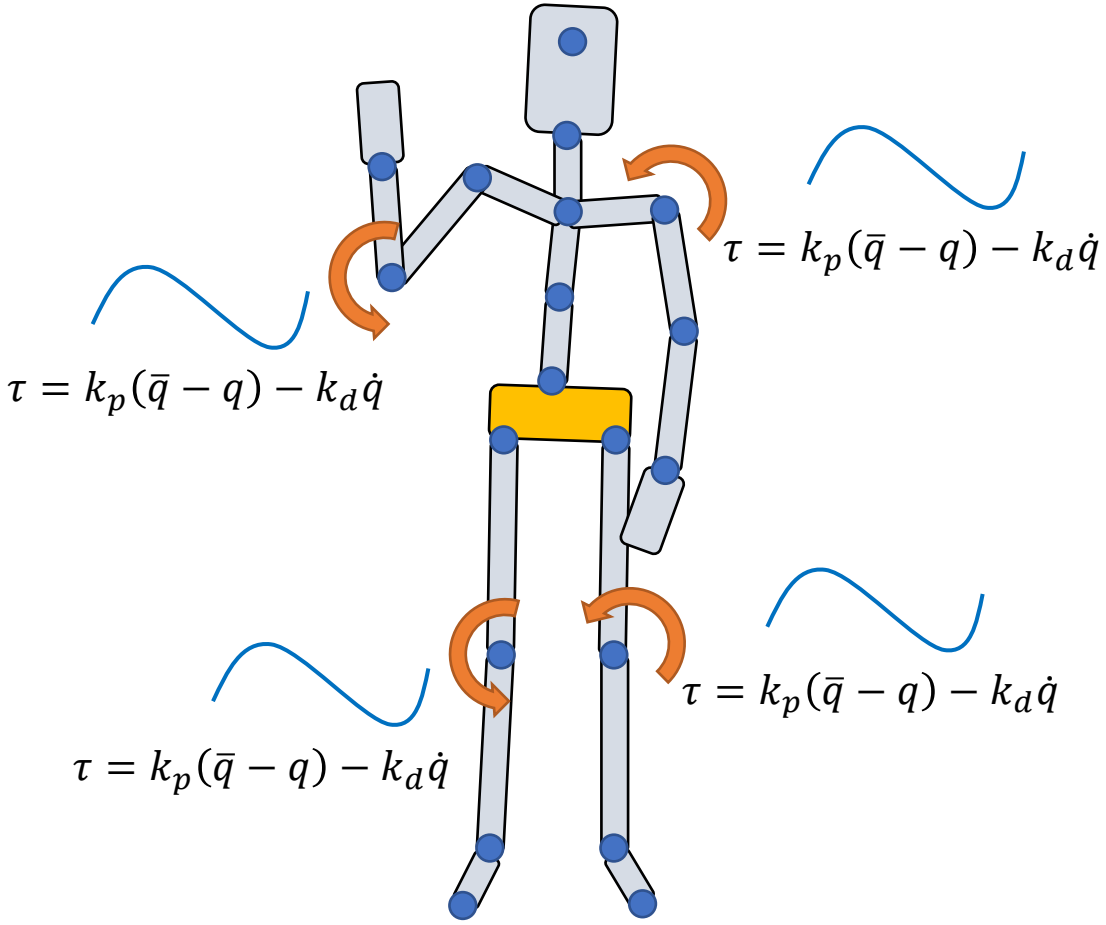


Large damping k_d

Tracking Controllers



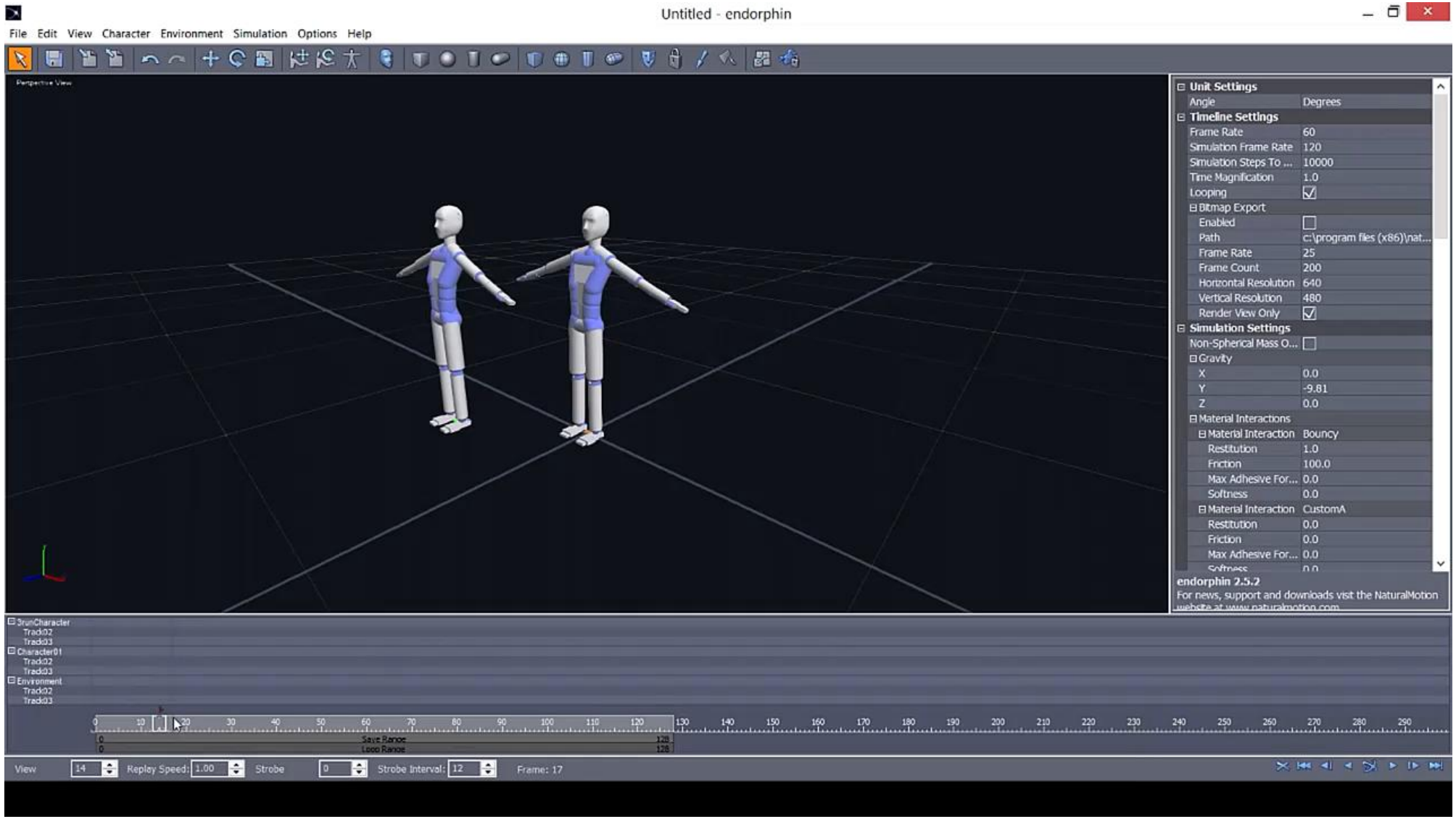
Full-body Tracking Controllers



Tracking a Trajectory

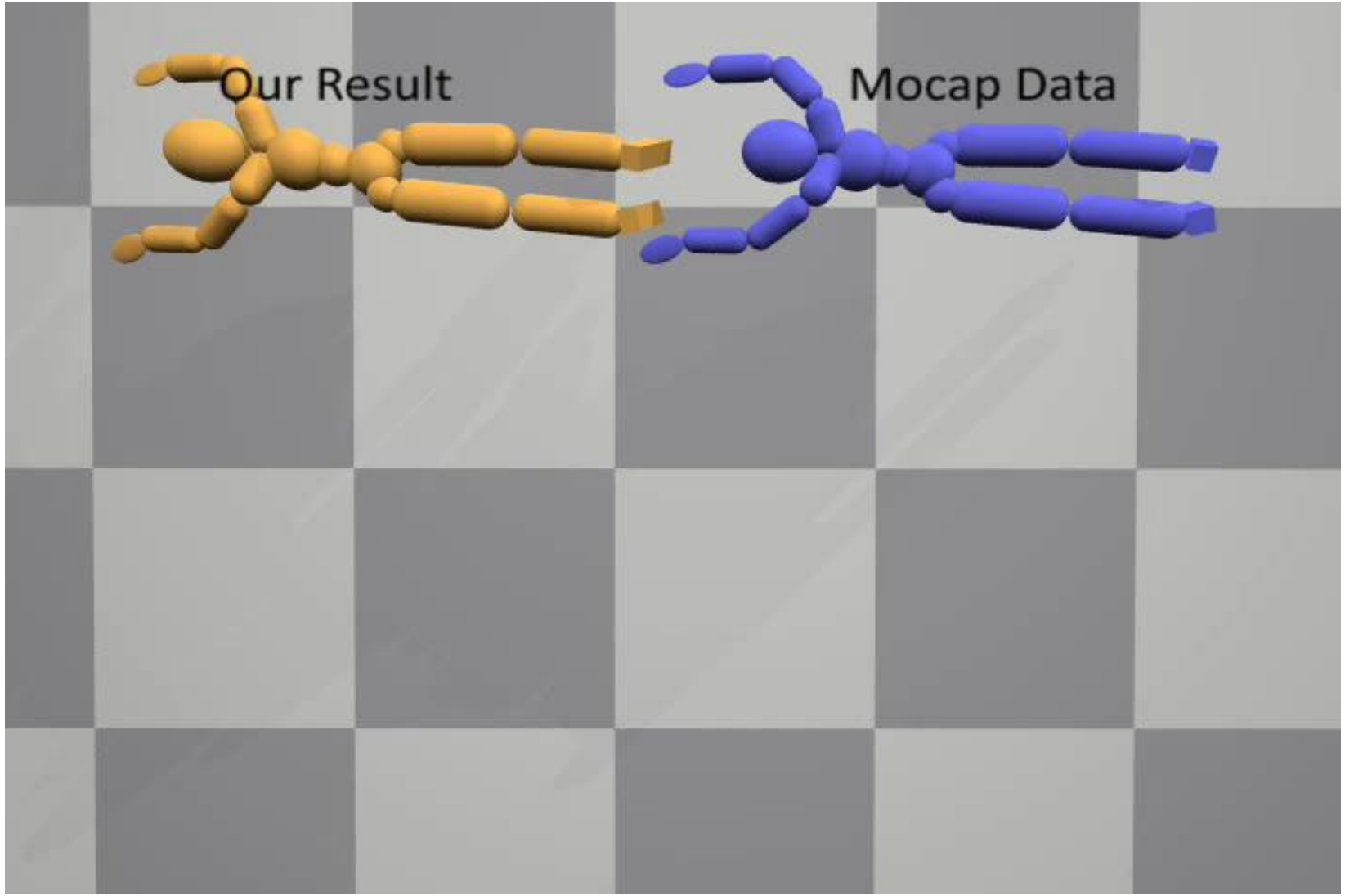
[Hodgins and Wooten 1995, Animating Human Athletics]

Trajectory Creation



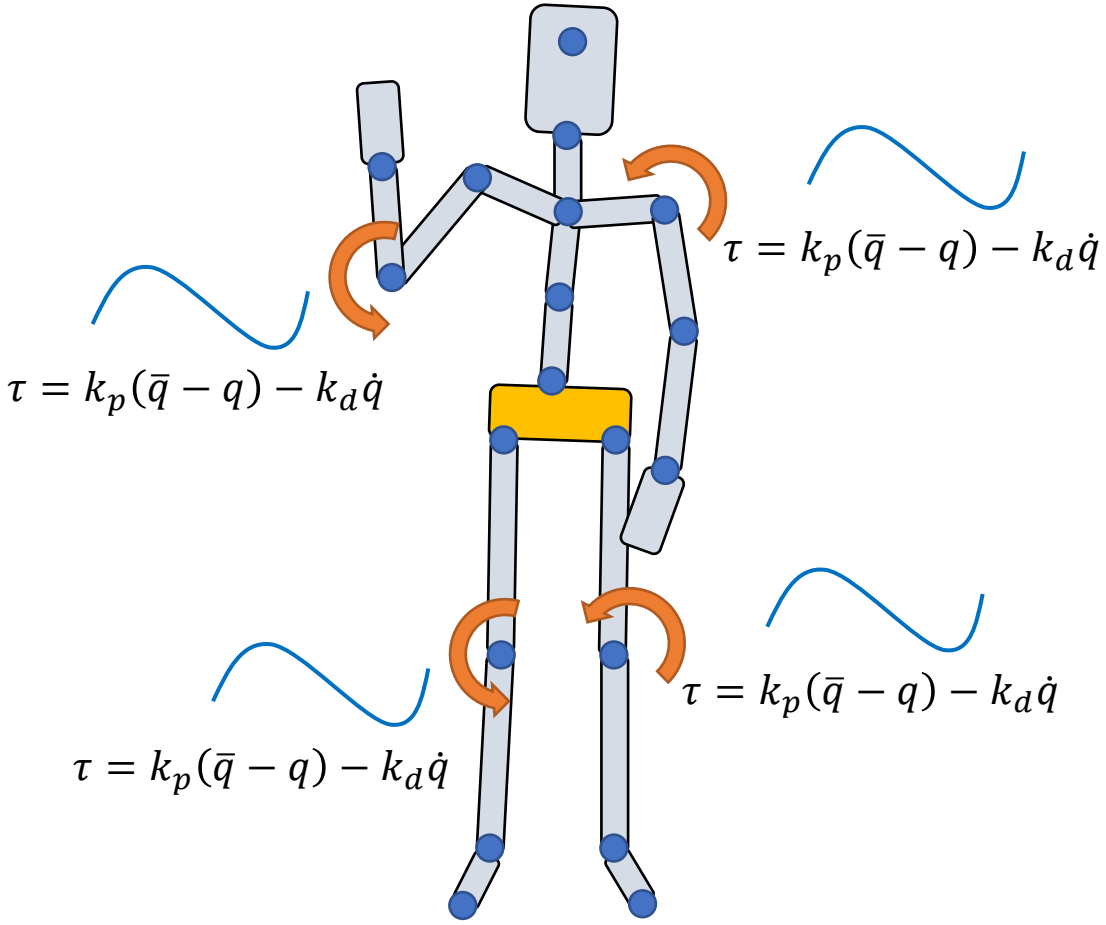
NaturalMotion - Endorphin

Tracking Mocap



[SAMCON – Liu et al 2010]

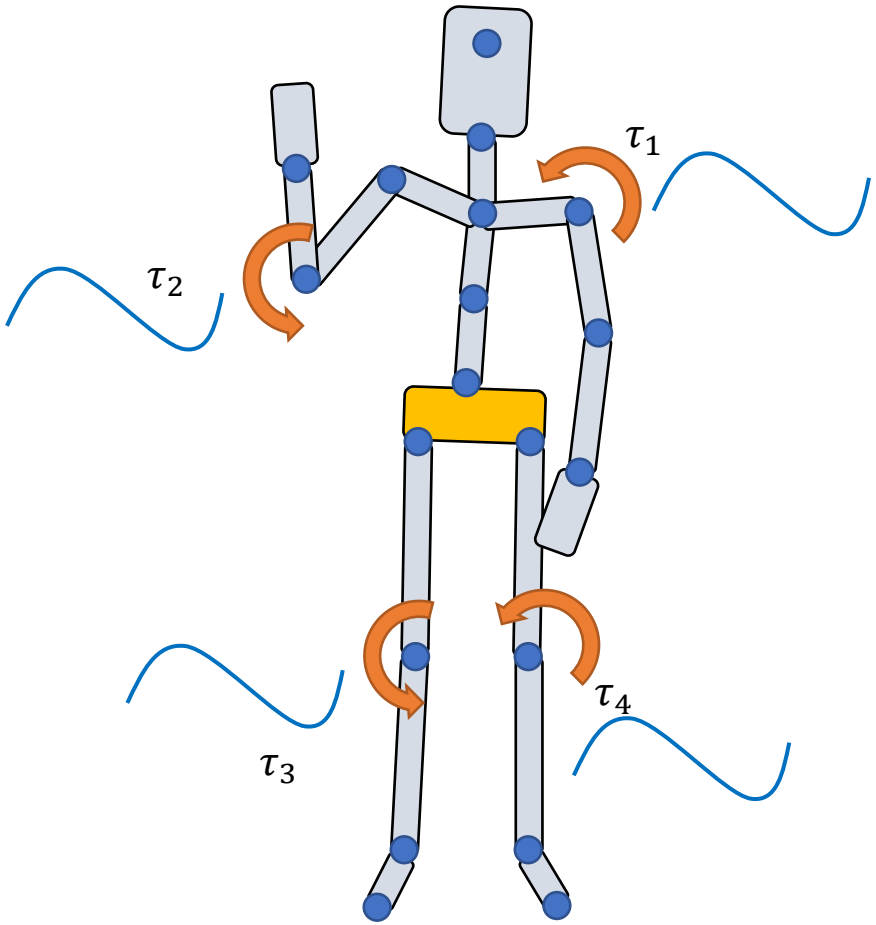
Full-body Tracking Controllers



Is PD control a feedforward control?

a feedback control?

Tracking Mocap with Joint Torques



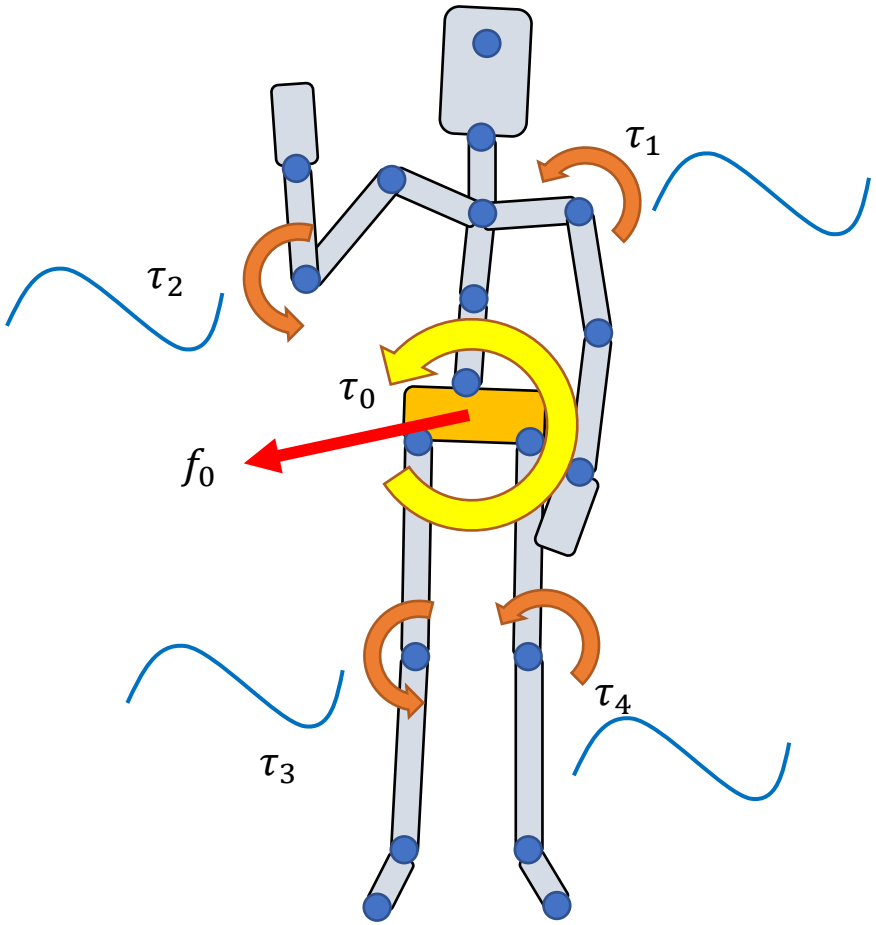
τ_j: joint torques

Apply τ_j to "child" body

Apply $-\tau_j$ to "parent" body

All forces/torques sum up to zero

Tracking Mocap with Residual Forces/Torques



τ_j: joint torques

Apply τ_j to "child" body

Apply $-\tau_j$ to "parent" body

All forces/torques sum up to zero

f_0, τ_0: root force / torque

Apply f_0 to the root body

Apply τ_0 to the root body

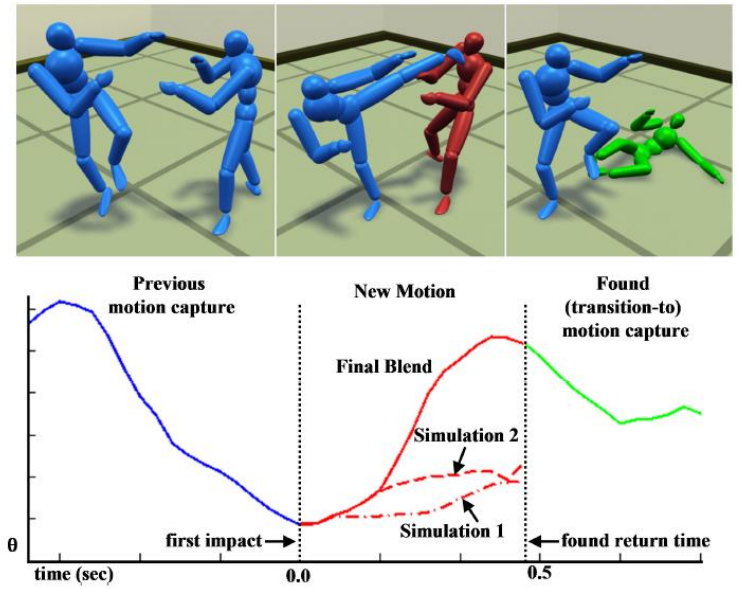
Non-zero net force/torque on the character!

Physically Plausible Animation

Party Animals

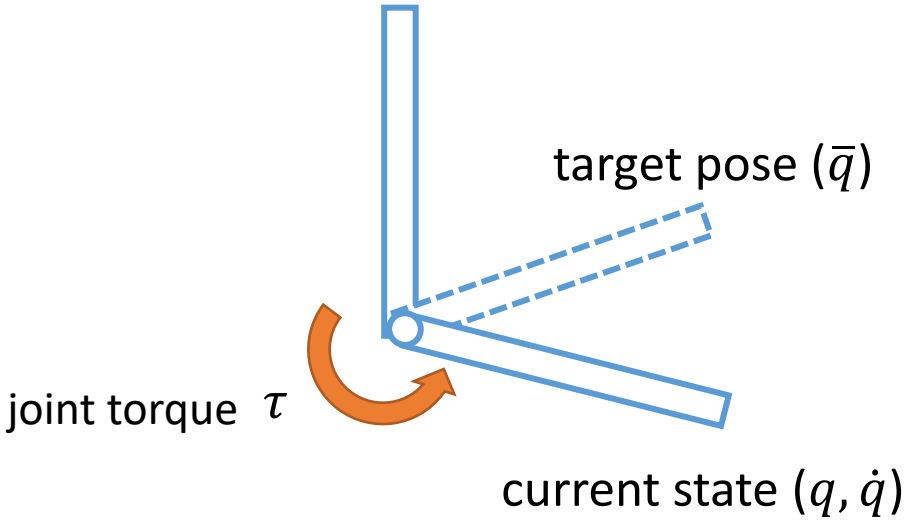
Totally Accurate Battle Simulator
https://www.youtube.com/watch?v=WFKGWfdG3bU

Mixture Simulation and Mocap



Zordan et al. 2005
Dynamic response for motion capture animation

PD Control for Characters



target pose

$$\tau = k_p(\bar{q} - q) - k_d\dot{q}$$

proportional control

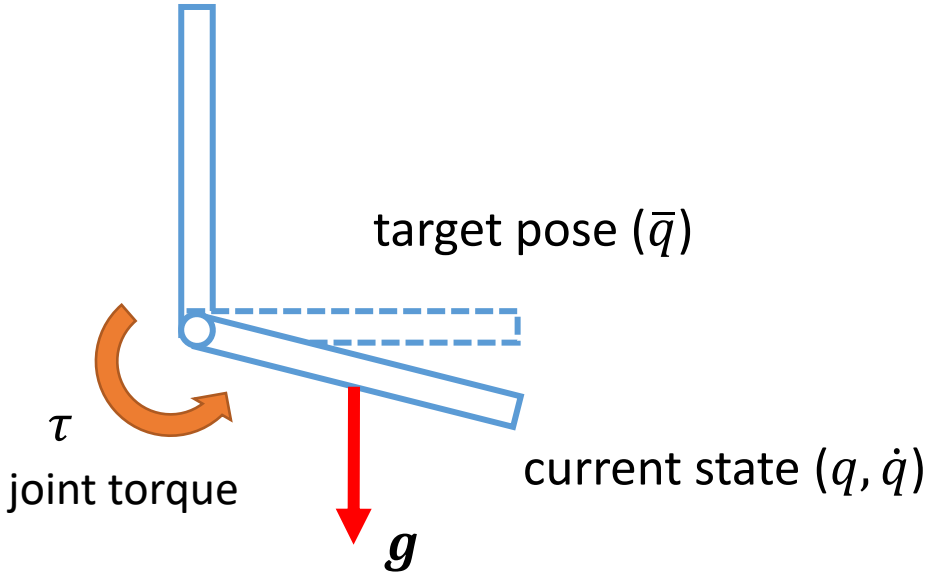
derivative control

Problems with PD Control

PD control computes torques based on errors

- Steady state error

This arm never reaches the target angle under gravity

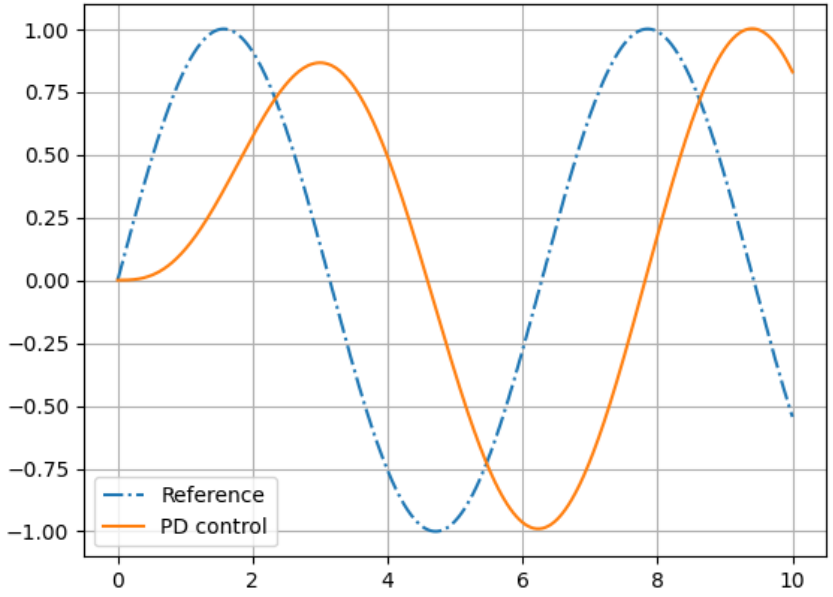


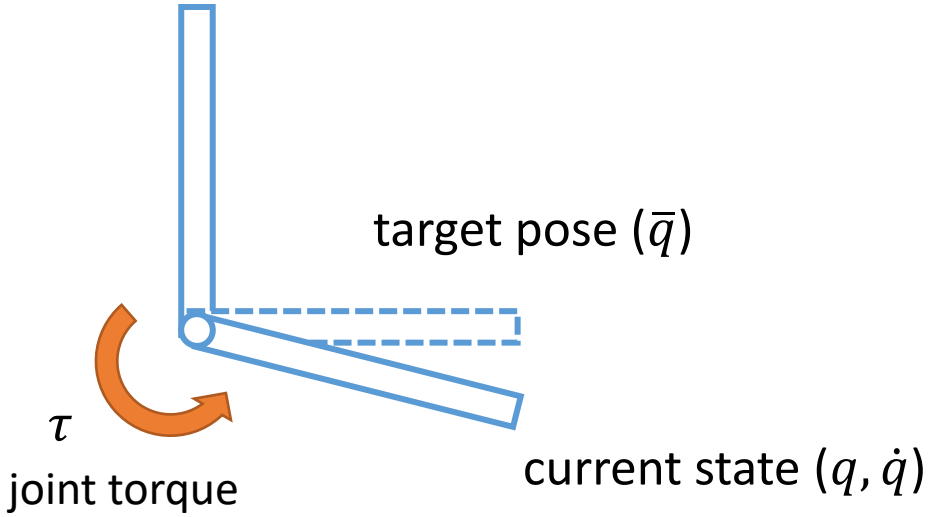
$$\tau = k_p(\bar{q} - q) - k_d\dot{q}$$

Problems with PD Control

PD control computes torques based on errors

- Steady state error
- Motion falls behind the reference





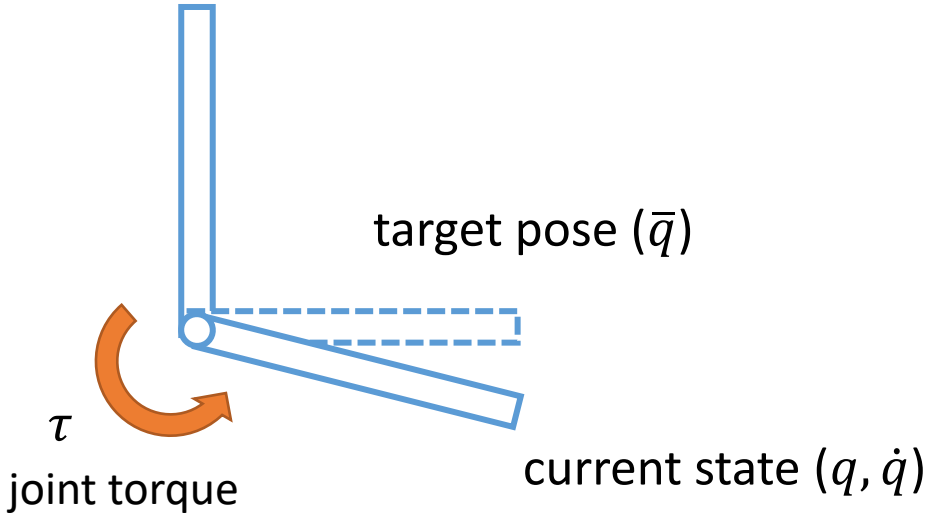
$$\tau = k_p(\bar{q} - q) - k_d\dot{q}$$

Problems with PD Control

PD control computes torques based on errors

- Steady state error
- Motion falls behind the reference

High-gain (k_p) control is more precise…



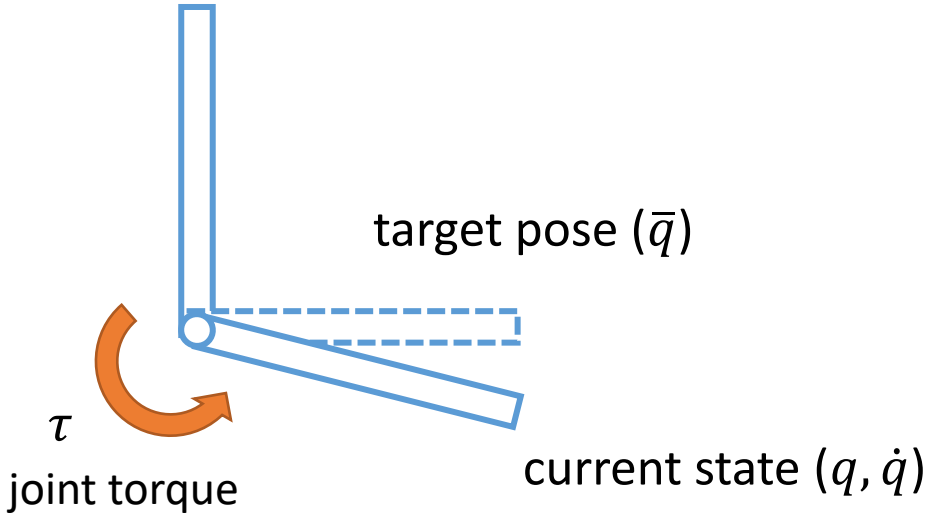
$$\tau = k_p(\bar{q} - q) - k_d\dot{q}$$

Problems with PD Control

PD control computes torques based on errors

- Steady state error
- Motion falls behind the reference

High-gain (k_p) control is more precise but less stable…



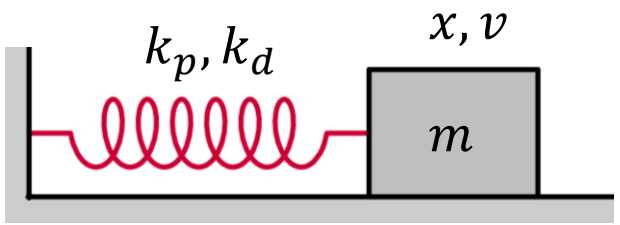
$$\tau = k_p(\bar{q} - q) - k_d\dot{q}$$

Stability of PD Control

Semi-implicit Euler Integration

$$v_{n+1} = v_n + h\frac{f}{m}$$

$$x_{n+1} = x_n + hv_{n+1}$$



$$f = -k_p x - k_d v$$

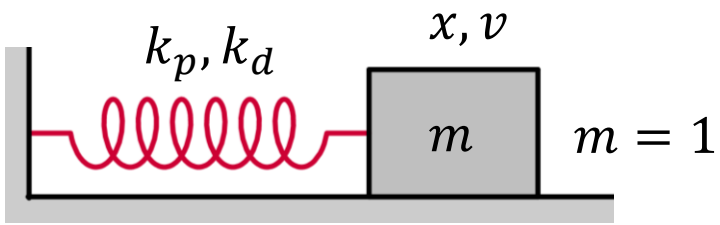
h: simulation time step

Stability of PD Control

Semi-implicit Euler Integration

$$v_{n+1} = v_n + hf$$

$$x_{n+1} = x_n + hv_{n+1}$$



$$f = -k_p x - k_d v$$

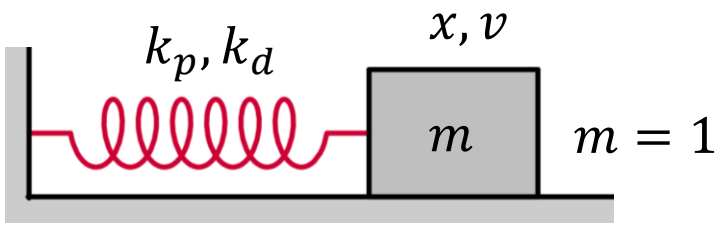
h: simulation time step

Stability of PD Control

Semi-implicit Euler Integration

$$v_{n+1} = v_n + h\left(-k_p x_n - k_d v_n\right)$$

$$x_{n+1} = x_n + h v_{n+1}$$



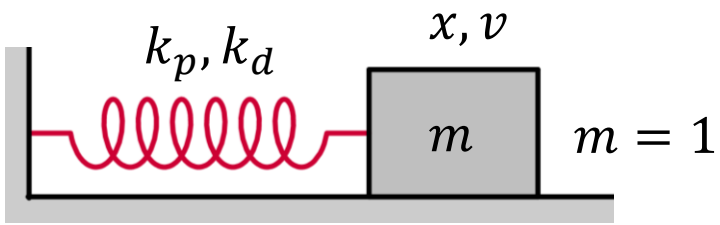
$$f = -k_p x - k_d v$$

h: simulation time step

Stability of PD Control

Semi-implicit Euler Integration

$$\begin{bmatrix} v_{n+1} \\ x_{n+1} \end{bmatrix} = \begin{bmatrix} 1 - k_d h & -k_p h \\ h(1 - k_d h) & 1 - k_p h^2 \end{bmatrix} \begin{bmatrix} v_n \\ x_n \end{bmatrix}$$



$$f = -k_p x - k_d v$$

h: simulation time step

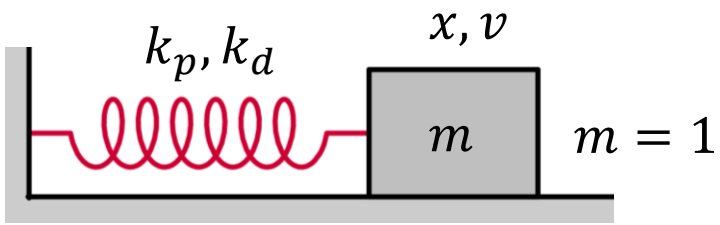
Stability of PD Control

Semi-implicit Euler Integration

$$\begin{bmatrix} v_{n+1} \\ x_{n+1} \end{bmatrix} = \begin{bmatrix} 1 - k_d h & -k_p h \\ h(1 - k_d h) & 1 - k_p h^2 \end{bmatrix} \begin{bmatrix} v_n \\ x_n \end{bmatrix}$$

$$\boldsymbol{s}_{n+1} = A\boldsymbol{s}_n$$

$$\boldsymbol{s}_{n+1} = A^n \boldsymbol{s}_1$$



$$f = -k_p x - k_d v$$

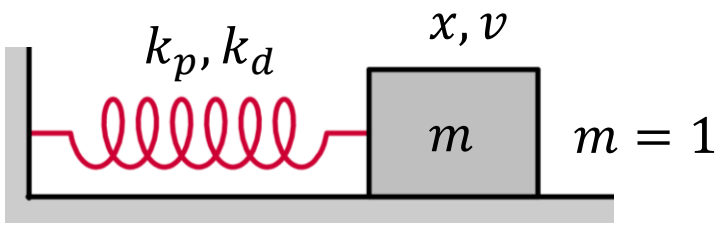
h: simulation time step

Stability of PD Control

$$\boldsymbol{s}_{n+1} = A\boldsymbol{s}_n \quad \Rightarrow \quad \boldsymbol{s}_{n+1} = A^n \boldsymbol{s}_1$$

Assume A has two eigenvalues λ_1, λ_2 and two eigen vectors $\boldsymbol{v}_1, \boldsymbol{v}_2$. Note $\lambda_i \in \mathbb{C}$

$$A = P \begin{bmatrix} \lambda_1 & \\ & \lambda_2 \end{bmatrix} P^{-1} \qquad P = \begin{bmatrix} \vdots & \vdots \\ \boldsymbol{v}_1 & \boldsymbol{v}_2 \\ \vdots & \vdots \end{bmatrix}$$



$$f = -k_p x - k_d v$$

h: simulation time step

$$A = \begin{bmatrix} 1 - k_d h & -k_p h \\ h(1 - k_d h) & 1 - k_p h^2 \end{bmatrix}$$

Stability of PD Control

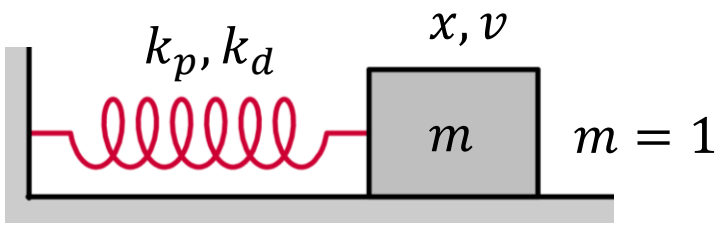
$$\boldsymbol{s}_{n+1} = A\boldsymbol{s}_n \quad \Rightarrow \quad \boldsymbol{s}_{n+1} = A^n \boldsymbol{s}_1$$

Assume A has two eigenvalues λ_1, λ_2 and two eigen vectors $\boldsymbol{v}_1, \boldsymbol{v}_2$. Note $\lambda_i \in \mathbb{C}$

$$A = P \begin{bmatrix} \lambda_1 & \\ & \lambda_2 \end{bmatrix} P^{-1} \qquad P = \begin{bmatrix} \vdots & \vdots \\ \boldsymbol{v}_1 & \boldsymbol{v}_2 \\ \vdots & \vdots \end{bmatrix}$$

$$\Downarrow$$

$$\boldsymbol{s}_{n+1} = P \begin{bmatrix} \lambda_1 & \\ & \lambda_2 \end{bmatrix} P^{-1} \boldsymbol{s}_n$$



$$f = -k_p x - k_d v$$

h: simulation time step

$$A = \begin{bmatrix} 1 - k_d h & -k_p h \\ h(1 - k_d h) & 1 - k_p h^2 \end{bmatrix}$$

Stability of PD Control

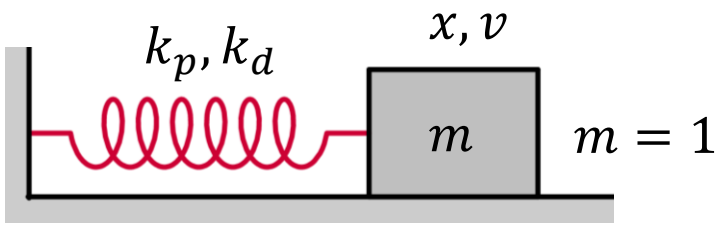
$$\boldsymbol{s}_{n+1} = A\boldsymbol{s}_n \quad \Rightarrow \quad \boldsymbol{s}_{n+1} = A^n \boldsymbol{s}_1$$

Assume A has two eigenvalues λ_1, λ_2 and two eigen vectors $\boldsymbol{v}_1, \boldsymbol{v}_2$. Note $\lambda_i \in \mathbb{C}$

$$A = P \begin{bmatrix} \lambda_1 & \\ & \lambda_2 \end{bmatrix} P^{-1} \qquad P = \begin{bmatrix} \vdots & \vdots \\ \boldsymbol{v}_1 & \boldsymbol{v}_2 \\ \vdots & \vdots \end{bmatrix}$$

$$\Downarrow$$

$$\boldsymbol{z}_{n+1} = \begin{bmatrix} \lambda_1 & \\ & \lambda_2 \end{bmatrix} \boldsymbol{z}_n \qquad \boldsymbol{z} = P^{-1}\boldsymbol{s}$$



$$f = -k_p x - k_d v$$

h: simulation time step

$$A = \begin{bmatrix} 1 - k_d h & -k_p h \\ h(1 - k_d h) & 1 - k_p h^2 \end{bmatrix}$$

Stability of PD Control

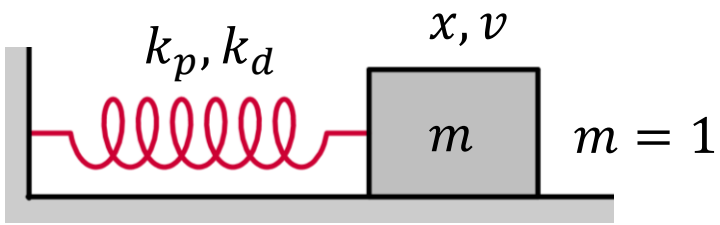
$$\boldsymbol{s}_{n+1} = A\boldsymbol{s}_n \quad \Rightarrow \quad \boldsymbol{s}_{n+1} = A^n \boldsymbol{s}_1$$

Assume A has two eigenvalues λ_1, λ_2 and two eigen vectors $\boldsymbol{v}_1, \boldsymbol{v}_2$. Note $\lambda_i \in \mathbb{C}$

$$A = P \begin{bmatrix} \lambda_1 & \\ & \lambda_2 \end{bmatrix} P^{-1} \qquad P = \begin{bmatrix} \vdots & \vdots \\ \boldsymbol{v}_1 & \boldsymbol{v}_2 \\ \vdots & \vdots \end{bmatrix}$$

$$\Downarrow$$

$$\boldsymbol{z}_{n+1} = \begin{bmatrix} \lambda_1 & \\ & \lambda_2 \end{bmatrix} \boldsymbol{z}_n \quad \Rightarrow \quad \boldsymbol{z}_{n+1} = \begin{bmatrix} \lambda_1 & \\ & \lambda_2 \end{bmatrix}^n \boldsymbol{z}_1$$



$$f = -k_p x - k_d v$$

h: simulation time step

$$A = \begin{bmatrix} 1 - k_d h & -k_p h \\ h(1 - k_d h) & 1 - k_p h^2 \end{bmatrix}$$

Stability of PD Control

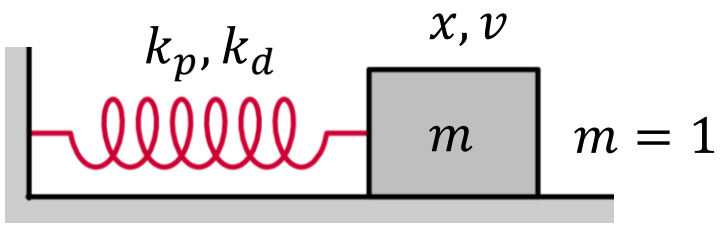
$$\boldsymbol{s}_{n+1} = A\boldsymbol{s}_n \quad \Rightarrow \quad \boldsymbol{s}_{n+1} = A^n \boldsymbol{s}_1$$

Assume A has two eigenvalues λ_1, λ_2 and two eigen vectors $\boldsymbol{v}_1, \boldsymbol{v}_2$. Note $\lambda_i \in \mathbb{C}$

$$A = P \begin{bmatrix} \lambda_1 & \\ & \lambda_2 \end{bmatrix} P^{-1} \qquad P = \begin{bmatrix} \vdots & \vdots \\ \boldsymbol{v}_1 & \boldsymbol{v}_2 \\ \vdots & \vdots \end{bmatrix}$$

$$\Downarrow$$

$$\boldsymbol{z}_{n+1} = \begin{bmatrix} \lambda_1 & \\ & \lambda_2 \end{bmatrix} \boldsymbol{z}_n \quad \Rightarrow \quad \boldsymbol{z}_{n+1} = \begin{bmatrix} \lambda_1^n & \\ & \lambda_2^n \end{bmatrix} \boldsymbol{z}_1$$



$$f = -k_p x - k_d v$$

h: simulation time step

$$A = \begin{bmatrix} 1 - k_d h & -k_p h \\ h(1 - k_d h) & 1 - k_p h^2 \end{bmatrix}$$

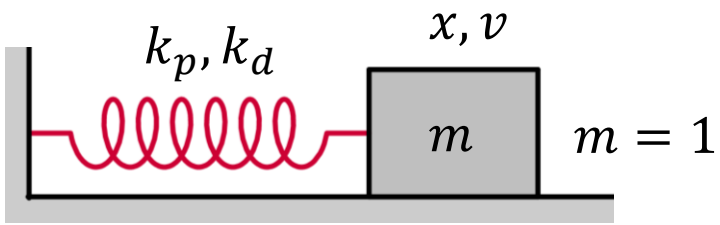
Stability of PD Control

$$\boldsymbol{z}_{n+1} = \begin{bmatrix} \lambda_1 & \\ & \lambda_2 \end{bmatrix} \boldsymbol{z}_n \Rightarrow \boldsymbol{z}_{n+1} = \begin{bmatrix} \lambda_1^n & \\ & \lambda_2^n \end{bmatrix} \boldsymbol{z}_1$$

$\lambda_1, \lambda_2 \in \mathbb{C}$ are eigenvalues of A

if $|\lambda_1| > 1$ $\Rightarrow$ $\lim_{n\to\infty} \|\boldsymbol{z}_n\| \to \infty$

The system is unstable!



$$f = -k_p x - k_d v$$

h: simulation time step

$$A = \begin{bmatrix} 1 - k_d h & -k_p h \\ h(1 - k_d h) & 1 - k_p h^2 \end{bmatrix}$$

Stability of PD Control

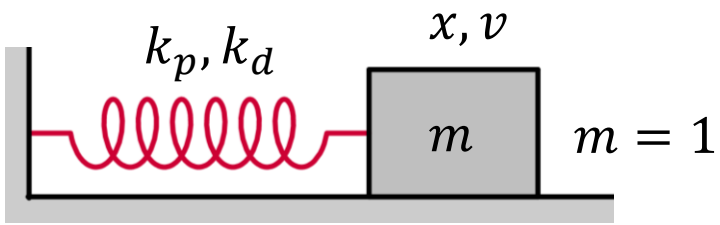
$$\boldsymbol{z}_{n+1} = \begin{bmatrix} \lambda_1 & \\ & \lambda_2 \end{bmatrix} \boldsymbol{z}_n \Rightarrow \boldsymbol{z}_{n+1} = \begin{bmatrix} \lambda_1^n & \\ & \lambda_2^n \end{bmatrix} \boldsymbol{z}_1$$

$\lambda_1, \lambda_2 \in \mathbb{C}$ are eigenvalues of A

if $|\lambda_1| > 1 \Rightarrow \lim_{n\to\infty} \|\boldsymbol{z}_n\| \to \infty$

The system is unstable!

Condition of stability: $|\lambda_i| \leq 1$ for all λ_i

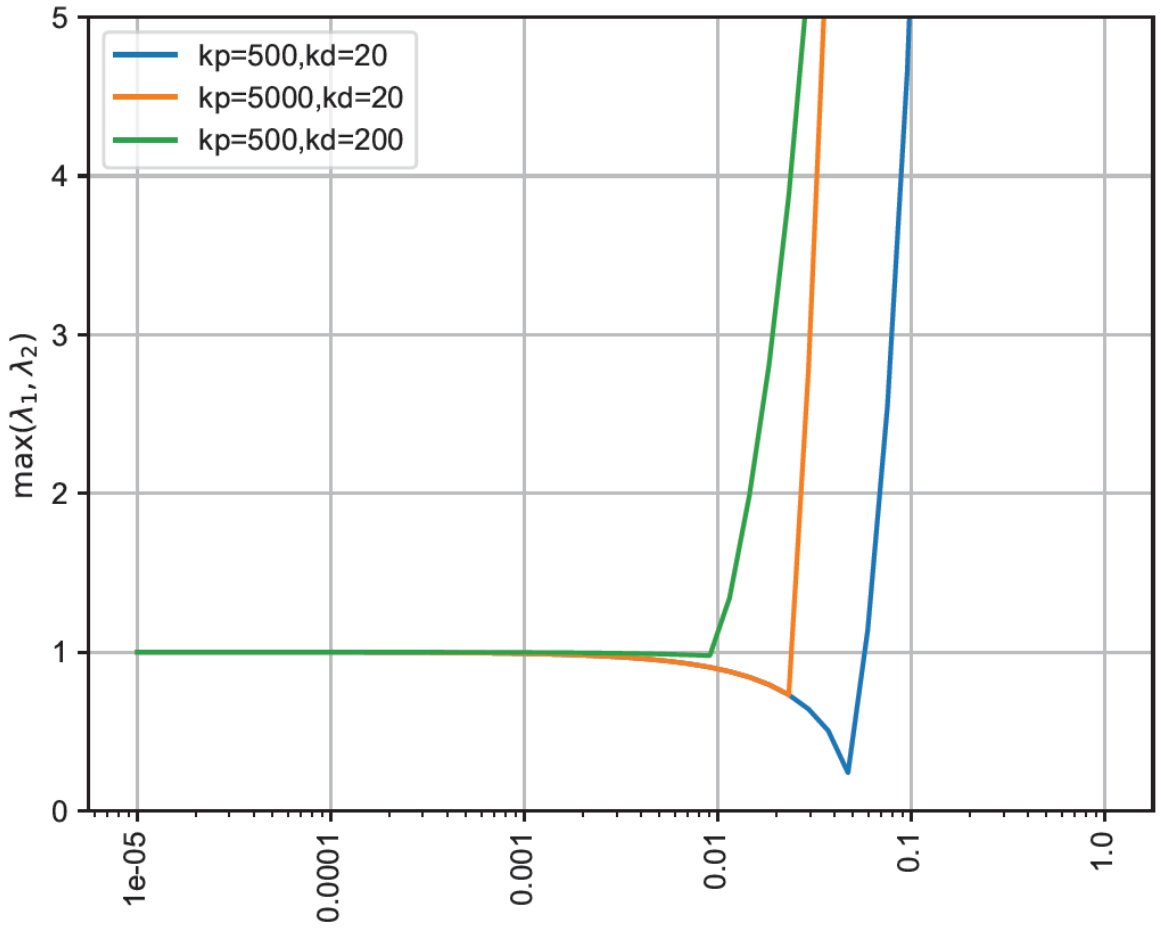


$$f = -k_p x - k_d v$$

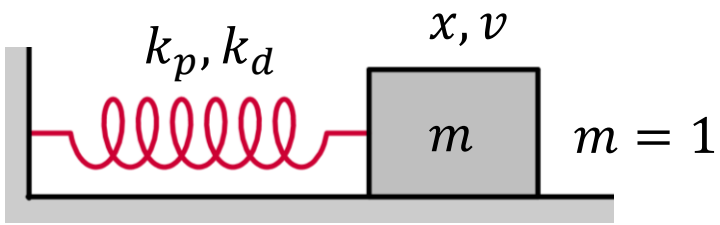
h: simulation time step

$$A = \begin{bmatrix} 1 - k_d h & -k_p h \\ h(1 - k_d h) & 1 - k_p h^2 \end{bmatrix}$$

Stability of PD Control



h: simulation time step

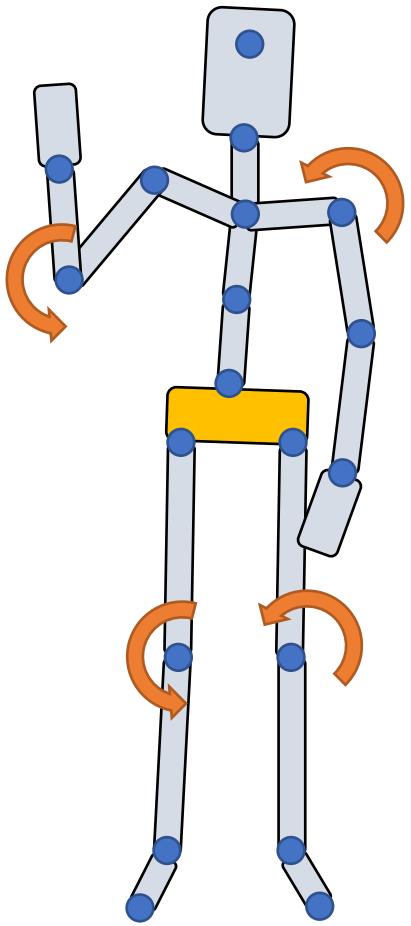


$$f = -k_p x - k_d v$$

h: simulation time step

$$A = \begin{bmatrix} 1 - k_d h & -k_p h \\ h(1 - k_d h) & 1 - k_p h^2 \end{bmatrix}$$

PD Control for Characters



$$\tau = k_p(\bar{q} - q) - k_d\dot{q}$$

- Determining gain and damping coefficients can be difficult…
  - A typical setting $k_p = 200$, $k_d = 20$ for a 50kg character
  - Light body requires smaller gains
  - Dynamic motions need larger gains
- High-gain/high-damping control can be unstable, so small times is necessary
  - $h = 0.5\sim1\text{ms}$ is often used, or $1000\sim2000\text{Hz}$
  - Higher gain/damping requires smaller time step

A More Stable PD Control

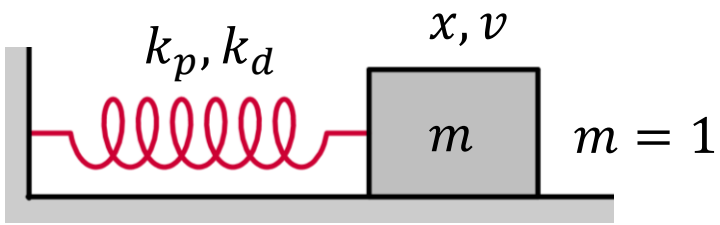
Semi-implicit Euler Integration

$$v_{n+1} = v_n + h\left(-k_p x_n - k_d v_n\right)$$

$$x_{n+1} = x_n + h v_{n+1}$$

$$v_{n+1} = v_n + h\left(-k_p x_n - k_d \boldsymbol{v_{n+1}}\right)$$

$$x_{n+1} = x_n + h v_{n+1}$$



$$f = -k_p x - k_d v$$

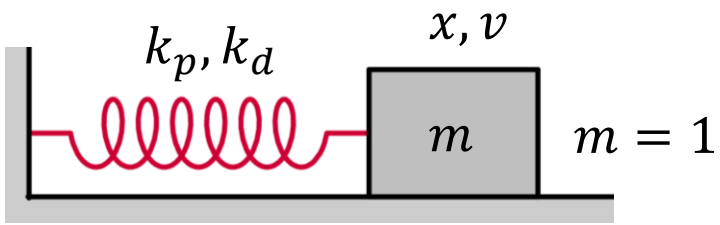
h: simulation time step

A More Stable PD Control

$$v_{n+1} = v_n + h\left(-k_p x_n - k_d \boldsymbol{v_{n+1}}\right)$$

$$x_{n+1} = x_n + h v_{n+1}$$

$$\begin{bmatrix} v_{n+1} \\ x_{n+1} \end{bmatrix} = \frac{1}{1 + hk_d} \begin{bmatrix} 1 & -k_p h \\ h & 1 + k_d h - k_p h^2 \end{bmatrix} \begin{bmatrix} v_n \\ x_n \end{bmatrix}$$

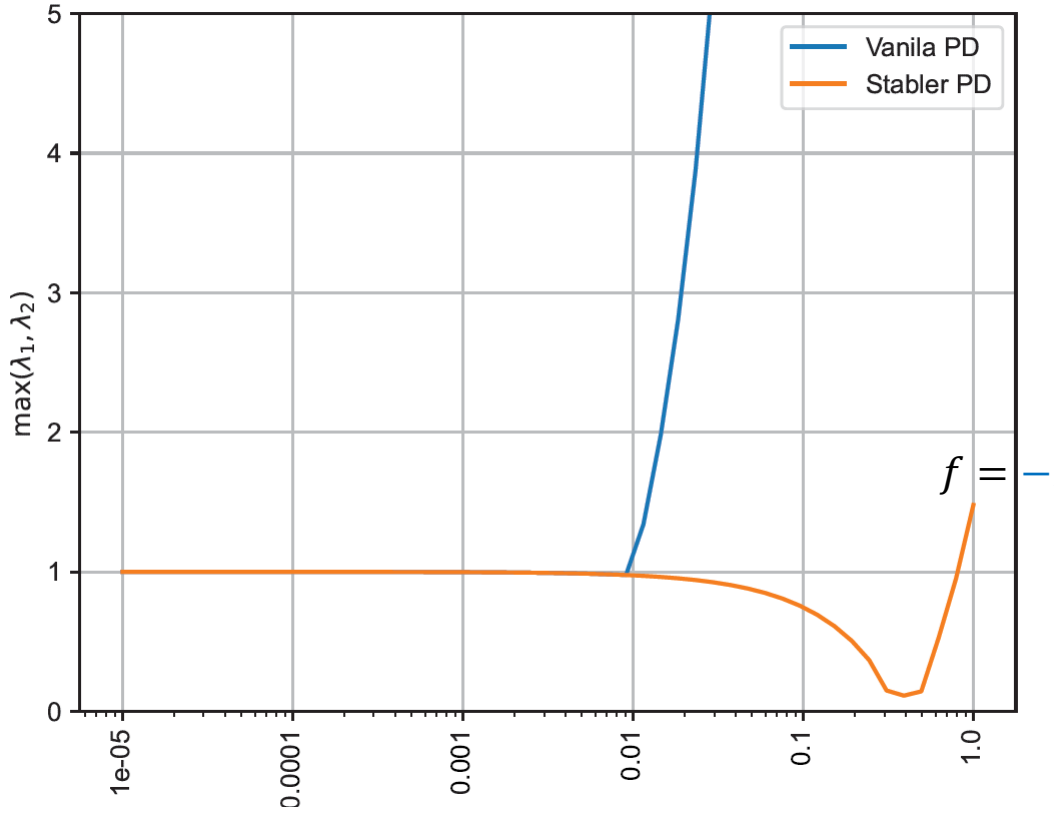


$$f = -k_p x - k_d v$$

h: simulation time step

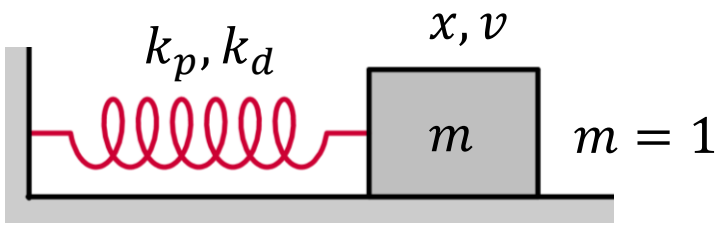
A More Stable PD Control

$$f = -k_p x_n - k_d v_{\boldsymbol{n}}$$



$$f = -k_p x_n - k_d v_{\boldsymbol{n+1}}$$

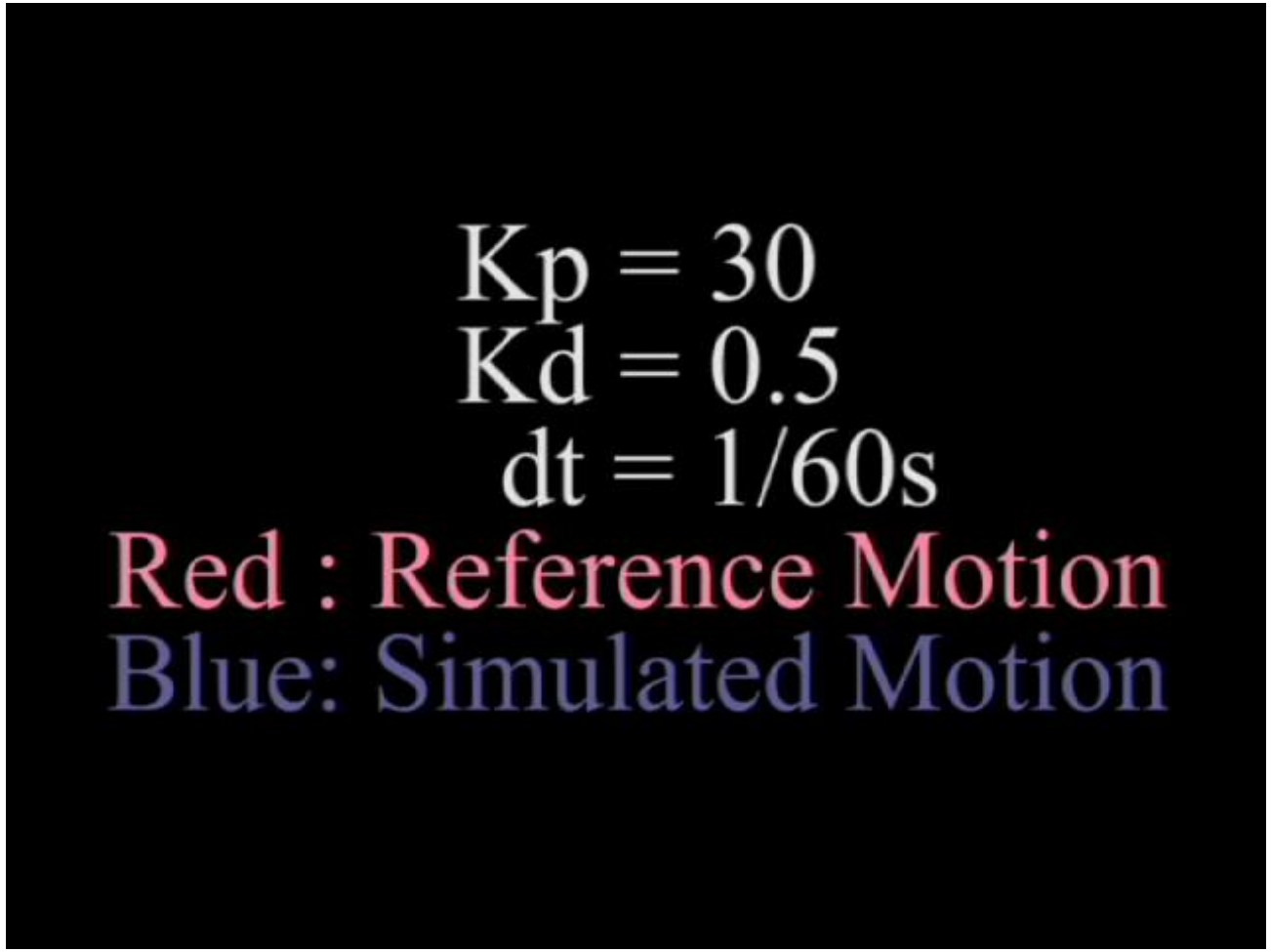
h: simulation time step



$$f = -k_p x - k_d v$$

h: simulation time step

Stable PD Control

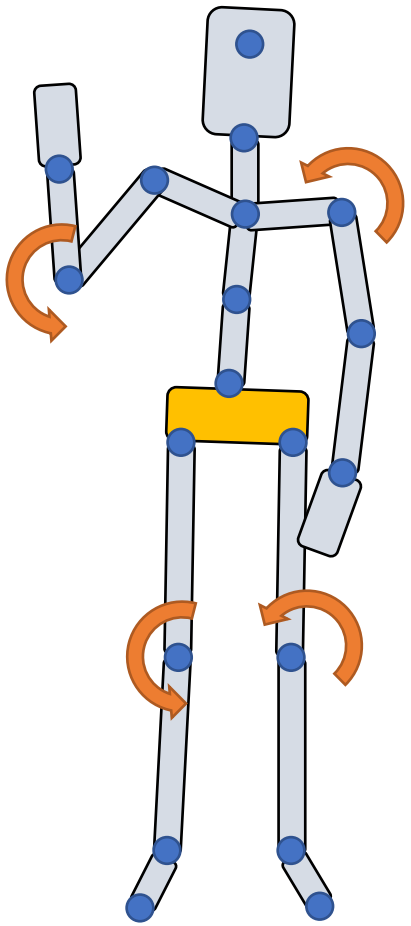


$$\tau_{\mathbf{int}} = -\mathbf{K}_p(\mathbf{q}^n + \dot{\mathbf{q}}^n \Delta t - \bar{\mathbf{q}}^{n+1}) - \mathbf{K}_d(\dot{\mathbf{q}}^n + \ddot{\mathbf{q}}^n \Delta t)$$

Stable Proportional-Derivative Controllers

Jie Tan* Karen Liu† Greg Turk‡
Georgia Institute of Technology

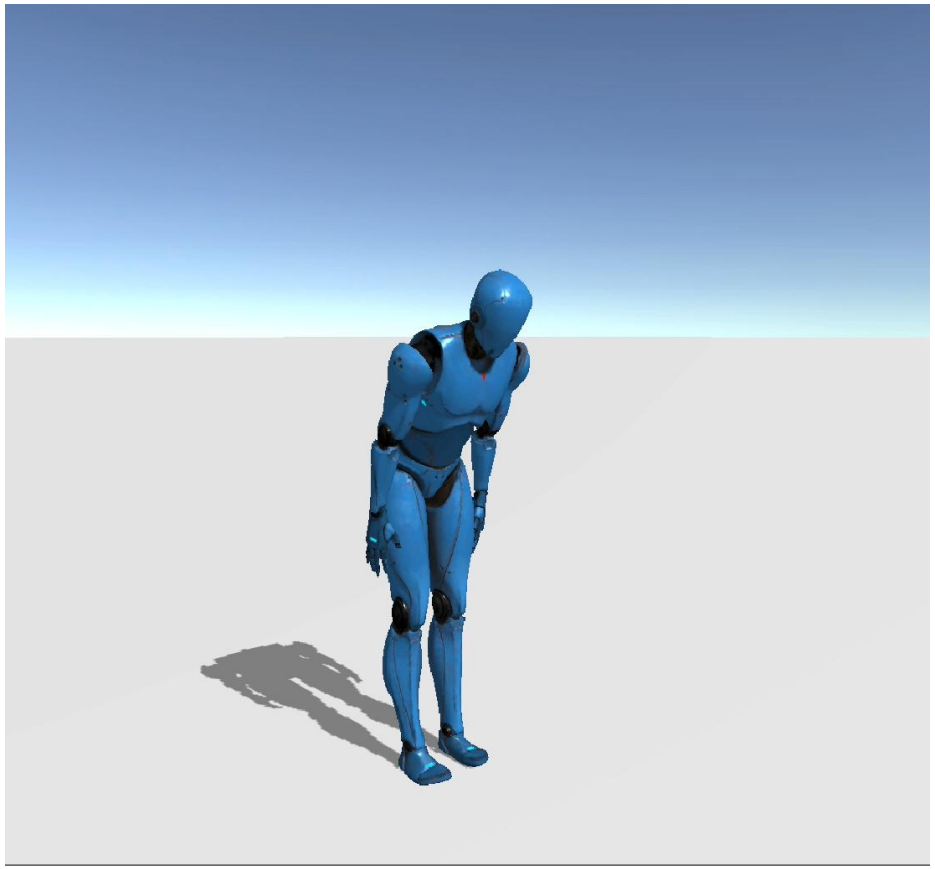
PD Control for Characters



$$\tau = k_p(\bar{q} - q) - k_d\dot{q}$$

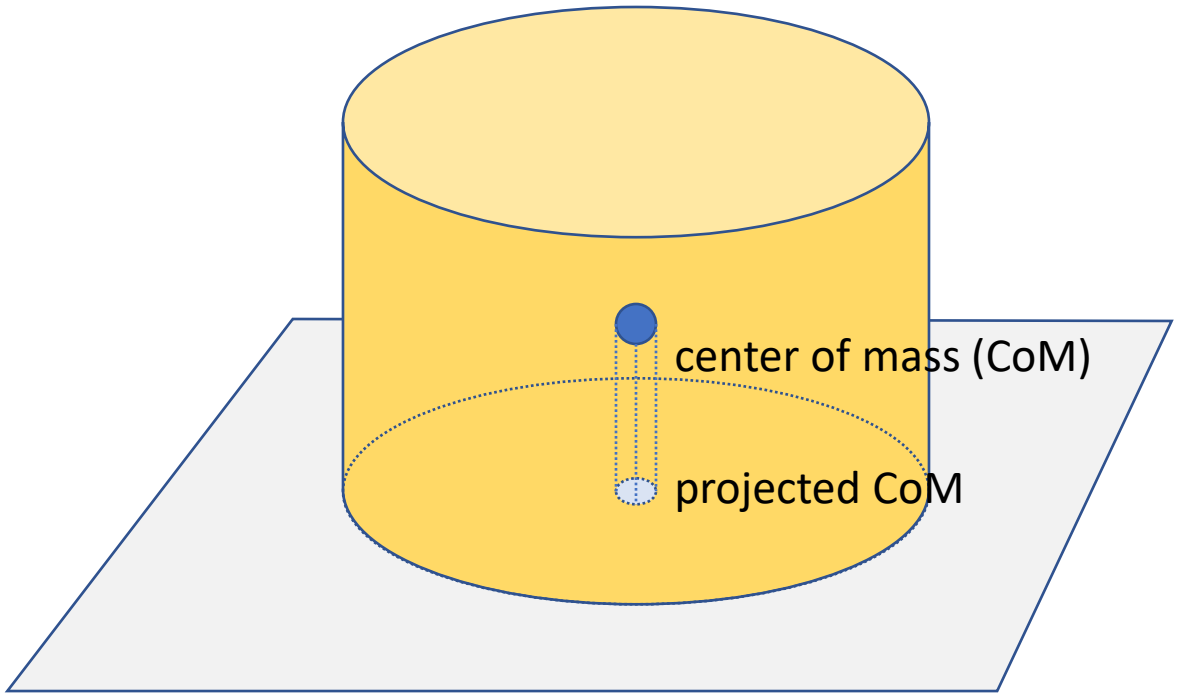
- Determining gain and damping coefficients can be difficult…
  - A typical setting $k_p = 200$, $k_d = 20$ for a 50kg character
  - Light body requires smaller gains
  - Dynamic motions need larger gains
- High-gain/high-damping control can be unstable, so small times is necessary
  - $h = 0.5\sim1\text{ms}$ is often used, or $1000\sim2000\text{Hz}$
  - $h = 1/120\text{s}\sim1/60\text{s}$, or 120Hz/60Hz with Stable PD
  - Higher gain/damping requires smaller time step

Static Balance



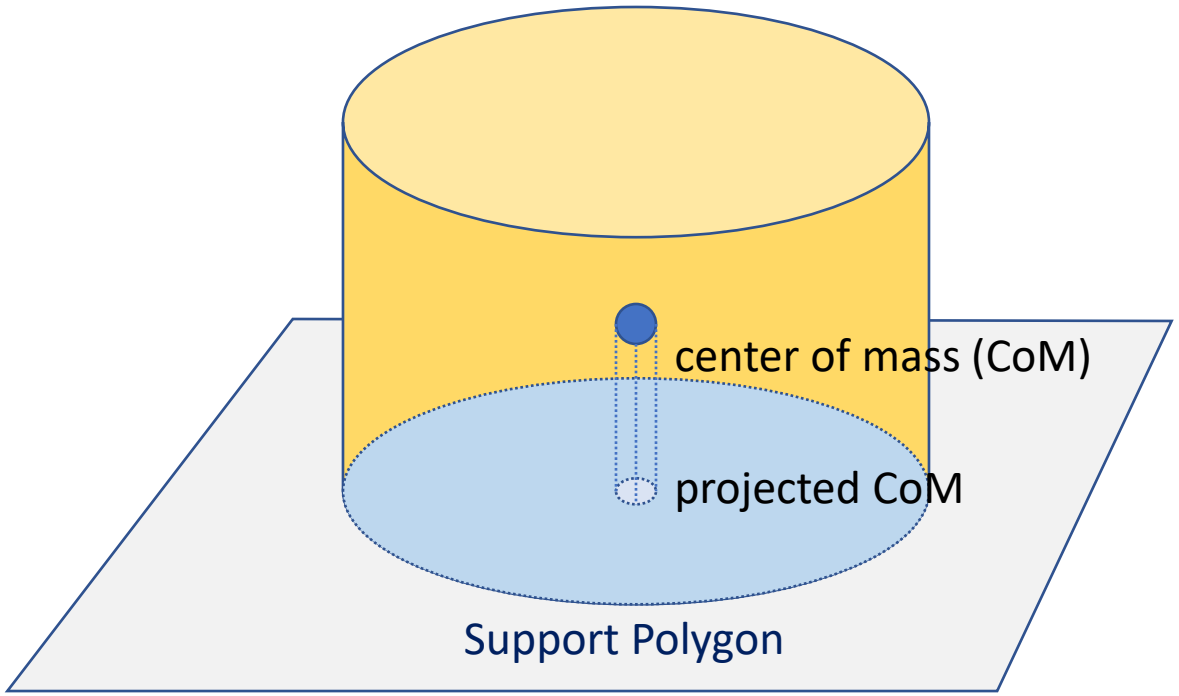
Static Balance

What is balance?



Static Balance

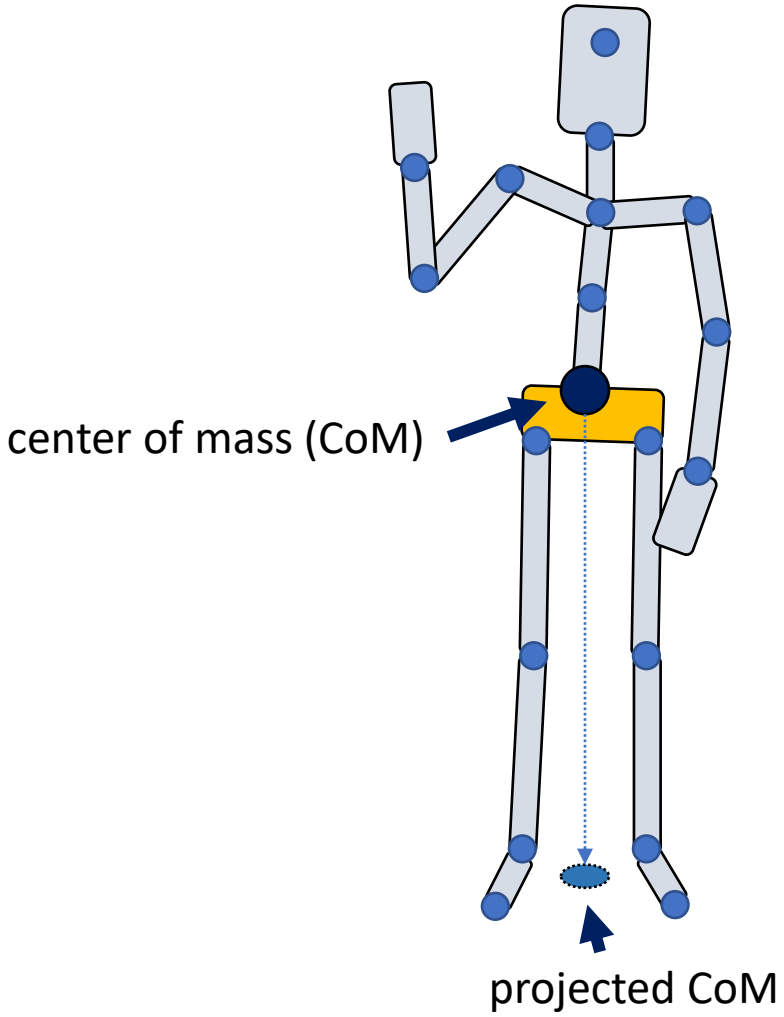
What is balance?



Static Balance

What is balance?

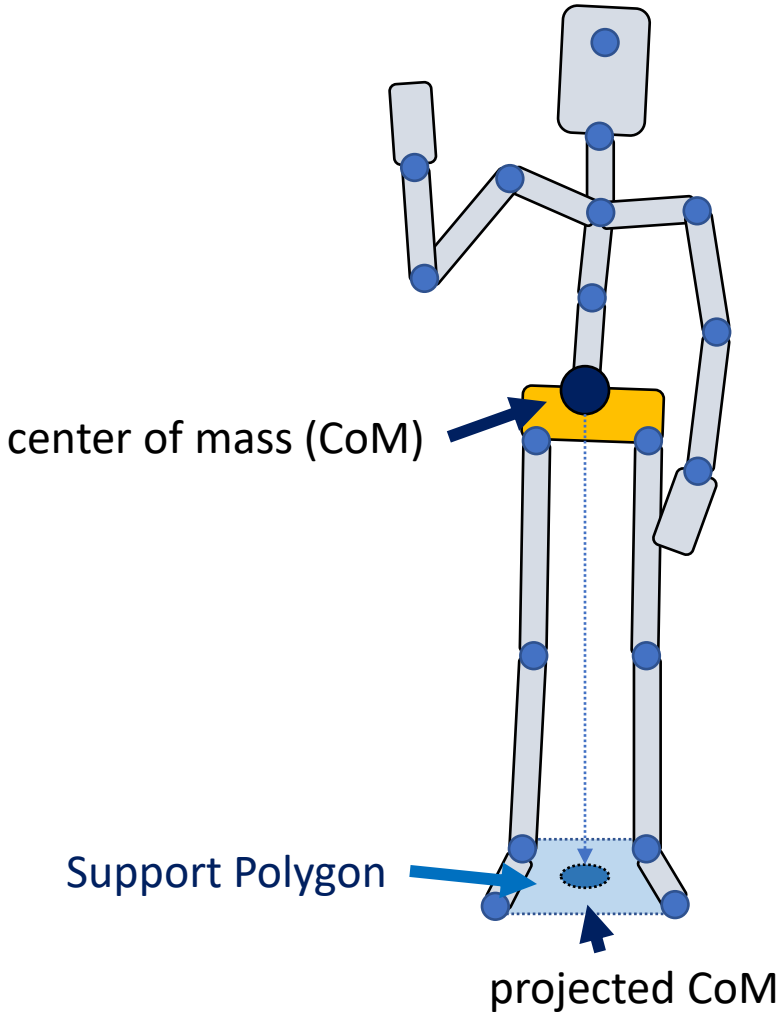
静平衡条件



support：两个脚构成的凸包

Static Balance

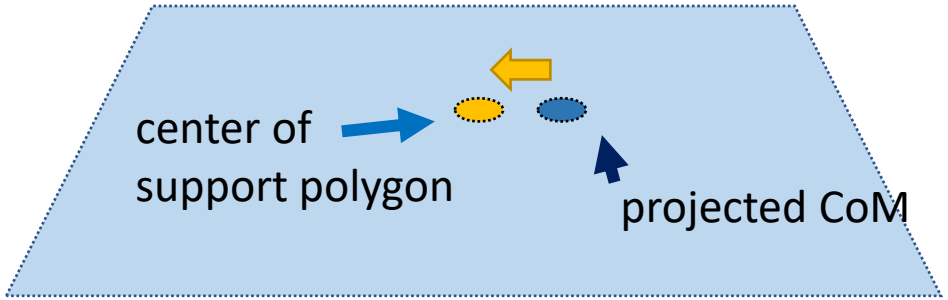
What is balance?

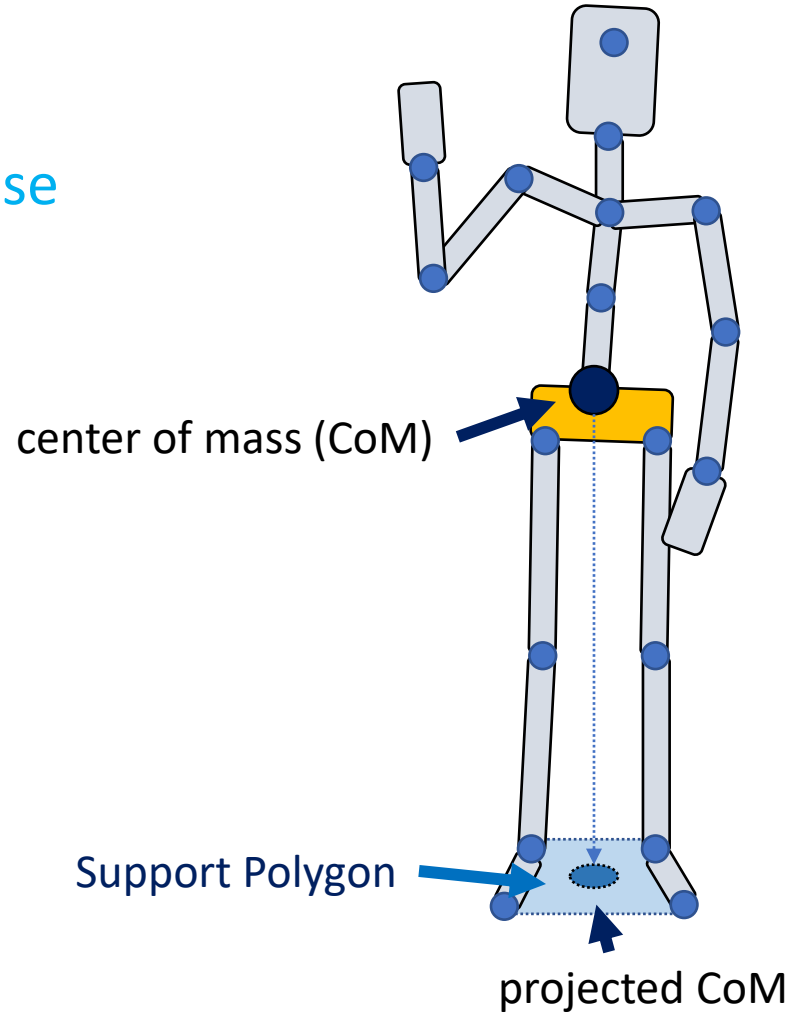


Static Balance

A simple strategy to maintain balance:

- Keep projected CoM close to the center of support polygon while tracking a standing pose





Static Balance

A simple strategy to maintain balance:

- Keep projected CoM close to the center of support polygon while tracking a standing pose
- Use PD control to compute feedback torque

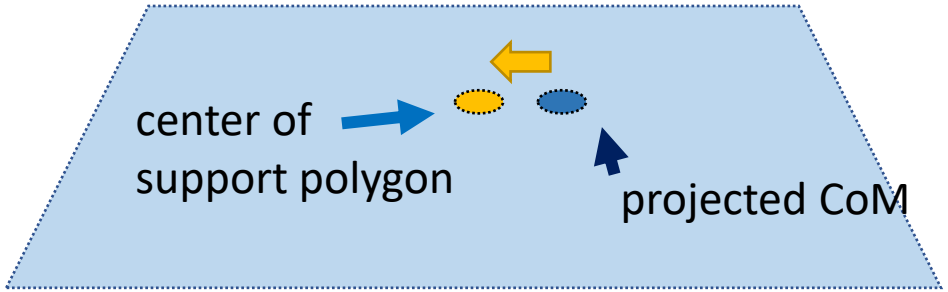
$$\tau = k_p(\bar{\boldsymbol{c}} - \boldsymbol{c}) - k_d\dot{\boldsymbol{c}}$$

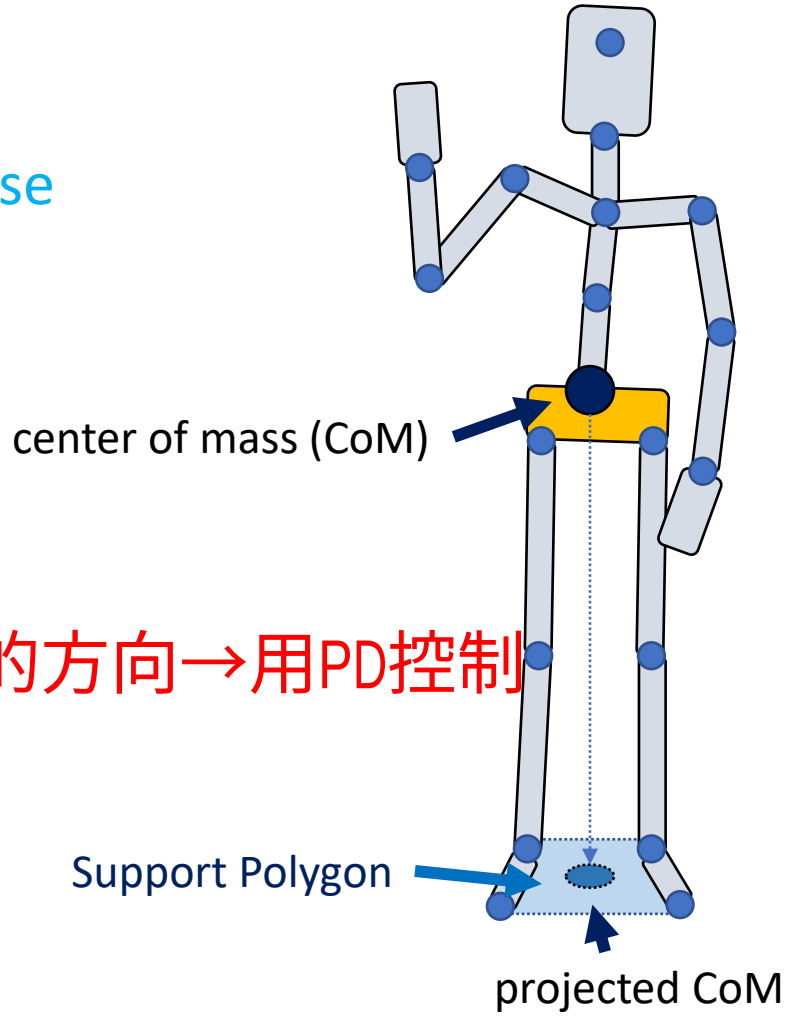
CoM velocity

desired CoM position (e.g. center of support polygon)

CoM position of the character

指定一个关节（如脚腕），和质心移动是相反的方向→用PD控制





Static Balance

A simple strategy to maintain balance:

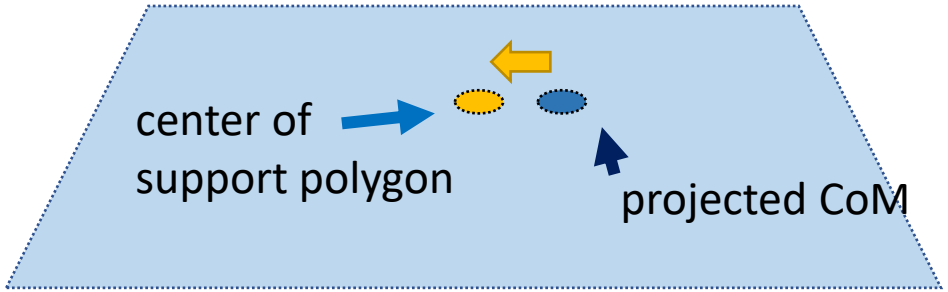
- Keep projected CoM close to the center of support polygon while tracking a standing pose
- Use PD control to compute feedback torque

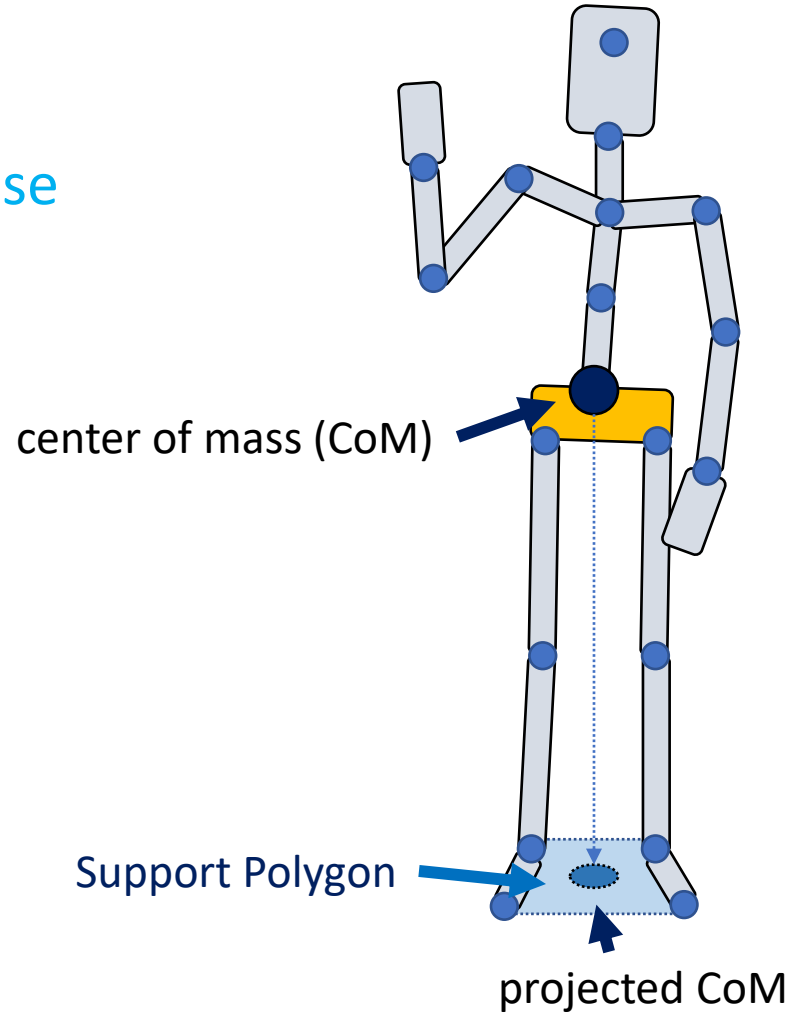
$$\tau = k_p(\bar{\boldsymbol{c}} - \boldsymbol{c}) - k_d\dot{\boldsymbol{c}}$$

gain coefficient

damping coefficient

need to tune manually…





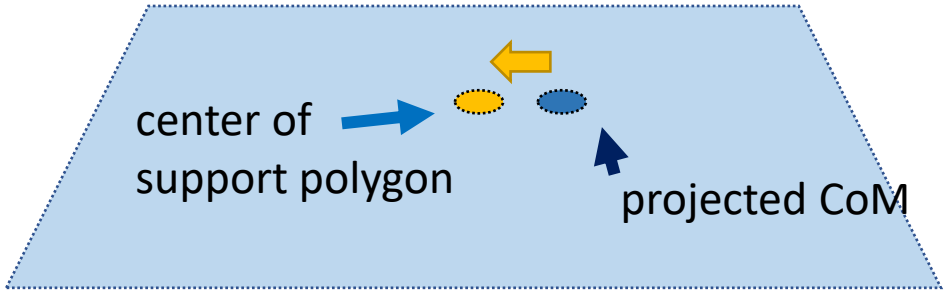
Static Balance

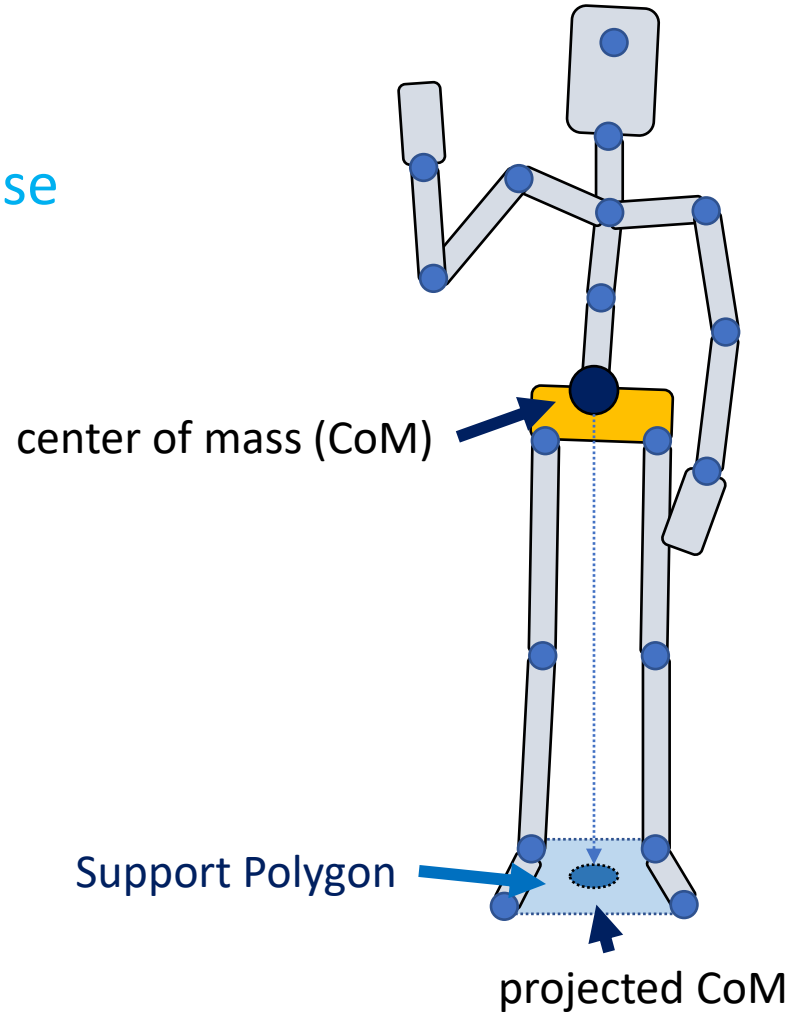
A simple strategy to maintain balance:

- Keep projected CoM close to the center of support polygon while tracking a standing pose
- Use PD control to compute feedback torque

$$\tau = k_p(\bar{\boldsymbol{c}} - \boldsymbol{c}) - k_d\dot{\boldsymbol{c}}$$

- Apply the feedback torque at ankles (ankle strategy) or hips (hip strategy)



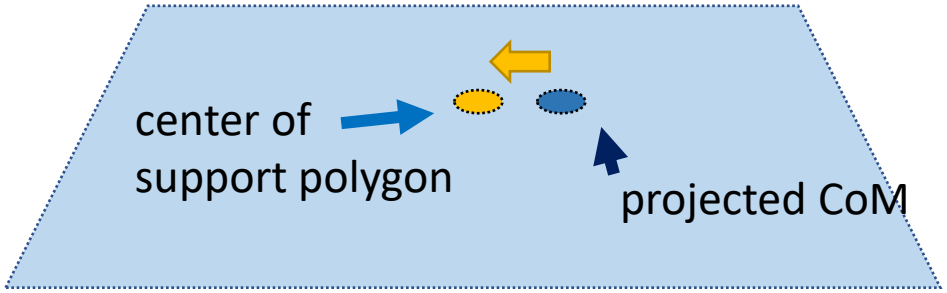


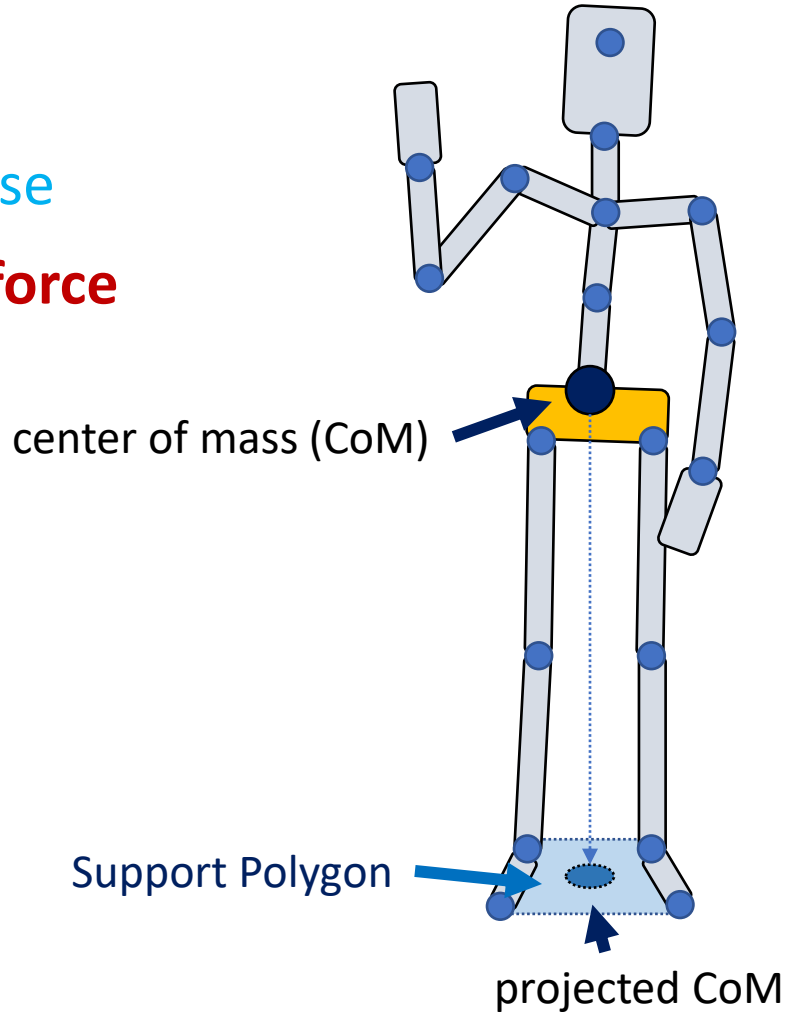
Static Balance

A simple strategy to maintain balance:

- Keep projected CoM close to the center of support polygon while tracking a standing pose
- Use PD control to compute feedback **virtual force**

$$f = k_p(\bar{\boldsymbol{c}} - \boldsymbol{c}) - k_d\dot{\boldsymbol{c}}$$





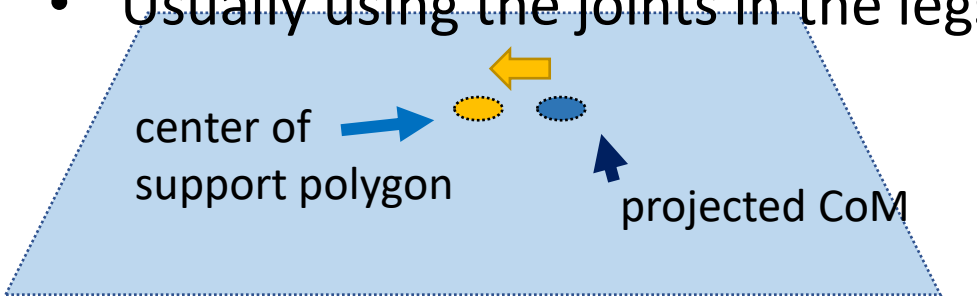
Static Balance

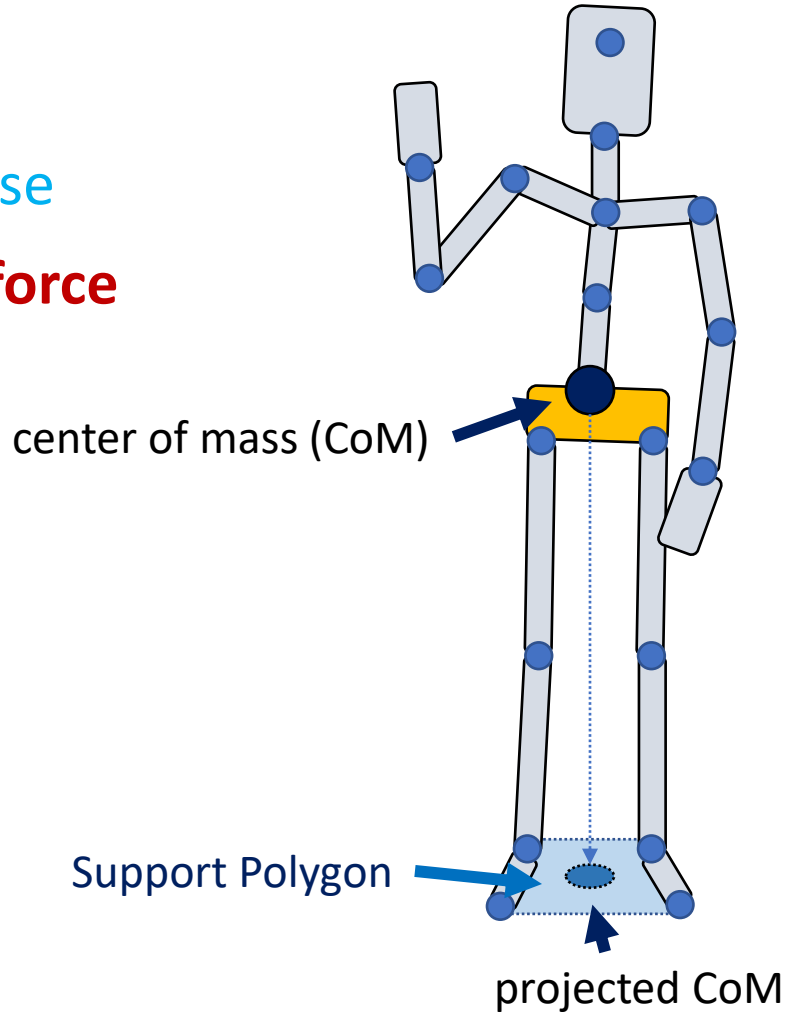
A simple strategy to maintain balance:

- Keep projected CoM close to the center of support polygon while tracking a standing pose
- Use PD control to compute feedback **virtual force**

$$f = k_p(\bar{\boldsymbol{c}} - \boldsymbol{c}) - k_d\dot{\boldsymbol{c}}$$

- Assuming f is applied to the CoM, compute necessary joint torques using Jacobian transpose control to achieve it
  - Usually using the joints in the legs

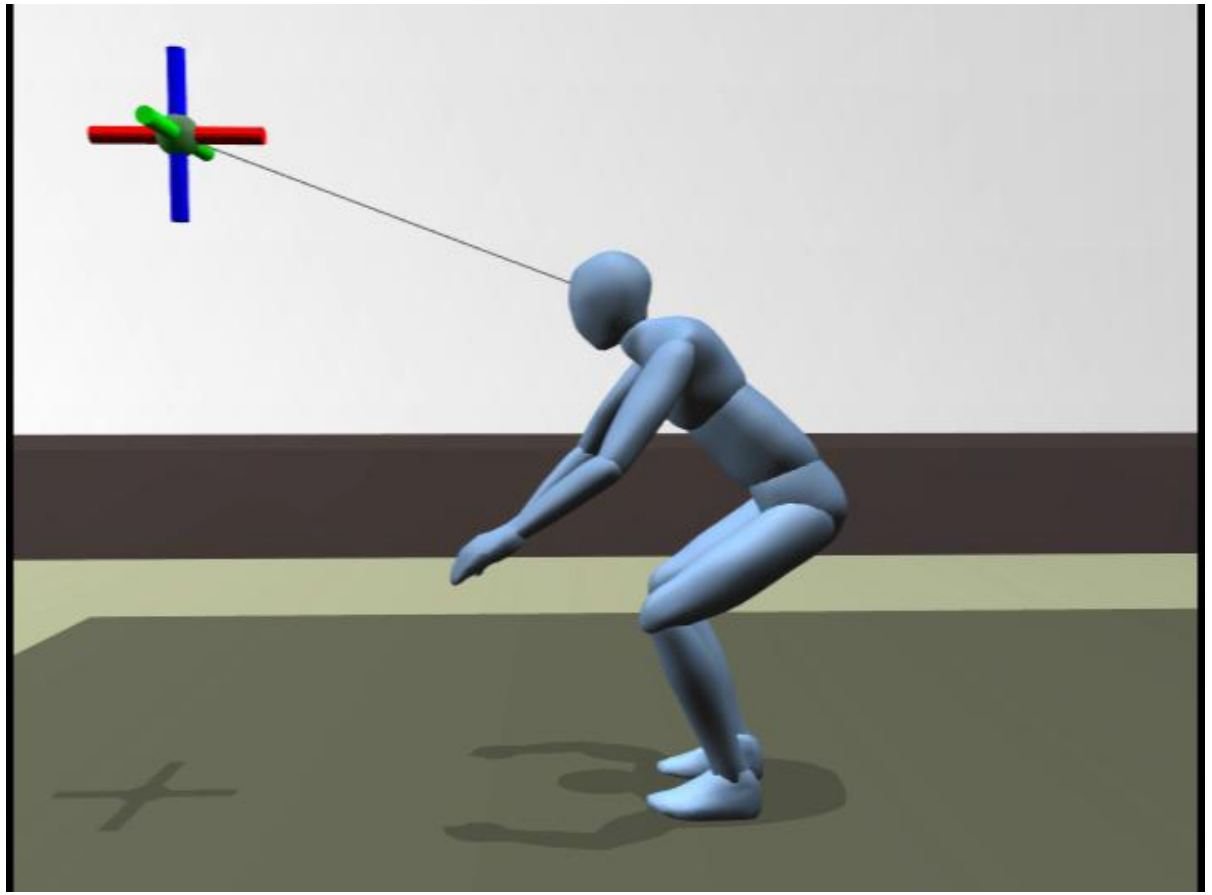




Static Balance

A fancier strategy:

- Mocap tracking as an objective function
- Controlling both the CoM position/momentum and the angular momentum
- Solve a one-step optimization problem to compute joint torques



[Macchietto et al. 2009 - Momentum Control for Balance]

Outline

- Simulating & Actuating Characters
  - Joint torques
- PD (Proportional-Derivative) control
  - Stable PD control
- Feedforward Motion Control
  - Residual force
- Feedback Motion Control
  - Static balance

Questions?